

Lecture #01: Relational Model & Algebra

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Databases

A *database* is an organized collection of interrelated data that models some aspect of the real world (e.g., modeling the students in a class or a digital music store). Databases are the core component of most computer applications. Computer Science is essentially taking in some input, performing some operations, and producing an output, which can be seen as a database at a high level as well.

People often confuse “databases” with “database management systems” (e.g., MySQL, Oracle, MongoDB, Snowflake). A database management system (DBMS) is the software that manages a database. Among many things, a DBMS is responsible for inserting, deleting, and retrieving data from a database.

2 Flat File Strawman

Consider that we want to store data for a music store like Spotify. We want to hold information about the artists and which albums those artists have released. This database has two entities: an Artists entity and an Albums entity.

We choose to store the database’s records as comma-separated value (csv) files that the DBMS manages, and each entity is stored in its own file (an `artists.csv` and `albums.csv`).

In this example, artists each have a name, year, and country attribute. Albums have name, artist, and year attributes.

The following is an example CSV file for information about artists with the schema (name, year, country):

```
"Wu-Tang Clan", 1992, "USA"  
"Notorious BIG", 1992, "USA"  
"GZA", 1990, "USA"
```

The application has to parse these files each time it wants to read or update any records. As an example, if we want to get the year that GZA went solo, we will need some program like the following to open the file, read it, and then parse each line to check for a match in the artist name.

```
for line in file.readlines():  
    record = parse(line)  
    if record[0] == "GZA" :  
        print(int(record[1]))
```

Issues with Flat Files

There are several problems with using flat files as our database. Here are some questions to consider:

- **Data Integrity** How do we ensure that the artist is the same for each album entry? What if somebody overwrites the album year with an invalid string? What if there are multiple artists on an album? What happens if we delete an artist that has albums?

- **Implementation** How do you find a particular record? What if we now want to create a new application that uses the same database? What if that application is running on a different machine? What if two threads try to write to the same file at the same time?
- **Durability** What if the machine crashes while our program is updating a record? What if we want to replicate the database on multiple machines for high availability? Some database systems will sacrifice safety and durability by default to make database operations appear faster.

3 Database Management System

A **DBMS** is software that allows applications to store and analyze information in a database.

A general-purpose DBMS is designed to allow the definition, creation, querying, update, and administration of databases in accordance with some *data model*.

A **data model** is a collection of concepts for describing the data in database. Some examples include:

- **Relational** (most common)
- **NoSQL** (key/value, document, graph)
- **Array / Matrix / Vector** (for machine learning)

A **schema** is a description of a particular collection of data using a given data model. This defines the structure of data for a data model. Otherwise, you have random bits with no meaning.

Common Data Models

- Relational (Most DBMSs)
- Key/Value (Simple Apps/Caching)
- Graph (NoSQL)
- Document/XML/Object (NoSQL)
- Wide-Column/Column-family (NoSQL)
- Array / Matrix / Vector (Machine Learning/Science)
- Hierarchical (Obsolete/Legacy/Rare)
- Network (Obsolete/Legacy/Rare)
- Multi-Value (Obsolete/Legacy/Rare)

Early DBMSs

In the late 1960s, early DBMSs required developers to write queries using procedural code (e.g. IDS, IMS, CODASYL). The developers had to choose access paths and execution ordering based on the current database contents. So, if the data changes, then the developer must rewrite the query code.

And writing queries looks like manually coding loops to traverse data structures (e.g., a nested loop over artists and albums to find matches). You must explicitly tell the system how to navigate the data such as the order of traversal. Certain assumptions about the database (e.g. data size) may hold today but can change over time. Since the execution plan is hard-coded, queries can become inefficient or even invalid as the data grows or changes.

4 Relational Model

Ted Codd at IBM Research in the late 1960s noticed that people were rewriting DBMSs every time they wanted to change the physical layer. In 1969, he proposed the relational model as a potential solution to this.

The relational data model defines three concepts:

- **Structure:** The definition of relations and their contents independent of their physical representation. Each relation has a set of attributes. Each attribute has a domain of values.
- **Integrity:** Ensure the database's contents satisfy certain constraints. An example of a constraint would be that the age of a person cannot be a negative number.
- **Manipulation:** Declarative API for accessing and modifying a database's contents via relations(sets). Programmers only specify the desired result; the database system will decide the most efficient query plan to execute.

The relational model provides data independence that isolates the user/application from low-level data representation. The user only worries about high-level application logic. DBMS optimizes the layout according to operating environment, database contents, and workload. And it will re-optimize if/when these factors change. So the change in the physical data will not break the applications.

A **relation** is an unordered set that contains the relationship of attributes that represent entities. Since the relationships are unordered, the DBMS can store them in any way it wants, allowing for optimization. *It is possible to have repeated / duplicated elements in a relation as long as their primary key is different.*

A **tuple** is a set of attribute values (also known as its *domain*) in the relation. In the past, values had to be atomic or scalar, but now values can also be lists or nested data structures. Every attribute can be a special value, NULL, which means for a given tuple the attribute is undefined.

A relation with n attributes is called an *n -ary relation*. We will interchangeably use *relation* and *table* in this course. An n -ary relation is equivalent to a table with n columns.

A relation's *primary key* uniquely identifies a single tuple in a table. Some DBMSs automatically create an internal primary key if you do not define one. It can auto-generate unique primary keys via an identity column.

A *foreign key* specifies that an attribute from one relation maps to a tuple in another relation. Generally, the foreign key will point / be equal to a primary key in another table.

A *constraint* is a user-defined condition that must hold for *any* instance of the database. Unique key and referential (foreign key) constraints are the most common. Another common example of constraint would be to prohibit a column from having NULL values.

5 Data Manipulation Languages (DMLs)

DMLs refer to the API that a DBMS exposes to applications to store and retrieve information from a database. There are two classes of languages for Manipulating a database:

- **Procedural:** The query specifies the (high-level) execution strategy the DBMS should use to find the desired result based on sets / bags. For example, use a for loop to scan all records and count how many records are there to retrieve the number of records in the table.
- **Non-Procedural (Declarative):** The query specifies only *what* data is wanted and not *how* to find it. For example, we can use SQL `SELECT COUNT(*) FROM artist` to count how many records are there in the table.

6 Relational Algebra

Relational Algebra is a set of fundamental operations to retrieve and manipulate tuples in a relation. Each operator takes in one or more relations as inputs, and outputs a new relation. To write queries we can "chain" these operators together.

Select

Select takes in a relation and outputs a subset of the tuples from that relation that satisfy a selection predicate. The predicate acts as a filter, and we can combine multiple predicates using conjunctions and disjunctions.

Syntax: $\sigma_{\text{predicate}}(R)$.

Example: $\sigma_{\text{a_id}='a2'}(R)$

SQL: `SELECT * FROM R WHERE a_id = 'a2'`

Projection

Projection takes in a relation and outputs a relation with tuples that contain only specified attributes. You can rearrange the ordering of the attributes in the input relation as well as manipulate the values.

Syntax: $\pi_{A1,A2,\dots,A_n}(R)$.

Example: $\pi_{\text{b_id}-100, \text{a_id}}(\sigma_{\text{a_id}='a2'}(R))$

SQL: `SELECT b_id-100, a_id FROM R WHERE a_id = 'a2'`

Union

Union takes in two relations and outputs a relation that contains all tuples that appear in at least one of the input relations. Note: The two input relations have to have the exact same attributes.

Syntax: $(R \cup S)$.

SQL: `(SELECT * FROM R) UNION ALL (SELECT * FROM S)`

Intersection

Intersection takes in two relations and outputs a relation that contains all tuples that appear in both of the input relations. Note: The two input relations have to have the exact same attributes.

Syntax: $(R \cap S)$.

SQL: (SELECT * FROM R) INTERSECT (SELECT * FROM S)

Difference

Difference takes in two relations and outputs a relation that contains all tuples that appear in the first relation but not the second relation. Note: The two input relations have to have the exact same attributes.

Syntax: $(R - S)$.

SQL: (SELECT * FROM R) EXCEPT (SELECT * FROM S)

Product

Product takes in two relations and outputs a relation that contains all possible combinations for tuples from the input relations.

Syntax: $(R \times S)$.

SQL: (SELECT * FROM R) CROSS JOIN (SELECT * FROM S), or simply SELECT * FROM R, S

Join

Join takes in two relations and outputs a relation that contains all the tuples that are a combination of two tuples where for each attribute that the two relations share, the values for that attribute of both tuples is the same.

Syntax: $(R \bowtie S)$.

SQL: SELECT * FROM R JOIN S USING (ATTRIBUTE1, ATTRIBUTE2...)

Observation

Relational algebra defines the fundamental operations to retrieve and manipulate tuples in a relation. It also defines an ordering of the high-level steps to compute a query.

For example, $\sigma_{b_id=102}(R \bowtie S)$ represents joining R and S and then selecting / filtering the result. However, $(R \bowtie (\sigma_{b_id=102}(S)))$ will do the selection on S first, and then join the result of the selection with R .

These two statements will always produce the same answer. However, if S has 1 billion tuples and there is only 1 tuple in S with $b_id=102$, then $(R \bowtie (\sigma_{b_id=102}(S)))$ will be significantly faster than $\sigma_{b_id=102}(R \bowtie S)$. How would you know this if you were using Pandas or another procedural DML?

A better approach is to state the high-level result you want (retrieve the joined tuples from R and S where b_id equals 102), and let the DBMS decide the steps it should take to compute the query.

In SQL (a declarative language) we only express what we want to be computed and we do **not** specify how to compute the result. The DBMS is responsible for finding the best strategy to execute the query (through Query Optimization). This powerful abstraction has made SQL the de facto standard for writing queries on a relational DBMS since the user of the DBMS does not need to know anything about the internals and can query the database in the most efficient way.

7 Other Data Models

Document Data Model

The document data model is a collection of record documents containing a hierarchy of named field/value pairs.

A field's value can be either a scalar type, an array of values, or a pointer to another document.

Modern implementations use JSON. Older systems use XML or custom object representations.

The Document Model has some use cases but still runs into many of the problems discussed in the flat file strawman example.

Vector Data Model

The vector data model represents one-dimensional arrays used for nearest-neighbor search (exact or approximate).

Vector databases are generally used for semantic search on embeddings generated by ML-trained transformer models (think ChatGPT), and native integration with modern ML tools and APIs (e.g., LangChain, OpenAI). At their core, these systems use specialized indexes to perform NN searches quickly.

Recently, many relational DBMSs have shipped vector index features or extensions (pgvector) that allow NN search within the relational model.

Lecture #02: Modern SQL

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/fall2026/>

Carnegie Mellon University

Andy Pavlo

1 SQL History

SQL is a declarative query language for relational databases. As opposed to imperative languages, in a declarative language the programmer/user only declares what needs to be done as opposed to how the operations should be done (e.g. Join these two tables). SQL was originally developed in the 1970s as part of the IBM **System R** project. IBM originally called it “SEQUEL” (Structured English Query Language). The name changed in the 1980s to just “SQL” (Structured Query Language).

Despite SQL being an old language, it is still being actively updated with new features every couple of years. Some of the major updates released with each new edition of the SQL standard are shown below:

- **SQL:1999** Regular Expressions, Triggers, Object-Oriented features
- **SQL:2003** XML, Windows, Sequences, Auto-Gen IDs
- **SQL:2008** Truncation, Fancy Sorting
- **SQL:2011** Temporal DBs, Pipelined DML
- **SQL:2016** JSON, Polymorphic tables
- **SQL:2023** Property Graph Queries, Multi-Dimensional Arrays

The minimum language syntax a system needs to support in order to claim that it supports SQL is SQL-92. Each vendor follows the standard to a certain degree and there are many proprietary extensions.

2 Relational Languages

The language is comprised of different classes of commands:

1. **Data Manipulation Language (DML):** SELECT, INSERT, UPDATE, and DELETE statements.
2. **Data Definition Language (DDL):** Schema definitions for tables, indexes, views, and other objects.
3. **Data Control Language (DCL):** Security and access controls.
4. It also includes view definition, integrity and referential constraints, and transactions.

Relational algebra (which is the algebra that SQL is based on) uses **sets** (unordered collections which do not allow duplicates). However, SQL is based on **bags** (unordered collections which allow duplicates) to avoid the extra work of removing duplicates by default. Duplicates can still be removed via features like the DISTINCT keyword.

3 Example Database

Here is the schema of a database we will use in our examples:

```
CREATE TABLE student (  
  sid INT PRIMARY KEY,  
  name VARCHAR(16),  
  login VARCHAR(32) UNIQUE,  
  age SMALLINT,  
  gpa FLOAT  
);  
  
CREATE TABLE course (  
  cid VARCHAR(32) PRIMARY KEY,  
  name VARCHAR(32) NOT NULL  
);  
  
CREATE TABLE enrolled (  
  sid INT REFERENCES student (sid),  
  cid VARCHAR(32) REFERENCES course (cid),  
  grade CHAR(1)  
);
```

Figure 1: Example database used for lecture

4 Aggregates

An aggregation function takes in a bag of tuples as its input and then produces a single scalar value as its output. Aggregate functions can (almost) only be used in a SELECT output list.

- AVG(COL): The average of the values in COL
- MIN(COL): The minimum value in COL
- MAX(COL): The maximum value in COL
- SUM(COL): The sum of the values in COL
- COUNT(COL): The number of tuples in the relation

Example: *Get # of students with a '@cs' login.*

The following three queries are equivalent:

```
SELECT COUNT(*) FROM student WHERE login LIKE '@cs';
```

```
SELECT COUNT(login) FROM student WHERE login LIKE '@cs';
```

```
SELECT COUNT(1) FROM student WHERE login LIKE '@cs';
```

Some aggregate functions (e.g. COUNT, SUM, AVG) support the DISTINCT keyword:

Example: *Get # of unique students and their average GPA with a '@cs' login.*

```
SELECT COUNT(DISTINCT login)  
FROM student WHERE login LIKE '@cs';
```

A single SELECT statement can contain multiple aggregates:

Example: *Get # of students and their average GPA with a '@cs' login.*

```
SELECT AVG(gpa), COUNT(sid)
FROM student WHERE login LIKE '@cs';
```

Output of other columns outside of an aggregate is undefined (e.cid is undefined below).

Example: *Get the average GPA of students in each course.*

```
SELECT AVG(s.gpa), e.cid
FROM enrolled AS e JOIN student AS s
ON e.sid = s.sid;
```

Most real-world database systems will error in this case, but some systems such as SQLite will allow it by picking an arbitrary value. The SQL:2023 standard now supports the ANY_VALUE aggregation function which does the same thing.

Example: *Get the average GPA of students in each course.*

```
SELECT AVG(s.gpa), ANY_VALUE(e.cid)
FROM enrolled AS e JOIN student AS s
ON e.sid = s.sid;
```

Non-aggregated values in SELECT output clause must appear in the GROUP BY clause. This will partition the tuples based off of the value and calculate the aggregates for each subset. In this case there will be a canonical value for each group.

Example: *Get the average GPA of students in each course.*

```
SELECT AVG(s.gpa), e.cid
FROM enrolled AS e JOIN student AS s
WHERE e.sid = s.sid
GROUP BY e.cid;
```

Grouping sets can be used to specify multiple groupings in a single query rather than using UNION on multiple queries. This results in the DBMS needing to only scan through the data once rather than multiple times.

Example: *Get the count of students by each course and grade, the count of students by course, and the total student count.*

```
SELECT c.name AS c_name, e.grade,
COUNT(*) AS num_students
FROM enrolled AS e
JOIN course AS c ON e.cid = c.cid
GROUP BY GROUPING SETS (
(c.name, e.grade),
(c.name),
());
```

The FILTER clause filters results *before* aggregation computation (i.e. filter out rows going into the aggregate function). This allows computing multiple aggregates on the same raw data but with different conditions. It is helpful for pivoting rows to columns, among other things.

Example: *Get the avg course grade of students enrolled in 15-445 and 15-721.*

```
SELECT
  AVG(s.gpa) FILTER(WHERE e.cid = '15-445') AS intro_db_avg_gpa,
  AVG(s.gpa) FILTER(WHERE e.cid = '15-721') AS adv_db_avg_gpa
FROM enrolled AS e JOIN student AS s
ON e.sid = s.sid
```

The HAVING clause filters output results based on aggregation computation (i.e. filters out groups as opposed to filtering rows which is what the WHERE clause does). This makes HAVING behave like a WHERE clause for a GROUP BY.

Example: *Get the set of courses in which the average student GPA is greater than 3.9.*

```
SELECT AVG(s.gpa) AS avg_gpa, e.cid
FROM enrolled AS e, student AS s
WHERE e.sid = s.sid
GROUP BY e.cid
HAVING avg_gpa > 3.9;
```

Note: this example uses a legacy implicit join syntax (here the DBMS can deduce that a join is required to handle the WHERE clause). You should always write out explicit joins in your queries.

The above query syntax is supported by many major database systems, but is not compliant with the SQL standard. To make the query standard compliant, we must repeat use of AVG(S.GPA) in the body of the HAVING clause.

```
SELECT AVG(s.gpa), e.cid
FROM enrolled AS e, student AS s
WHERE e.sid = s.sid
GROUP BY e.cid
HAVING AVG(s.gpa) > 3.9;
```

5 String Operations

The SQL standard says that strings are **case sensitive** and **single-quotes only**. Real-world systems will vary in how loose they are about both points (e.g. MySQL).

There are functions to manipulate strings that can be used in any part of a query.

Pattern Matching: The LIKE keyword is used for string matching in predicates.

- “%” matches any substrings (including empty).
- “_” matches any one character.

SIMILAR TO allows for regular expression matching but it is not supported across all systems as many have their own syntax instead.

String Functions SQL-92 defines string functions. Many database systems implement other functions in addition to those in the standard. Examples of standard string functions include SUBSTRING(S, B, E) and

UPPER(S).

Concatenation: Two vertical bars (“| |”) will concatenate two or more strings together into a single string (but different systems might use a different symbol or implement it as a function).

6 Date and Time

Databases generally want to keep track of dates and time, so SQL supports operations to manipulate DATE and TIME attributes. These can be used as either outputs or predicates.

Specific syntax for date and time operations can vary wildly across systems.

7 Output Control

Since results SQL are unordered, we must use the ORDER BY clause to impose a sort on tuples:

```
SELECT sid, grade FROM enrolled WHERE cid = '15-721'
ORDER BY grade;
```

The default sort order is ascending (ASC). We can manually specify DESC to reverse the order:

```
SELECT sid, grade FROM enrolled WHERE cid = '15-721'
ORDER BY grade DESC;
```

We can use multiple ORDER BY clauses to break ties or do more complex sorting:

```
SELECT sid, grade FROM enrolled WHERE cid = '15-721'
ORDER BY grade DESC, sid ASC;
```

We can also use any arbitrary expression in the ORDER BY clause:

```
SELECT sid FROM enrolled WHERE cid = '15-721'
ORDER BY UPPER(grade) DESC, sid + 1 ASC;
```

By default, the DBMS will return all of the tuples produced by the query. Many systems provide their own syntax for specifying how to get a set number of the first results from the output, but a common one is the LIMIT clause:

```
SELECT sid, name FROM student WHERE login LIKE '%@cs'
LIMIT 10;
```

We can also provide an offset to return a range in the results:

```
SELECT sid, name FROM student WHERE login LIKE '%@cs'
LIMIT 20 OFFSET 10;
```

Unless we use an ORDER BY clause with a LIMIT, the DBMS may produce different tuples in the result on each invocation of the query because the relational model does not impose an ordering.

SQL also allows you to store query results into a different table with the INTO keyword (some systems even allow redirection into a temporary table with INTO TEMPORARY).

8 Output Redirection

Instead of having the result a query returned to the client (e.g., terminal), you can tell the DBMS to store the results into another table. You can then access this data in subsequent queries.

- **New Table:** Store the output of the query into a new (permanent) table.

```
SELECT DISTINCT cid INTO CourseIds FROM enrolled;
```

- **Existing Table:** Store the output of the query into a table that already exists in the database. The target table must have the same number of columns with the same types as the target table, but the names of the columns in the output query do not have to match.

```
INSERT INTO CourseIds (SELECT DISTINCT cid FROM enrolled);
```

- **Temporary Table:** Store the output of the query into a temporary table created during the insertion. This table can then be used until the client disconnects.

```
SELECT DISTINCT cid INTO TEMPORARY CourseIds FROM enrolled;
```

9 Nested Queries

Nested queries invoke queries inside of other queries to execute more complex logic within a single query. Nested queries are often difficult to optimize.

The scope of the outer query is included in an inner query (i.e. the inner query can access attributes from outer query). The opposite is not true.

Inner queries can appear in almost any part of a query:

1. SELECT Output Targets:

```
SELECT (SELECT 1) AS one FROM student;
```

2. FROM Clause:

```
SELECT name
FROM student AS s, (SELECT sid FROM enrolled) AS e
WHERE s.sid = e.sid;
```

3. WHERE Clause:

```
SELECT name FROM student
WHERE sid IN ( SELECT sid FROM enrolled );
```

Example: *Get the names of students that are enrolled in '15-445'.*

```
SELECT name FROM student
WHERE sid IN (
  SELECT sid FROM enrolled
  WHERE cid = '15-445'
);
```

Note that sid has a different scope depending on where it appears in the query.

Example: *Find student record with the highest id that is enrolled in at least one course.*

```
SELECT student.sid, name
FROM student
JOIN (SELECT MAX(sid) AS sid
      FROM enrolled) AS max_e
ON student.sid = max_e.sid;
```

Nested Query Results Expressions:

- ALL: Must satisfy expression for all rows in sub-query.
- ANY: Must satisfy expression for at least one row in sub-query.
- IN: Equivalent to =ANY().
- EXISTS: At least one row is returned.

Example: Find all courses that have no students enrolled in it.

```
SELECT * FROM course
WHERE NOT EXISTS(
  SELECT * FROM enrolled
  WHERE course.cid = enrolled.cid
);
```

10 Lateral Joins

The LATERAL operator allows a nested query to reference attributes in other nested queries that precede it. You can think of lateral joins like a for loop that allows you to invoke another query for each tuple in a table.

Example: Calculate the number of students enrolled in each course and the average GPA. Sort by enrollment count in descending order..

Once we have gotten the course records, we can think of this query like below. For each course:

- Compute the number of enrolled students in this course
- Compute the average GPA of the enrolled students in this course

```
SELECT * FROM course AS c
LATERAL (SELECT COUNT(*) AS cnt FROM enrolled
        WHERE enrolled.cid = c.cid) AS t1,
LATERAL (SELECT AVG(gpa) AS avg FROM student AS s
        JOIN enrolled AS e ON s.sid = e.sid
        WHERE e.cid = c.cid) AS t2;
```

11 Common Table Expressions

Common Table Expressions (CTEs) are an alternative to windows or nested queries when writing more complex queries. They provide a way to write auxiliary statements for use in a larger query. A CTE can be thought of as a temporary table that is scoped to a single query.

The WITH clause binds the output of the inner query to a temporary table with the same name.

Example: *Generate a CTE called cteName that contains a single tuple with a single attribute set to “1”. Select all attributes from cteName.*

```
WITH cteName AS (  
    SELECT 1  
)  
SELECT * FROM cteName;
```

We can bind output columns to names before the AS:

```
WITH cteName (col1, col2) AS (  
    SELECT 1, 2  
)  
SELECT col1 + col2 FROM cteName;
```

A single query may contain multiple CTE declarations:

```
WITH cte1 (col1) AS (SELECT 1), cte2 (col2) AS (SELECT 2)  
SELECT * FROM cte1, cte2;
```

Adding the RECURSIVE keyword after WITH allows a CTE to reference itself. This enables the implementation of recursion in SQL queries. With recursive CTEs, SQL is provably Turing-complete, implying that it is as computationally expressive as more general purpose programming languages (ignoring the fact that it is a bit more cumbersome).

Example: *Print the sequence of numbers from 1 to 10.*

```
WITH RECURSIVE cteSource (counter) AS (  
    ( SELECT 1 )  
    UNION  
    ( SELECT counter + 1 FROM cteSource  
      WHERE counter < 10 )  
)  
SELECT * FROM cteSource;
```

12 Window Functions

A window function performs “sliding” calculation across a set of tuples that are related. Window functions are similar to aggregations, but tuples are not collapsed into a singular output tuple.

The conceptual execution for window functions can be imagined as such (*note that not all window functions will behave like this*):

1. The table is partitioned
2. Each partition is sorted (if there is an ORDER BY clause)
3. For each record, it creates a window spanning multiple records
4. Finally the output is computed for each window

Functions: The window function can be any of the aggregation functions that we discussed above. There are also also special window functions:

1. ROW_NUMBER: The number of the current row.

2. RANK: The order position of the current row.

Grouping: The OVER clause specifies how to group together tuples when computing the window function. Use PARTITION BY to specify group.

```
SELECT cid, sid, ROW_NUMBER() OVER (PARTITION BY cid)
FROM enrolled ORDER BY cid;
```

We can also put an ORDER BY within OVER to ensure a deterministic ordering of results even if database changes internally.

```
SELECT *, ROW_NUMBER() OVER (ORDER BY cid)
FROM enrolled ORDER BY cid;
```

IMPORTANT: The DBMS computes RANK after the window function sorting, whereas it computes ROW_NUMBER before the sorting.

Example: Find the student with the second highest grade for each course.

```
SELECT * FROM (
  SELECT *, RANK() OVER (PARTITION BY cid
    ORDER BY grade ASC) AS rank
  FROM enrolled) AS ranking
WHERE ranking.rank = 2;
```

Note that we order by ASC because the grades are A, B, C instead of number grades.

Lecture #03: Database Storage (Part I)

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

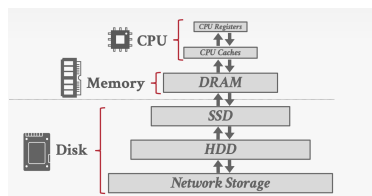
Andy Pavlo

1 Storage

In this class, we focus on a “disk-oriented” DBMS architecture that assumes that the primary storage location of the database is on non-volatile disk(s). This is in contrast to “memory-oriented” DBMS, where tables are stored in DRAM.

1.1 Storage Hierarchy

We illustrate the storage hierarchy for “disk-oriented” DBMS. devices that are closest to the CPU are at the top of the storage hierarchy. This is the fastest storage, but it is also the smallest and most expensive. The further you get away from the CPU, the larger but slower the storage devices get. These devices also get cheaper per GB. There’s also a demarcation line in the middle of the hierarchy that separates volatile devices from non-volatile devices.



Volatile Devices

- Volatile means that the device does not retain its state after power loss. Therefore, the data that is stored in such devices can be lost.
- Volatile storage supports fast random access with byte-addressable locations. This means that the program can jump to any byte address and get the data that is there (e.g. in DRAM).
- For our purposes, we will always refer to this volatile storage class as “memory.”

Non-Volatile Devices

- Non-volatile devices do retain their state even when the machine/computer is off or power loss occurs. Therefore, the data that these devices store can be retrieved even after the machine/computer shuts down and restarts (e.g. disk).
- Non-Volatile devices are block/page addressable. This means that in order to read a value at a particular offset (byte), the program first has to load the 4 KB page into memory that holds the value that the program wants to read.
- Non-volatile storage is traditionally better at sequential access (reading contiguous blocks of data because of its architecture e.g. magnetic hard drive).
- We will refer to this as “disk.” We will not make a (major) distinction between solid-state storage (SSD) and spinning hard drives (HDD).

2 Access Time and Access Pattern

There is a large contrast between latencies accessing volatile vs. non-volatile devices.

As an example to illustrate: suppose that reading data from the L1 cache reference took one second, then reading from an SSD would take 4.4 hours, and reading from an HDD would take 3.3 weeks.

There are two major access patterns, *random access* and *sequential access*. As a general rule, this differentiation does not contribute to latency when accessing *volatile memory*, but plays a significant role in *non-volatile memory* accesses.

On real-world hardware the differences between their access latencies are significant. Random access on non-volatile storage is almost always slower than sequential access. **The DBMS will always target maximizing sequential access.** Some systems will avoid blocking on random writes by writing sequentially into a buffer and later perform random disk writes in the background.

3 Disk-Oriented DBMS Overview

System Design Goals A design goal of the disk-oriented DBMS is to allow it to manage databases that exceeds the amount of memory available. As this requires frequent data movement between memory and disk, the DBMS should manage disk read and write carefully to avoid long stalls on disk I/O and maximize sequential access when possible.

The database is stored on disk, and the data within the database files are organized into pages, with the first page being the *directory page*.

To operate on the data, the DBMS needs to bring the data into memory. It does this by having a *buffer pool* that manages the data movement back and forth between disk and memory.

The DBMS also has an execution engine that will execute queries. The execution engine will ask the buffer pool for a specific page, and the buffer pool will take care of bringing that page into memory and giving the execution engine a pointer to that page in memory. The buffer pool manager will ensure that the page is there while the execution engine operates on that part of memory.

4 File Storage

In its most basic form, a DBMS stores a database as files on disk. Some may use a file hierarchy, others may use a single file (e.g. SQLite).

File Format. The DBMS traditionally store files in a proprietary format that are specific to the DBMS, therefore only the specific DBMS knows how to decipher their contents. More recently there are also portable file formats that are open specs which allow all DBMSs to read and write from them. For either approach, the OS does not know anything about the contents of these files.

File System. The DBMS typically runs on off-the-shelf file system provided by the OS. There are some high-end enterprise systems that could inject a custom file system into the OS specific for the DBMS (e.g. Oracle ASM)

File Storage Manager. The DBMS's *storage manager* is responsible for managing a database's files. It represents the files as a collection of pages. It also keeps track of what data has been read and written to pages as well how much free space there is in these pages.

A DBMS typically does not maintain multiple copies of a page on disk. Replication is not handled on storage manager level. Typically they are handled either in the file system below the storage manager (e.g.

RAID), or above the storage manager where there could be logical copies of tuples.

5 Database Pages

The DBMS organizes the database across one or more files in fixed-size blocks of data called *pages*. Pages can contain different kinds of data (tuples, indexes, etc). Most systems will not mix these types within pages. Some systems will require that pages are *self-contained*, meaning that all the information needed to read each page is on the page itself.

Each page is given a unique identifier (page ID). If the database is a single file, then the page id can just be the file offset. A page ID could be unique per DBMS instance, per database, or per table. Most DBMSs have an indirection layer that maps a page id to a file path and offset. The upper levels of the system will ask for a specific page number. Then, the storage manager will have to turn that page number into a file and an offset to find the page.

Most DBMSs use fixed-size pages to avoid the engineering overhead needed to support variable-sized pages. For example, with variable-size pages, deleting a page could create a hole in files that the DBMS cannot easily fill with new pages.

There are three concepts of pages in DBMS:

1. Hardware page (usually 4 KB).
2. OS page (4 KB).
3. Database page (1-16 KB).

Optimal database page size depends on the environment, database contents, and expected workload. DBMSs that specialize in read-only workloads tend to have larger page sizes ($\geq 1\text{MB}$), while those that specialize in write-heavy workloads tend to have smaller pages (4-16KB).

The storage device guarantees an atomic write of the size of the hardware page. If the hardware page is 4 KB and the system tries to write 4 KB to the disk, either all 4 KB will be written, or none of it will. This means that if our database page is larger than our hardware page, the DBMS will have to take extra measures to ensure that the data gets written out safely since the program can get partway through writing a database page to disk when the system crashes.

6 Database Heap

There are a couple of ways to manage pages in files on the disk (e.g. {Tree, ISAM, Hashing} File Organization), and *Heap File Organization* is one of those ways. A *heap file* is an unordered collection of pages where tuples are stored in random order.

A common architecture for the DBMS to locate a page on disk given a `page_id` is *page directory*, which are special pages that contain one location entry for each logical database objects (e.g. data pages, index pages). The page directory has to be synchronized with the actual pages. The DBMS also tracks metadata about pages' contents, include the amount of free space on each page, a list of free/empty pages, and the page types.

7 Page Layout

Every page includes a header that records meta-data about the page's contents:

- Page size.
- Checksum.

- DBMS version.
- Transaction visibility.
- Self-containment. (Some systems like Oracle require this.)

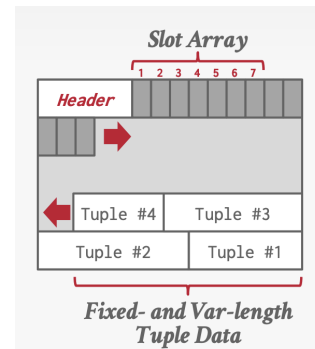
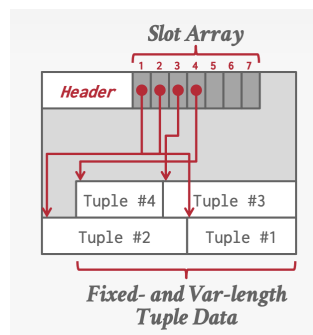
There are three main approaches to laying out data in pages: (1) tuple-oriented, (2) log-structured, and (3) index-oriented.

Tuple-Oriented

In tuple-oriented storage, the entire tuple is stored in the page. A strawman approach to laying out tuples in a page is to keep track of how many tuples the DBMS has stored in the page and then append to the end every time a new tuple is added. However, problems arise when tuples are deleted or when tuples have variable-length attributes. A common layout scheme that solves this is *slotted pages*.

Slotted Pages: Page maps slots to offsets.

- Most common approach used in DBMSs today.
- Header keeps track of the number of used slots, the offset of the starting location of the last used slot, and a slot array, which keeps track of the location of the start of each tuple.
- To add a tuple, the slot array will grow from the beginning to the end, and the data of the tuples will grow from end to the beginning. The page is considered full when the slot array and the tuple data meet.



Log-structured and index-oriented storage are covered in lecture 05.

8 Record IDs

The DBMS assigns each logical tuple a unique *record identifier* that represents its physical location in the database (e.g. file id, page id, slot number). Most DBMSs do not store ids in the tuple. Common record id size varies from 4 bytes to 10 bytes. Since these are physical locations within the DBMS, the application **cannot** rely on these IDs.

9 Tuple Layout

A tuple is essentially a sequence of bytes (these bytes do not have to be contiguous). It is the DBMS's job to interpret those bytes into attribute types and values.

Tuple Header: Contains meta-data about the tuple.

- Visibility information for the DBMS's concurrency control protocol (i.e., information about which transaction created/modified that tuple, will be covered later in the semester).
- Bit Map for NULL values.

- Note that the DBMS does not need to store meta-data about the schema of the database here.

Tuple Data: Actual data for attributes.

- Attributes are typically stored in the order that you specify them when you create the table.
- Attributes must be word aligned.
- Most DBMSs do not allow a tuple to exceed the size of a page.

10 Data Representation

Data representation defines the attribute storage format. Storage formats for common data types are described below.

INTEGER / BIGINT / SMALLINT / TINYINT

Storage layout follows native format in C/C++.

FLOAT / REAL vs. NUMERIC / DECIMAL

Storage layout follows IEEE-754 Standard or Fixed-point Decimals.

- **Variable precision numbers**

Are "native" C/C++ types stored as specified by IEEE-754. They are **inexact** but typically **faster** than fixed precision numbers due to CPU ISA's instruction and register support.

- **Fixed precision numbers**

Allows arbitrary precision and scale. One of many possible implementations is storing in exact, variable-length binary.

VARCHAR / VARBINARY / TEXT / BLOB

Stored as header with length with data bytes **or** pointer to data page and offset.

- **Overflow Pages**

When the value cannot fit in a single page, separate *overflow pages* are used and record id to the tuple is stored inline. Optimizations are possible to compress the overflow page; or to store the prefix of the value inline to reduce indirection when scanning.

- **External Value Storage**

Some system allow large value to be stored in external file and treated as *BLOB* type. DBMS cannot manipulate the content of external file, therefore has no durability and transaction guarantees on it.

TIME / DATE / TIMESTAMP / INTERVAL

Stored as 32/64-bit integer of micro or milli-seconds since Unix epoch.

Null Data Types

There are several approaches to storing null data types.

- **Null Column Bitmap Header**

Store bitmap in centralized header, bit is set when the corresponding attribute is NULL. Most common approach in row-stores.

- **Special Values**

Use a special placeholder for NULL for a data type (e.g. INT32_MIN). Most common in column-stores.

- **Per-Attribute Null Flag**

Stores a flag per-attribute that marks a value is null. Undesirable because the extra bit per-attribute would need to be padded for alignment.

11 DBMS vs. OS

A high-level design goal of the DBMS is to support databases that exceed the amount of available memory. Since reading/writing to disk is expensive, disk use must be carefully managed. We do not want large stalls from fetching something from disk to slow down everything else. We want the DBMS to be able to process other queries while it is waiting to get the data from disk.

This high-level design goal is like virtual memory, where there is a large address space and a place for the OS to bring in pages from disk.

One way to achieve this virtual memory is by using `mmap` to map the contents of a file in a process' address space, which makes the OS responsible for moving pages back and forth between disk and memory. Unfortunately, this means that if `mmap` hits a page fault, the process will be blocked.

- You never want to use `mmap` in your DBMS if you need to write.
- The DBMS (almost) always wants to control things itself and can do a better job at it since it knows more about the data being accessed and the queries being processed.
- The operating system is not your friend.

It is possible to use the OS by using:

- `madvise`: Tells the OS know when you are planning on reading certain pages.
- `mlock`: Tells the OS to not swap memory ranges out to disk.
- `msync`: Tells the OS to flush memory ranges out to disk.

We do not advise using `mmap` in a DBMS for correctness and performance reasons.

Even though the system will have functionalities that seem like something the OS can provide, having the DBMS implement these procedures itself gives it better control and performance.

Lecture #04: Memory & Disk Management

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Introduction

This semester, our focus will be on *disk-oriented database management systems*. A disk-oriented architecture means that the DBMS's primary storage location is in persistent storage, like a hard drive (HDD) or flash storage (SSDs). This is different from an in-memory DBMS, where data is stored in volatile memory.

In the Von Neumann architecture, data must be in memory before we can operate on it (a few exceptions apply for running operations directly on persistent storage). Any DBMS must be able to efficiently move data back and forth between disk and memory if it wants to operate on large amounts of data. The DBMS can achieve this with a Buffer Pool Manager. A diagram of this interaction is shown in Figure 1.

At a high level, the *buffer pool manager* is responsible for moving physical pages of data back and forth from buffers in main memory to persistent storage. It also behaves as a cache, keeping frequently used pages in memory for faster access, and evicting unused or cold pages back out to storage.

From the DBMS's perspective, it should “appear” as if the entire database resides in memory, when in reality the database might occupy more space than the available memory in the system. It should not have to worry about how data is fetched into memory or how it is managed. The DBMS only requires valid pointers to memory locations to perform its operations.

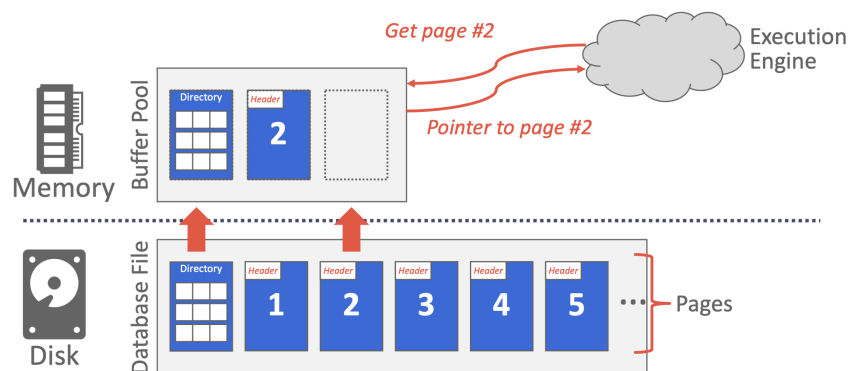


Figure 1: Disk-oriented DBMS.

There are two main things that we will try to optimize for:

1. **Spatial Control**, which refers to where pages are physically located on disk. The goal of spatial control is to keep pages that are used together often as physically close together as possible on disk. This can potentially help with prefetching and other optimizations.
2. **Temporal Control**, which refers to when pages have been brought into memory and when they should be written back out to disk. Temporal control aims to minimize the number of stalls from having to read data from disk.

2 Buffer Pool

The buffer pool is an in-memory cache of pages between memory and disk. It is essentially a large memory region allocated inside of the database to temporarily store pages. It is organized as an array of fixed-size **frames**. When

the DBMS requests a page, the buffer pool manager first checks if the page is already stored in a frame of memory, and if it is not found, the page is read / copied into a free frame from disk. We consider the buffer pool manager as a write-back cache, where dirty pages are buffered and not written to disk immediately on mutation. This is in contrast to a write-through cache, where any changes are immediately propagated to disk.

The DBMS needs this buffer pool memory for many different things (just as most computer programs need memory for different types of data structures):

- Tuple Storage and Indexes
- Sorting and Join Buffers
- Query and Dictionary Caches
- Maintenance and Log Buffers

Depending on the implementation, the things listed above do not always have to be backed by disk / the buffer pool manager, and can simply rely on memory allocators like malloc.

A **page directory** is also maintained on disk, which is the mapping from page IDs to page locations in database files. All changes to the page directory must be recorded on disk to allow the DBMS to find on restart. It is often (not always) kept in memory to minimize latency to page accesses since it has to be read before accessing any physical page.

See Figure 2 for a diagram of the buffer pool's memory organization.

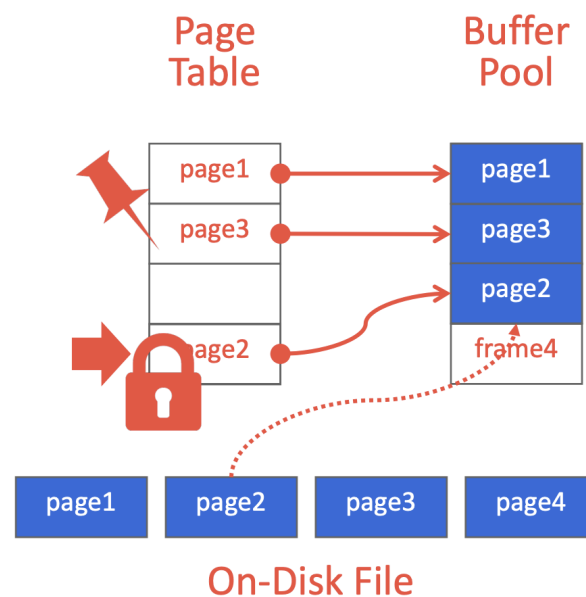


Figure 2: Buffer pool organization and meta-data

Buffer Pool Metadata

The buffer pool must maintain certain meta-data in order to be used efficiently and correctly.

The **page table** is an in-memory hash table that keeps track of pages that are currently in memory. It maps page IDs to frame locations in the buffer pool. Since the order of pages in the buffer pool does not necessarily reflect the order on the disk, this extra indirection layer allows for the identification of page locations in the pool. The page table also maintains additional meta-data per page, a dirty flag, and a pin / reference counter.

The **dirty flag** is set by a thread whenever it modifies a page. This indicates to the storage manager that the page must be written back to disk before eviction.

The **Pin / reference counter** tracks the number of threads that are currently accessing that page (either reading or modifying it). A thread has to increment the counter before they access the page. If a page's pin count is greater than zero, then the storage manager is not allowed to evict that page from memory. Pinning does not prevent other transactions from accessing the page concurrently. If the buffer pool runs out of non-pinned pages to evict and the buffer pool is full, an out-of-memory error will be thrown.

The **Access Tracking Information** tracks what transactions/who is accessing the page.

Page Table vs. Page Directory

To clarify the difference between the page table and the page directory in database management systems:

- The **page directory** is the mapping from page ids to page locations on the physical database files. All changes must be recorded on disk to allow the DBMS to find them on restart.
- The **page table** is the mapping from page ids to a copy of the page in buffer pool frames. This is an in-memory data structure that does not need to be stored on disk.

3 Operating Systems vs. Database Management Systems

Locks vs. Latches

We need to make a distinction between locks and latches when discussing how the DBMS protects its internal elements.

Locks: A lock is a higher-level, logical primitive that protects the contents of a database (e.g., tuples, tables, databases) from other transactions. Database systems can expose to the user which locks are being held as queries are run. Locks need to be able to roll back changes.

Latches: A latch is a low-level protection primitive that the DBMS uses for the critical sections in its internal data structures (e.g., hash tables, regions of memory). Latches are held for only the duration of the operation being made. Latches do not need to be able to roll back changes. This is often implemented with simple language primitives like mutexes and/or conditional variables.

Why not use the OS?

You may wonder why we need to use a buffer pool manager instead of simply relying on the Operating System's page cache. There are several problems with using something like memory-mapped files (mmap):

1. Transaction Safety: The OS can flush dirty pages at any time.
2. I/O Stalls: The DBMS doesn't know which pages are in memory. The OS might stall a thread on a page fault.
3. Error Handling: It is difficult to validate pages. Any page access can cause a SIGBUS signal that the DBMS then must handle.
4. Performance Issues: Internal OS data structure contention. TLB shootdowns.

There are *some* solutions to *some* of these problems (madvise, mlock, msync), but using these syscalls to get the OS to behave correctly is just as onerous as managing memory yourself.

The DBMS almost always wants to control things itself and can do a better job than the OS:

1. Flushing dirty pages to disk in the correct order.
2. Specialized prefetching.
3. Better buffer replacement policies.
4. Thread / process scheduling.

The Operating System is ***not*** your friend.

4 Buffer Replacement Policies

When the DBMS needs to free up a frame to make room for a new page, it must decide which page to evict from the buffer pool, and uses a *replacement policy* to make this decision. The implementation goals of replacement policies are correctness, accuracy, speed, and metadata overhead.

Note: Remember that a pinned page cannot be evicted.

Least Recently Used (LRU)

The Least Recently Used replacement policy maintains a timestamp of when each page was last accessed. The DBMS evicts the page with the oldest timestamp. This timestamp can be stored in a separate data structure, such as a queue that keeps pages in sorted order to reduce the search time on eviction. However, keeping the data structure sorted and storing a large timestamp has a prohibitive overhead.

CLOCK

The CLOCK policy is an approximation of LRU without needing a separate timestamp per page. In the CLOCK policy, each page is given a reference bit. When a page is accessed, it is set to 1. **Note:** Some implementations may allow an actual ref counter greater than 1.

To visualize this, organize the pages in a circular buffer with a “clock hand”. When an eviction is requested, sweep the hand and check if a page’s bit is set to 1. If yes, set it to zero, if no, then evict, bring in the new page in its place, and move the hand forward. Additionally, the clock remembers the position between evictions.

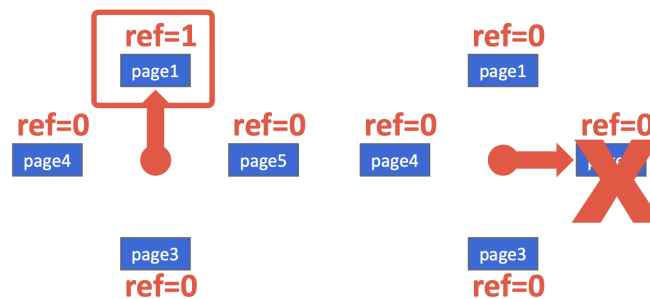


Figure 3: Visualization of CLOCK replacement policy. Page 1 is referenced and set to 1. When the clock hand sweeps, it sets the reference bit for page 1 to 0 and evicts page 5.

Issues

There are a number of problems with LRU and CLOCK replacement policies.

LRU and CLOCK are both susceptible to **sequential flooding**, where the buffer pool’s contents are polluted due to a sequential scan. Since sequential scans read many pages quickly, the buffer pool fills up and pages from other queries are evicted as they would have earlier timestamps. In this scenario (or for analytical workloads), the most recently used page is often the best page to evict.

LRU also does not account for the frequency of accesses. For example, if we have a page that is frequently accessed over time, we do not want to evict it if it wasn’t accessed recently. **Least-Frequently Used** maintains an access count for each page and then evicts pages with the lowest count. However, it ignores time and may accumulate stale pages.

Alternatives

There are three solutions to address the shortcomings of LRU and CLOCK policies.

One solution is **LRU-K** which tracks the history of the last K references as timestamps and computes the interval between subsequent accesses. This history is used to predict the next time a page is going to be accessed. However, this again has a higher storage overhead. Additionally, need to maintain an in-memory “ghost cache” of recently evicted pages to prevent thrashing and have some history for recently evicted pages.

An approximation for LRU-2 done by MySQL is to have single linked list with two entry points. Whenever a page is accessed and it is already in the old list put it at the start of the young queue, else put it at the start of the old list. Evictions always are taken from end of the old list.

Another optimization is **localization** per query. The DBMS chooses which pages to evict on a per query / transaction basis. This minimizes the pollution of the buffer pool from each query.

Lastly, **priority hints** allow transactions to tell the buffer pool whether page is important or not based on the context of each page during query execution.

ARC: Adaptive Replacement Cache

ARC was developed by IBM Research and supports both LRU and LFU by dynamically adjusting the sizes of two lists based on workload access patterns. ARC uses ghost lists to remember recent evictions. ARC relies on a target size parameter **p** to adjust how much to favor recency versus frequency.

1. Recency List **T1**: Holds pages that have been accessed once recently.
2. Frequency List **T2**: Holds pages that have been accessed at least twice.
3. Target Size Parameter **p** adjusts how much to favor recency (T1) versus frequency (T2)

Dirty Pages

Pages / frames keep track of a dirty flag / bit that denotes a page that has been modified by the DBMS. There are two cases in handling eviction of dirty pages. The fast path is when the page is not dirty, and the buffer pool manager can simply drop it. The slow path is when the page is dirty, and the buffer pool manager must write the changes back to disk to ensure data synchronization between memory and disk.

One way to avoid the problem of having to incur the cost of writing out pages is **background writing**. Through background writing, the DBMS can periodically walk through the page table and write dirty pages to disk. When a dirty page is safely written, the DBMS can either evict the page or just unset the dirty flag.

5 Disk I/O and OS Cache

The OS / hardware will try to maximize disk bandwidth by reordering and batching I/O requests. But they do not know which I/O requests are more important than others.

The DBMS maintains internal queue(s) to track page read/write requests from the entire system. The priority of the tasks are determined based on several factors:

- Sequential vs. Random I/O
- Critical Path Task vs. Background Task
- Table vs. Index vs. Log vs. Ephemeral Data
- Transaction Information
- User-based SLAs

The OS doesn't know these things and is going to get into the way. Most disk operations go through the OS API. Unless explicitly told otherwise, the OS maintains its own filesystem cache (aka page / buffer cache).

Most DBMS use Direct I/O (via the `O_DIRECT` flag) to bypass the OS's cache to avoid redundant copies of pages and having to manage different eviction policies. **PostgreSQL** is an example of a database system that uses the OS's page cache.

Side Note: `fsync` by default has silent errors and on errors marks the page as clean.

Again, the Operating System is ***not*** your friend.

6 Buffer Pool Optimizations

There are several ways to optimize a buffer pool to tailor it to the application's workload.

Multiple Buffer Pools

The DBMS can maintain multiple buffer pools for different purposes (i.e. per-database buffer pool, per-page type buffer pool). Then, each buffer pool can adopt local policies tailored to the data stored inside of it. This method can help reduce latch contention and improve locality. Object IDs and hashing are two approaches to mapping desired pages to a specific buffer pool. **Object IDs** involve extending the record IDs to have an object identifier. A mapping from objects to specific buffer pools can be maintained via these object IDs. This allows a finer-grained control over buffer pool allocations but has a storage overhead. Another approach is **hashing**, where the DBMS hashes the page ID to select which buffer pool to access. This is a more general and uniform approach.

Pre-fetching

The DBMS can also be optimized by pre-fetching pages based on the query plan. While the first set of pages is being processed, the second can be pre-fetched into the buffer pool. This method is commonly used by DBMSs when accessing many pages sequentially during a sequential scan. It is also possible for a buffer pool manager to prefetch leaf pages in a tree index data structure benefiting index scans. Note that this does not necessarily need to be the next physical page on disk, but instead the next logical page in the leaf scan.

Scan Sharing (Synchronized Scans)

Query cursors can reuse data retrieved from storage or operator computations. This allows multiple queries to attach to a single cursor that scans a table. If a query starts a scan and there is another active scan, then the DBMS will attach the second query's cursor to the existing cursor. The DBMS keeps track of where the second query joined with the first so that it can finish the scan when it reaches the end of the data structure. This satisfies correctness since the order of scans is not guaranteed by a DBMS and is often useful when a table is frequently scanned.

Note: Continuous scan sharing is an academic prototype that constantly runs a sequential scan for certain tables and queries can hop on and off. However, this is very wasteful and costly when paying per I/O.

Buffer Pool Bypass

The sequential scan operator will not store fetched pages in the buffer pool to avoid overhead. Instead, memory is local to the running query. This works well if an operator needs to read a large sequence of pages that are contiguous on disk and will not be used again. Buffer Pool Bypass can also be used for temporary data (sorting, joins).

7 Conclusion

The DBMS can almost always manage memory better than the OS. It can leverage the semantics about the query plan to make better decisions:

- Evictions
- Allocations
- Pre-fetching

To reiterate, the Operating System is ***not*** your friend.

Lecture #05: Database Storage (Part II)

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Tuple-Oriented Storage

The most common way to store tuples on disk is the Tuple-Oriented Storage architecture, using the slotted-page scheme described in previous lectures. Tuples are retrieved using its record ID:

- Check the page directory to find the page position on disk.
- Fetch the page from disk into memory (into the buffer pool).
- Use the slot array to find the tuple's offset within the page.

Inserting a new tuple is simple:

- Check the page directory to find a page with a free slot.
- Fetch the page from disk into memory.
- Use the slot array to check if there is enough free space in the page.
- If not, find another page with a free slot or create a new page.
- Insert the tuple into the page and update the slot array.

However, updating tuples can become expensive:

- Navigate to the tuple using the record ID with the same steps as retrieval.
- If the new value fits in the same space, update in place.
- Otherwise, mark the old value as deleted and insert the new value as if it were a new tuple.

Therefore, while tuple-oriented storage is great for reads, there are several problems associated with it:

- **Fragmentation:** Deletion of tuples can leave gaps in the pages, making them not fully utilized.
- **Useless Disk I/O:** Due to the block-oriented nature of non-volatile storage, the whole block needs to be fetched to update a tuple.
- **Random Disk I/O:** The disk reader could have to jump to 20 different places to update 20 different tuples, which can be very slow.

What if we were working on a system which only allows creation of new pages and no in-place updates (e.g. HDFS, Google Colossus, certain object stores)? The log-structured storage model works with this assumption and addresses some of the problems listed above.

2 Log-Structured Storage

Instead of storing tuples in pages and updating them in-place, Log-Structured Storage maintains a log that records changes to tuples. This idea is based on log-structured file systems (LSFS) ¹ and log-structured merge trees (LSM Tree)².

The DBMS applies changes to an in-memory data structure (**MemTable**) and writes out the changes sequentially to disk (**SSTable**). The records stored in these structures contain the tuple's unique identifier,

¹<https://doi.org/10.1145/146941.146943>

²<https://doi.org/10.1007/s002360050048>

the type of operation (PUT/DELETE), and for a PUT operation, the contents of the tuple. Effectively, you care about the latest values for each key (most recent PUT/DELETE).

Logs are first stored in **MemTable** through fast, in-memory operations. Once **MemTable** fills up, the DBMS serializes the logs it stores and writes them to disk as an **SSTable**. The DBMS also sorts each **SSTable** by key before writing it to disk. Since the **SSTables** are immutable and written to disk sequentially, this results in less random disk I/O. This workload also maps nicely to **append-only** storage like many cloud storage options, etc.

To read a record, the DBMS first checks **MemTable** to see whether it exists there. If the key does not exist in **MemTable**, then the DBMS has to check the **SSTables** at each level. A brute force solution is to scan down the **SSTables** from newest to oldest and perform binary search within each **SSTable** to find the most recent contents of the tuple, which can be slow. To avoid this, the DBMS can maintain an in-memory **SummaryTable** to track additional metadata like min/max key per **SSTable** and a key filter (e.g., Bloom filter) per level.

Compaction

In a write-heavy workload, the DBMS will accumulate a large number of **SSTables** on disk. Thus, the DBMS can periodically use a sort-merge algorithm to combine **SSTables** by taking only the most recent change for each tuple. This reduces wasted space and speeds up reads.

There are many ways to compact log files. In **Universal Compaction**, **SSTables** reside in a single "universal" level. DBMS will trigger compaction when size thresholds are met or too many **SSTables** overlap in key ranges. This approach works better for insert-heavy workloads and time-oriented queries. In **Level Compaction**, the smallest files are level 0. Level 0 files can be compacted to create a bigger level 1 file, level 1 files can be compacted to a level 2 file, etc. **SSTables** in the same level are managed with sorted and non-overlapping key ranges (except for level 0, which may have overlapping key ranges). **Level Compaction** works well with read-heavy workloads.

Tradeoffs

The tradeoffs of using Log-Structured Storage are summarized below:

- Fast sequential writes, good for append only storage.
- Reads may be slow.
- Compaction is expensive.
- Write amplification (for each logical write, there could be multiple physical writes during the compaction process).

3 Index-Organized Storage

Both slotted-page storage and log-structured storage rely on an additional index to find individual tuples because the tables are inherently unsorted. In the **index-organized storage** scheme, the DBMS directly stores a table's tuples as the value of an index data structure (e.g. B+ tree, skip list, trie). The DBMS uses a page layout similar to a slotted page, and tuples are typically sorted in the page based on key.

4 System Catalogs

In order for the DBMS to decipher the contents of tuples, it maintains an internal catalog containing metadata about its databases.

Metadata Contents:

- The tables, views, columns and procedures the database has as well as any indexes on those tables.
- Users of the database and what permissions they have.
- Internal statistics about tables (i.e., max value of an attribute).

Most DBMSs store their catalog inside of themselves as tables. They use special code to “bootstrap” these catalog tables.

Lecture #06: Storage Models & Compression

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Database Workloads

OLTP: Online Transaction Processing

An OLTP workload is characterized by fast, short running operations, repetitive operations and simple queries that operate on single entity at a time. OLTP workloads typically handle more writes than reads, and only read/update a small amount of data each time.

An example of an OLTP workload is the Amazon storefront. Users can add things to their cart and make purchases, but the actions only affect their account. If you're a startup, this is usually what you build first (Postgres is usually the right choice).

OLAP: Online Analytical Processing

An OLAP workload is characterized by long running, complex queries (which often involves computing aggregates) and reads on large portions of the database. In OLAP workloads, the database system is often analyzing and deriving new data from existing data collected on the OLTP side.

An example of an OLAP workload that analyzes data collected on the OLTP side is personalized Amazon shopping ads. The website analyzes all of data collected from users' carts and purchases, and then selects different ads for different users.

HTAP: Hybrid Transaction + Analytical Processing

A new type of workload (which has become popular recently) is HTAP, where OLTP and OLAP workloads are present together on the same database instance. For example, SingleStore claims that they are HTAP.

Workload Focus:

- OLTP - Simple Queries, Write Heavy
- OLAP - Complex Queries, Read Heavy
- HTAP - A mix of both the above

2 Storage Models

There are different ways to store tuples in pages. The **storage model** of a database specifies how it physically organizes tuples on disk and in-memory. The storage model impacts the performance characteristics for different workloads and influences the design choices for the rest of the DBMS.

N-Ary Storage Model (NSM)

In the n-ary storage model, the DBMS stores all of the attributes for a single tuple (row) contiguously in a single page. Thus, it is also known as a **row store**. This approach is ideal for OLTP workloads where requests are write-heavy and accesses are mostly individual entities. It is ideal because it takes only one

fetch to be able to get all of the attributes for a single tuple. NSM pages are typically some constant multiple of **4KB** hardware pages. The DBMS identifies a physical tuple using its record ID (page # + slot #).

Advantages:

- Fast inserts, updates, and deletes.
- Good for queries that need the entire tuple (OLTP).
- Can use index-oriented physical storage for clustering.

Disadvantages:

- Inefficient for scanning large portions of the table and/or a subset of the attributes.
- Poor memory locality in access patterns.
- Difficult to apply compression because of multiple value domains within a single page.

Decomposition Storage Model (DSM)

In the decomposition storage model, the DBMS stores a single attribute (column) for all tuples contiguously in a block of data. Thus, it is also known as a **column store**. This model is ideal for OLAP workloads with many read-only queries that perform large scans over a subset of the table's attributes.

Advantages:

- Reduces the amount of I/O wasted per query because the DBMS only reads the attributes that it needs for a given query.
- Better (faster) query processing because of increased locality and cached data reuse.
- Better data compression.

Disadvantages:

- Slow for point queries, inserts, updates, and deletes because of tuple splitting/stitching.

To put the tuples back together when using a column store, there are two common approaches:

- The most commonly used approach is **fixed-length offsets**. Here, the value in a given column will belong to the same tuple as the value in another column at the same offset. Therefore, every single value within the column will have to be the same length. To deal with variable length fields, we can use padding (wasteful space) or dictionary compression.
- A less common approach is to use **embedded tuple ids**. Here, for every attribute in the columns, the DBMS stores a tuple id (ex: a primary key) with it. Then, the system would also store a mapping to tell it how to jump to every attribute that has that id. Note that this method has a large storage overhead because it needs to store a tuple id for every attribute entry (this is a bad idea!).

Partition Attributes Across (PAX)

In the hybrid Partition Attributes Across storage model, the DBMS vertically partitions attributes within a database page. The goal of doing so is to get the benefit of faster processing on columnar storage while retaining the spatial locality benefits of row storage.

In PAX, the rows are horizontally partitioned into groups of rows. Within each row group, the attributes are vertically partitioned into columns. Each row group is similar to a column store for its subset of the rows. Examples of PAX file formats are Parquet, ORC, and Arrow. Note that when DBMS say they are a column store, they are most likely using PAX.

A PAX file has a global header containing a directory with offsets to the file's row groups, and each row group maintains its own header with meta-data about its contents.

3 Database Compression

Compression is widely used in disk-based DBMSs as disk I/O is (almost) always the main bottleneck. It is especially popular in systems that have read-only analytical workloads. The DBMS can fetch more useful tuples, if they have been compressed beforehand at the cost of greater computational overhead for compression and decompression.

In-memory DBMSs are more complicated since they do not have to fetch data from disk to execute a query. Memory is much faster than disks, but compressing the database reduces DRAM requirements and processing. They have to strike a balance between **speed** vs. **compression ratio**. Compressing the database reduces DRAM requirements and may decrease CPU costs during query execution.

Given this, we want a database compression scheme to have the following properties:

- Must produce fixed-length values. The only exception is variable length data stored in separate pools. This is because the DBMS should follow word-alignment and be able to access data using offsets.
- Allow the DBMS to postpone decompression as long as possible during query execution (**late materialization**).
- Must be a **lossless** scheme because people do not like losing data. Any kind of **lossy** compression has to be performed at the application level.

Compression Granularity

The kind of data we want to compress greatly affects which compression schemes can be used. There are four levels of compression granularity:

- **Block Level:** Compress a block of tuples for the same table.
- **Tuple Level:** Compress the contents of the entire tuple (NSM only).
- **Attribute Level:** Compress a single attribute value within one tuple. Can target multiple attributes for the same tuple.
- **Columnar Level:** Compress multiple values for one or more attributes stored for multiple tuples (DSM only). This allows for more complicated compression schemes.

4 Naive Compression

The DBMS compresses data using a general purpose algorithm (e.g., Deflate, LZ0, LZ4, Snappy, Oracle OZIP, Zstd, Lizard). Although there are several compression algorithms that the DBMS could use, engineers often choose ones that often provides lower compression ratio in exchange for faster compression/decompression.

The DBMS compresses disk pages, pads them to a power of 2KBs and stores them into the buffer pool. However, every time the DBMS tries to read/update data, the compressed data in the buffer pool must first be decompressed. For blind writes, no decompression required.

Optionally, some DBMS attach a mod log to each compressed page. When a DBMS writes to the page, it writes to the mod log without decompressing the page. Similarly, when the DBMS reads, if the data is found in the mod log, no decompressing is necessary. When there are spare cycles or the mod log is full, the DBMS decompresses the page, applies the updates from the mod log, then compresses again.

Since accessing data requires decompression of compressed data, this limits the scope of the compression scheme. If the goal is to compress the entire table into one giant block, using naive compression schemes would be impossible since the whole table needs to be compressed/decompressed for every access. Therefore, MySQL breaks the table into smaller chunks since the compression scope is limited.

Another problem is that these naive schemes also do not consider the high-level meaning or semantics of the data. The algorithm is oblivious to both the structure of the data, and how the query is planning to access the data. Thus, this eliminates the opportunity to utilize late materialization, since the DBMS cannot tell when it can delay the decompression of data.

5 Columnar Compression

We now introduce built-in compression schemes that allow DBMSs to read tuples without having to decompress them to their original form. Note that some of these compression schemes can be composed for even greater performance and compression.

Run-Length Encoding (RLE)

RLE compresses runs (consecutive instances) of the same value in a single column into triplets:

- The value of the attribute
- The start position in the column segment (offset)
- The number of elements in the run (length)

The DBMS should sort the columns intelligently beforehand to maximize compression opportunities. This clusters duplicate attributes, thereby increasing the compression ratio. Note that the effectiveness of RLE greatly depends on the underlying data characteristics (e.g. number and frequency of attributes in each data).

Bit-Packing Encoding

When all values for an attribute are less than the value's declared largest size, store them with fewer bits.

Patching / Mostly Encoding

Bit-packing variant that uses a special marker to indicate when a value exceeds the largest size and then maintains a look-up table to store them. Use when values are "mostly" less than the largest size.

Bitmap Encoding

The DBMS stores a separate bitmap for each unique value of a particular attribute where an offset in the vector corresponds to a tuple. The i^{th} position in the bitmap corresponds to the i^{th} tuple in the table and indicates whether that value is present in the attribute of that tuple. The bitmap is typically segmented into chunks to avoid allocating large blocks of contiguous memory.

This approach is only practical if the value cardinality is low, since the size of the bitmap is linearly proportional to the cardinality of the attribute value. If the cardinality of the value is high, then the bitmap can become larger than the original data set.

There are compressed data structures for sparse data sets. For example, Roaring Bitmaps store dense chunks using uncompressed bitmaps, whereas sparse chunks are stored using bitpacked arrays of 16-bit integers.

Delta Encoding

Instead of storing exact values, record the difference between values that follow each other in the same column. The base value can be stored in-line or in a separate look-up table. We can also use RLE on the stored deltas to get even better compression ratios.

Dictionary Compression

The most common database compression scheme is dictionary encoding. The DBMS replaces frequent patterns in values with smaller codes. It then stores only these codes and a data structure (i.e. the dictionary) that maps these codes to their original value. A dictionary compression scheme needs to support fast encoding/decoding, and needs to be order-preserving if it needs to work with range queries.

Encoding and Decoding: The dictionary needs to decide how to **encode** (convert uncompressed value into its compressed form) and **decode** (convert compressed value back into its original form) data. As such, it is not possible to use hash functions.

The encoded values also need to support sorting in the same order as original values (i.e. **order-preserving encodings**). This ensures that the compressed queries run on compressed data return results that are consistent with uncompressed queries run on the original data. This order-preserving property allows operations to be performed directly on the codes.

On certain queries (such as string match), queries can be executed faster.

6 Bifurcated Environment

Since an OLAP DBMS is superior for analytical queries than an OLTP DBMS, we should always use an OLAP DBMS for them. But if new data arrives at an OLTP DBMS, then we need to transfer data from the OLTP DBMS to the OLAP DBMS (typically called the data warehouse/lakehouse). The technique we use is called **ETL: Extract, Transform, Load**. This allows us to clean and normalize data across different schemas stored in the OLTP DBMSes.

Alternatively, an HTAP combines both OLTP and OLAP into one system. There are two main approaches. In **fractured mirrors**, all updates are first entered into the NSM, then eventually converted into a column store and copied into the DSM mirror. Analytical queries can now run on the DSM mirror. In **delta store**, all updates are entered into the delta store (NSM). A background thread migrates updates from the delta store and applies them to the DSM data. Note that a tuple can only exist in either NSM or DSM, not both (unlike fractured mirrors).

Lecture #07: Hash Tables

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Data Structures

A DBMS uses various data structures for many different parts of the system internals. Some examples include:

- **Internal Meta-Data:** This is data that keeps track of information about the database and the system state.
Ex: Page tables, page directories
- **Core Data Storage:** Data structures store the actual tuples (records) in the database.
- **Temporary Data Structures:** The DBMS can build ephemeral data structures on the fly while processing a query to speed up execution.
Ex: hash tables for joins
- **Table Indices:** Additional data structures that help efficiently locate specific tuples.

There are two key design decisions to consider when implementing data structures in a DBMS:

1. **Data organization:** We need to consider the layout of the data structure and how it impacts performance. For example, a data structure that may be inefficient for memory reads due to poor sequential access might perform better in disk-based DBMS systems, where sequential performance is more crucial.
2. **Concurrency:** We also need to ensure that multiple threads can access the data structure simultaneously without causing conflicts, while maintaining data integrity and correctness.

2 Hash Table

A hash table implements an associative array abstract data type that maps keys to values. It provides on average $O(1)$ operation complexity ($O(n)$ in the worst-case) and $O(n)$ storage complexity. Note that even with $O(1)$ operation complexity on average, there are constant factor optimizations which are important to consider in the real world. There is no sequential access with respect to other keys, meaning the order of keys in the hash table is not preserved, and navigating through the elements in any specific sequence is not possible.

A hash table implementation is comprised of two parts:

- **Hash Function:** This tells us how to map a large key space into a smaller domain. It is used to compute an index into an array of buckets or slots. When choosing a hash function, we need to consider the tradeoff between speed and the chance of collisions (when two keys map to the same slot). At one extreme, a hash function that always returns the same value is very fast but causes constant collisions. At the other extreme, a "perfect" hash function avoids collisions entirely but is too slow to compute. The best solution lies somewhere in between, balancing speed with a low collision rate.
- **Hashing Scheme:** This tells how to handle key collisions after hashing. We need to balance the trade-off between allocating a larger hash table to reduce collisions and performing additional operations when collisions do occur.

3 Hash Functions

A *hash function* takes in any key as its input. It then returns an integer representation of that key (i.e., the “hash”). The function’s output is deterministic (i.e., the same key should always generate the same hash output).

The DBMS need not use a cryptographically secure hash function (e.g., SHA-256) because we do not need to worry about protecting the contents of keys. These hash functions are primarily used internally by the DBMS and thus information is not leaked outside of the system. In general, we only care about the hash function’s speed and collision rate.

The current state-of-the-art hash function is Facebook XXHash3.

4 Static Hashing Schemes

A static hashing scheme is one where the size of the hash table is fixed and known before. This means that if the DBMS runs out of storage space in the hash table, then it has to rebuild a larger hash table from scratch, which is very expensive. Typically the new hash table is twice the size of the original hash table.

To reduce the number of wasteful comparisons, it is important to avoid collisions of hashed key. Typically, we use twice the number of slots as the number of expected elements.

The following assumptions usually do not hold in reality:

1. The number of elements is known ahead of time and fixed.
2. Each key is unique.
3. Perfect hash function guarantees no collisions (if $\text{key1} \neq \text{key2}$ then $\text{hash}(\text{key1}) \neq \text{hash}(\text{key2})$)

Therefore, we need to choose the hash function and hashing schema appropriately.

4.1 Linear Probe Hashing

This is the most basic hashing scheme. It is also typically the fastest. It uses a circular buffer of array slots. The hash function maps keys to slots. Compared to bucket chained hashing, it offers reduced memory overhead by avoiding pointer storage and better cache performance due to contiguous memory access. However, it can suffer from primary clustering when the load factor becomes high.

Insertion - For insertions, when a collision occurs, we linearly search the subsequent slots until an open one is found, looping around from the end to the start of the array if necessary.

Lookup - For lookups, we can check the slot the key hashes to, and search linearly until we find the desired entry. If we reach an empty slot or we iterated over every slot in the hashtable, then the key is not in the table. Note that this means we have to store both key and value in the slot so that we can check if an entry is the desired one.

Deletion - Deletions are more tricky. We have to be careful about just removing the entry from the slot, as this may prevent future lookups from finding entries that have been put below the now empty slot. There are two solutions to this problem:

- The most common approach is to use “tombstones”. Instead of deleting the entry, we replace it with a “tombstone” entry which tells future lookups to keep scanning. Note that insertions are able to insert into tombstone indices.
- The other option is to shift the adjacent data after deleting an entry to fill the now empty slot. However, we must be careful to only move the entries which were originally shifted. This is rarely implemented in practice as it is extremely expensive when we have a large number of keys.

Storing Keys/Values: Two approaches for storing key/value entries in a hash table:

- Fixed-length Key/Values: These are stored directly within the hash table pages. Optionally, the key's hash can be stored alongside for faster comparisons.
- Variable-length Key/Values: These are handled by inserting the key/value data into a separate temporary table. The hash table then stores the hash as the key and a record ID pointing to the corresponding entry in the temporary table as the value.

Non-unique Keys: In the case where the same key may be associated with multiple different values or tuples, there are two approaches.

- Separate Linked List: Instead of storing the values with the keys, we store a pointer to a separate storage area that contains a linked list of all the values, which may overflow to multiple pages.
- Redundant Keys: The more common approach is to simply store the same key multiple times in the table. Everything with linear probing still works even if we do this.

Optimizations: There are several ways to further optimize this hashing scheme:

- Specialized hash table implementations based on the data type or size of keys: These could differ in the way they store data, perform splits, etc. For example, if we have string keys, we could store smaller strings in the original hash table and only a pointer or hash for larger strings.
- Storing metadata in a separate array: An example would be to store empty slot/tombstone information in a packed bitmap as a part of the page header or in a separate hash table, which would help us avoid looking up deleted keys.
- Maintaining versions for the hash table and its slots: Since allocating memory for a hash table is expensive, we may want to reuse the same memory repeatedly. To clear out the table and invalidate its entries, we can increment the version counter of the table instead of marking each slot as deleted/empty. A slot can be treated as empty if there is a mismatch between the slot version and table version.

4.2 Cuckoo Hashing

Instead of using a single hash table, this approach maintains multiple hashtables with different hash functions. The hash functions are the same algorithm (e.g., XXHash, CityHash); they generate different hashes for the same key by using different seed values.

When we insert, we check every table and choose one that has a free slot (if multiple have one, we can compare things like load factor, or more commonly, just choose a random table). If no table has a free slot, we choose (typically a random one) and evict the old entry. We then rehash the old entry into a different table. In rare cases, we may end up in a cycle. If this happens, we can rebuild all of the hash tables with new hash function seeds (less common) or rebuild the hash tables using larger tables (more common).

Cuckoo hashing guarantees $O(1)$ lookups and deletions, but insertions may be more expensive.

Professor's note: The essence of cuckoo hashing is that multiple hash functions map a key to different slots. In practice, cuckoo hashing is implemented with multiple hash functions that map a key to different slots in a single hash table. Further, as hashing may not always be $O(1)$, cuckoo hashing lookups and deletions may cost more than $O(1)$.

5 Dynamic Hashing Schemes

The static hashing schemes require the DBMS to know the number of elements it wants to store. Otherwise it has to rebuild the table if it needs to grow/shrink in size.

Dynamic hashing schemes are able to resize the hash table on demand without needing to rebuild the entire table. The schemes perform this resizing in different ways that can either maximize reads or writes.

5.1 Chained Hashing

This is the most common dynamic hashing scheme. The DBMS maintains a linked list of buckets for each slot in the hash table. Keys that hash to the same slot are simply inserted into the linked list for that slot. To look up an element, we hash to its bucket and then scan through the linked list to find it.

Optimization: Each bucket pointer can maintain a filter (e.g., a Bloom filter) to quickly check whether a key may exist in the bucket. If the filter indicates that the key is not present, the DBMS can avoid scanning the bucket. It allows false positives but never false negatives.

5.2 Extendible Hashing

Extendible hashing is an improved variant of chained hashing that splits buckets instead of allowing chains to grow indefinitely. This approach allows multiple slot locations in the hash table to point to the same bucket chain. The core idea behind rebalancing the hash table is to move bucket entries on split and increase the number of bits examined to find entries in the hash table. This means that the DBMS only has to move data within the buckets of the split chain; all other buckets remain untouched.

- The DBMS maintains a global and local depth bit counts. These bit counts determine the number of most significant bits we need to look at to find buckets in the slot array.
- When a bucket is full, the DBMS splits the bucket and re-distributes its elements. If the local depth of the split bucket is less than the global depth, then the new bucket is just added to the existing slot array. Otherwise, the DBMS doubles the size of the slot array to accommodate the new bucket and increments the global depth counter.

5.3 Linear Hashing

Instead of immediately splitting a bucket when it overflows, this scheme maintains a *split pointer* that keeps track of the next bucket to split. No matter whether this pointer is pointing to the bucket that overflowed, the DBMS always splits. The overflow criterion is left up to the implementation.

- When any bucket overflows, split the bucket at the pointer location. Add a new slot entry and a new hash function, and apply this function to rehash the keys in the split bucket.
- If the original hash function maps to a slot that has previously been pointed to by the split pointer, apply the new hash function to determine the actual location of the key.
- When the pointer reaches the very last slot, delete the original hash function (since all buckets are now addressed using the new hash function) and move the pointer back to the beginning.

If the highest bucket below the split pointer is empty, we can also remove the bucket and move the split pointer in the reverse direction, thereby shrinking the size of the hash table.

The key advantage of linear hashing is that the table grows or shrinks incrementally, avoiding the high cost of a full rebuild.

Lecture #08: Indexes & Filters I

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Indexes

There are several data structure choices in DBMS for purposes such as internal metadata, core data storage, temporary data structures, or indexes. Our focus in this lecture will be on indexes. In the previous lecture we considered hash tables, which is not always the best option for indexes since they cannot support range scans nor partial key look-ups.

An *index* is a replica of a subset of a table's attributes that is organized and/or sorted for efficient access to the location of specific tuples. So instead of performing a sequential scan, the DBMS can perform a lookup on the index to find certain tuples more quickly. The DBMS ensures that the contents of the tables and the indexes are always logically in sync.

There exists a trade-off between the number of indexes to create per database. Although having more indexes makes looking up queries faster, indexes also use storage and require maintenance. Plus, there are concurrency concerns with respect to keeping them in sync. It is the DBMS's job to figure out the best indexes to use to execute queries.

2 B+Tree

A *B+Tree* is a self-balancing tree data structure that keeps data sorted and allows searches, sequential access, insertions, and deletions in $\mathcal{O}(\log(n))$. It is optimized for disk-oriented DBMSs that read/write large blocks of data as it converts what would have potentially been random I/O to sequential I/O.

Almost every modern DBMS that supports order-preserving indexes uses a B+Tree. There is a specific data structure called a *B-Tree*, but people also use the term to generally refer to a class of data structures. The primary difference between the original *B-Tree* and the B+Tree is that B+Trees store values *only in leaf nodes*. Modern B+Tree implementations combine features from other B-Tree variants, such as the sibling pointers used in the B^{link} -Tree.

Formally, a B+Tree is an m -way search tree (where m represents the fanout, or the maximum number of children a node can have) with the following properties:

- It is perfectly balanced (i.e., every leaf node is at the same depth).
- Every inner node other than the root is at least half full ($\frac{m}{2} - 1 \leq \text{num of keys} \leq m - 1$).
- Every inner node with k keys has $k+1$ non-null children.

Every node in a B+Tree contains an array of key/value pairs.

For leaf nodes, the keys are derived from the attribute(s) that the index is based on. Although it is not necessary according to the definition, arrays at every node are almost always sorted by the keys. Two approaches for representing leaf node values are *record IDs* and *tuple data*. Record IDs refer to a pointer to the location of the tuple, usually the primary key. Leaf nodes that have tuple data store the the actual contents of the tuple in each node.

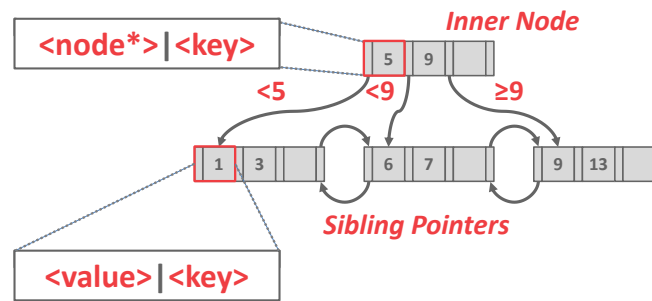


Figure 1: B+ Tree diagram

For inner nodes, the values contain pointers to other nodes, and the keys can be thought of as guide posts. They guide the tree traversal but do not represent the keys (and hence their values) on the leaf nodes. What this means is that you could potentially have a key in an inner node (as a guide post) that is not found on the leaf nodes.

Depending on the index specification, the DBMS will place null keys in either the first leaf node (i.e., NULL FIRST) or the last leaf node (i.e., NULL LAST). Sibling pointers (introduced by B^{link}-Tree) can be added across each level of the B+Tree, enabling range scans to be performed without backtracking and allowing efficient splits and merges during insertions and deletions from the B+Tree.

Insertion

To insert a new entry into a B+Tree, one must traverse down the tree and use the inner nodes to figure out which leaf node to insert the key into.

1. Find correct leaf L .
2. Add new entry into L in sorted order:
 - If L has enough space, the operation is done.
 - Otherwise split L into two nodes L_1 and L_2 . Redistribute entries evenly and copy up the middle key. Insert an entry pointing to L_2 into the parent of L .
 - If the parent overflows, repeat the split step for the parent.
3. To split an inner node, redistribute entries evenly, but push up the middle key.

Deletion

Whereas in inserts we occasionally had to split leaves when the tree got too full, if a deletion causes a tree to be less than half-full, we must merge in order to re-balance the tree.

1. Find correct leaf L .
2. Remove the entry:
 - If L is at least half full, the operation is done.
 - Otherwise, you can try to borrow from a sibling; update the parent key if needed.
 - If borrowing fails, merge L and a sibling. Delete the separator key (and pointer) entry in the parent for the node that was removed.
 - If the parent underflows, repeat the borrow and merge step for the parent.
3. To merge two inner nodes, pull down the key (that separates them) from their parent and merge it with the inner nodes.

Composite Index

The key is composed of multiple attributes. We can create composite index like:

```
CREATE INDEX abc_index ON table (a, b DESC, c NULLS FIRST);
```

Then we can use the index for queries like:

```
SELECT a, b, c FROM table
WHERE a = 1 AND b = 2 AND c = 3;
```

Note that the 'AND' operator is mostly necessary. Conditions with 'OR' are generally not supported.

Selection Conditions

Because B+Trees are in sorted order, look-ups have fast traversal and also do not require the entire key. The DBMS can use a B+Tree index if the query provides any of the attributes of the search key. This differs from a hash index, which requires all attributes in the search key.

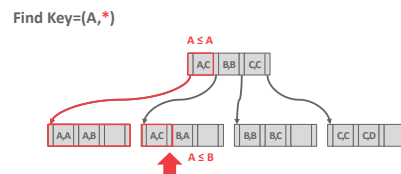


Figure 2: To perform a prefix search on a B+Tree, one looks at the first attribute on the key, follows the path down, and performs a sequential scan across the leaves to find all the keys that one wants.

Duplicate Keys

There are two approaches to duplicate keys in a B+Tree.

The first approach is to *append record IDs* as part of the key. Since each tuple's record ID is unique, this will ensure that all the keys are identifiable. The DBMS can use partial key lookup to find tuples.

The second approach is to allow leaf nodes to spill into *overflow nodes* that contain the duplicate keys. Although no redundant information is stored, this approach is more complex to maintain and modify.

Clustered Indexes

The table is stored in the sorted order specified by the primary key, as either heap- or index-organized storage. Since some DBMSs always use a clustered index, they will automatically make a hidden row id primary key if a table doesn't have an explicit one, but others cannot use them at all.

Index Scan Page Sorting

Since directly retrieving tuples from an unclustered index is inefficient, the DBMS can first figure out all the tuples that it needs and then sort them based on their page id. This way, each page will only need to be fetched exactly once.

3 B+Tree Design Choices

3.1 Node Size

Depending on the storage medium, we may prefer larger or smaller node sizes. For example, nodes stored on hard drives are usually in the order of megabytes in size to reduce the number of seeks needed to find data and amortize the expensive disk read over a large chunk of data, while in-memory databases may use page sizes as small as 512 bytes in order to fit the entire page into the CPU cache as well as to decrease data fragmentation. This choice can also be dependent on the type of workload, as point queries would prefer as small a page as possible to reduce the amount of unnecessary extra info loaded, while a large sequential scan might prefer large pages to reduce the number of fetches it needs to do.

3.2 Merge Threshold

While B+Trees have a rule about merging underfilled nodes after a delete, sometimes it may be beneficial to temporarily violate the rule to reduce the number of deletion operations. For instance, eager merging could lead to thrashing, where a lot of successive delete and insert operations lead to constant splits and merges. It also allows for batched merging where multiple merge operations happen all at once, reducing the amount of time that expensive write latches have to be taken on the tree.

There are merge strategies that keep underfilled nodes in the tree and rebuild the tree later, which make the tree unbalanced (as in Postgres). We will not discuss this in the lecture.

3.3 Variable Length Keys

Currently we have only discussed B+Trees with fixed length keys. However we may also want to support variable length keys, such as the case where a small subset of large keys lead to a lot of wasted space. There are several approaches to this:

1. **Pointers**

Instead of storing the keys directly, we could just store a pointer to the key. Due to the inefficiency of having to chase a pointer for each key, the only place that uses this method in production is embedded devices, where its tiny registers and cache may benefit from such space savings.

2. **Variable Length Nodes**

We could also still store the keys like normal and allow for variable length nodes. This is generally infeasible and largely not used due to the significant memory management overhead of dealing with variable length nodes.

3. **Padding**

Instead of varying the key size, we could set each key's size to the size of the maximum key and pad out all the shorter keys. In most cases this is a massive waste of memory, so you don't see this used by anyone either.

4. **Key Map/Indirection**

The method that nearly everyone uses is replacing the keys with an index to the key-value pair in a separate dictionary. This offers significant space savings and potentially shortcuts point queries (since the key-value pair the index points to is the exact same as the one pointed to by leaf nodes). Some databases (e.g. PostgreSQL) allow overflow within a given node to maintain a fixed number of keys by storing excess data in overflow pages.

3.4 Intra-Node Search

Once we reach a node, we still need to search within the node (either to find the next node from an inner node, or to find our key value in a leaf node). While this is relatively simple, there are still some tradeoffs

to consider:

1. **Linear**

The simplest solution is to just scan every key in the node until we find our key. On the one hand, we don't have to worry about sorting the keys, making insertions and deletes much quicker. On the other hand, this is relatively inefficient and has a complexity of $\mathcal{O}(n)$ per search. This can be vectorized using SIMD (or equivalent) instructions.

2. **Binary**

A more efficient solution for searching would be to keep each node sorted and use binary search to find the key. This is as simple as jumping to the middle of a node and pivoting left or right depending on the comparison between the keys. Searches are much more efficient this way, as this method only has the complexity of $\mathcal{O}(\ln(n))$ per search. However, insertions become more expensive as we must maintain the sort of each node.

3. **Interpolation**

Finally, in some circumstances we may be able to utilize interpolation to find the key. This method takes advantage of any metadata stored about the node (such as max element, min element, average, etc.) and uses it to generate an approximate location of the key. For example, if we are looking for 8 in a node and we know that 10 is the max key and $10 - (n + 1)$ is the smallest key (where n is the number of keys in each node), then we know to start searching 2 slots down from the max key, as the key one slot away from the max key must be 9 in this case. Despite being the fastest method we have given, this method is only seen in academic databases due to its limited applicability to keys with certain properties (like integers) and complexity.

4 Optimizations

4.1 Prefix Compression

Most of the time when we have keys in the same node there will be some partial overlap of some prefix of each key (as similar keys will end up right next to each other in a sorted B+Tree). Instead of storing this prefix as part of each key multiple times, we can simply store the prefix once at the beginning of the node and then only include the unique sections of each key in each slot.

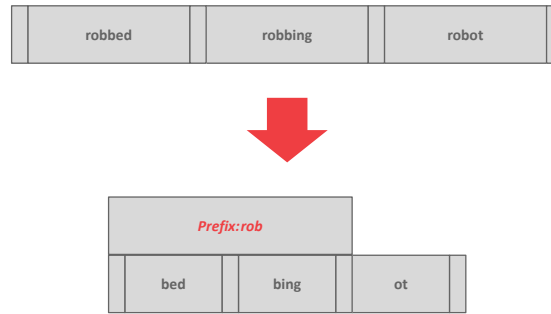


Figure 3: An example of prefix compression. Since the keys are in lexicographic order, they are likely to share some prefix.

4.2 Deduplication

In the case of an index which allows non-unique keys, we may end up with leaf nodes containing the same key over and over with different values attached. One optimization of this could be only writing the key once and then following it with all of its associated values.

4.3 Suffix Truncation

For the most part the key entries in inner nodes are just used as signposts and not for their actual key value (as even if a key exists in the index we still have to search to the bottom to ensure that it hasn't been deleted). We can take advantage of this by only storing the minimum prefix that is needed to correctly route probes into the correct node.

4.4 Pointer Swizzling

Because each node of a B+Tree is stored in a page from the buffer pool, each time we load a new page we need to fetch it from the buffer pool, requiring latching and lookups. To skip this step entirely, we could store the actual raw pointers in place of the page IDs (known as "swizzling"), preventing a buffer pool fetch entirely. Rather than manually fetching the entire tree and placing the pointers manually, we can simply store the resulting pointer from a page lookup when traversing the index normally. Note that we must track which pointers are swizzled and deswizzle them back to page ids when the page they point to is uninned and victimized.

4.5 Bulk Insert

When a B+Tree is initially built, having to insert each key the usual way would lead to constant split operations. Since we already give leaves sibling pointers, initial insertion of data is much more efficient if we construct a sorted linked list of leaf nodes and then easily build the index from the bottom up using the first key from each leaf node. Note that depending on our context we may wish to pack the leaves

as tightly as possible to save space or leave space in each leaf to allow for more inserts before a split is necessary.

4.6 Write-Optimized B+ Tree

Split / merge node operation are expensive. Therefore, some variants of B-Tree, such as B ϵ -Tree, logs changes in the internal node and lazily propagates the updates down to the leaf node later.

Lecture #09: Indexes II

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Index vs. Filters

An **index** is a data structure of a subset of a table's attributes that are organized and/or sorted to the location of specific tuples using those attributes, and it answers the question (where is the data?). While a **filter** is a data structure that answers set membership queries (does this element exist in the set?). If the DBMS knows an element is not in a set, we save time finding it in the set while it does not exist. For example, within a chained hash table, we can put a filter at each bucket pointer. If the filter says negative, we then know the key is not in the chain thus saving our time traversing through the whole chain, which is costly. We need both indexes and filters in a database, and sometimes putting a filter on the index will help speed up operations.

2 Bloom filter

A **Bloom filter** is a probabilistic filter implemented with bitmap. By probabilistic, it means a Bloom filter does not always give the correct answer to a set membership query (false positives). However, a Bloom filter guarantees that it will never has false negatives.

This implies that false positives could happen. The false positive rate can be calculated via Bloom Filter Calculator.

A Bloom filter needs to define

- Size of the bitmap
- Numbers of hash functions to use

Bloom filter API will only support two operations, and it is not possible to delete an element from the filter:

Insert(x)

For insertion, the pre-defined hash functions are used on the inserted element x . For each function's output hash value, we modular it with the bitmap size, then set the corresponding position in the bitmap to one. See Figure 1.

Lookup(x)

For lookup, a similar operation is done on element x . Each hash function takes x as input and modular the output value with bitmap size. If any of the corresponding positions in the bitmap is not one, a false is returned. Otherwise a true is returned (one has to go to the set and see if it's actually in it).

Other Variations

- **Counting Bloom filter:** Instead of bits, use integers to count the number of occurrences of a key in a set. This supports dynamically adding and removing keys.

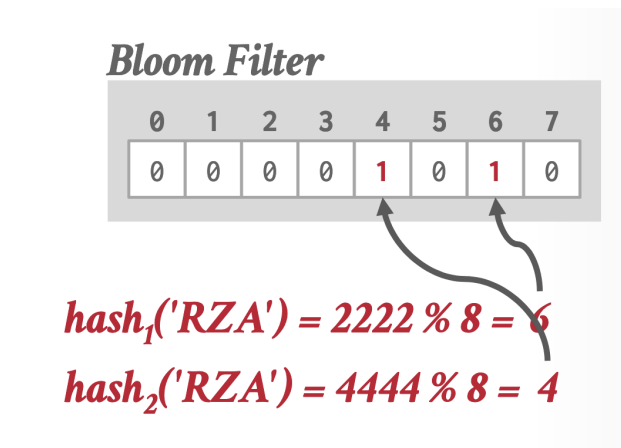


Figure 1: Insert 'RZA' into a Bloom filter with two hash functions

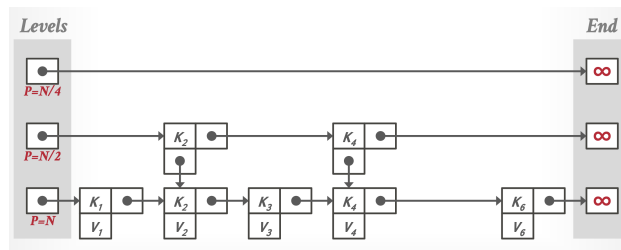


Figure 2: An overview of a skip list

- **Cuckoo filter:** Same idea to Cuckoo Hash, but store fingerprints of elements instead. Also supports dynamically adding and removing keys.
- **Succinct range filter:** An immutable compact trie that supports approximate exact matches and range filtering.

3 Skip List

A *Skip List* uses multiple levels of linked lists to skip some nodes and thus traverse faster. See Figure 2. Each level has $\frac{1}{2}$ the keys of the level below it: The bottom level contains all the keys in sorted order. Second level links every other key, and so on.

Like the B+tree, it stores keys in an ordered manner. However, it does not require rebalancing during insertion or deletion, and still provides $O(\log n)$ approximate search times. It is commonly seen in an in-memory data structure such as memtable.

Find

Go to the top-level linked list and traverse until the value is about to be greater than the target. Then go down to the next level and traverse the same way until it reaches the bottom list and then the target key.

Insert

Coins are flipped to decide until which level this new node is going to insert. Insertions on different levels are done from bottom to top, in order to keep the whole data structure intact. Otherwise, a reader from another thread may come across a node and find out there is no pointer to the lower level while the insertion is still not finished.

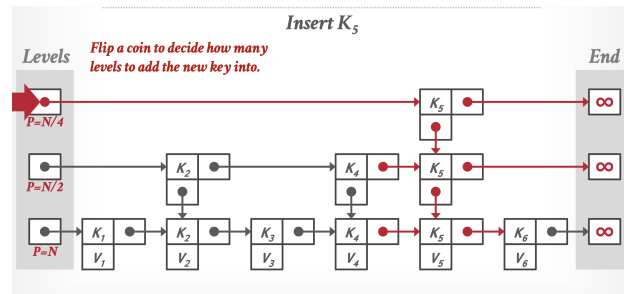


Figure 3: The skip list in Figure 2 after K_5 inserted. And it succeeded in two coin flips.

Note that if the linked list is in one direction, each level's insertion could be done by an atomic pointer in-memory swap (swap K_4 's next with K_5 's next pointer), thus no latch is needed.

Delete

Every node will have a boolean field to mark whether it is deleted or not. At first, a node is marked as deleted instead of removed from the data structure directly to prevent other reader threads from visiting the dead object. Actual deletions are expensive, so it is usually done by a background thread in a fixed interval. Any reader can ignore the deleted node and keep traversing the way down. Later when the node object is to be deleted, the top node will be removed before the bottom one to keep the data structure intact.

Conclusion

Advantages:

- Less memory usage if not including reverse pointer compared to B+tree.
- No rebalancing is needed while inserting and deleting.

Disadvantages:

- Not disk/cache friendly because they do not optimize locality of reference
- Reverse search is non-trivial, it becomes tricky to handle both ascending and descending scans.

4 Trie

Because a B+tree does not provide information about whether a node exists below an inner node or not, it's essential to go down to the leaf node to find out a node does not exist. It costs one buffer pool page miss per tree level.

A *trie* is an order-preserving data structure that stores keys as digits. A trivial way is to make characters (a byte) as digits for strings or bits for other data types. These digits form a tree structure to represent prefixes of every entry inserted into this trie. The tree shape depends on keys and lengths, does not depend on insertion order. And it does not require rebalancing. All operations have $O(k)$ complexity where k is the length of the key.

The span of each trie level is the number of bits that each partial key/digit represents. If the digit and its prefix exist in the corpus, then a pointer to the next level node is stored at that digit, otherwise, a null is stored. So a n -way Trie means each node will have a fan-out of n .

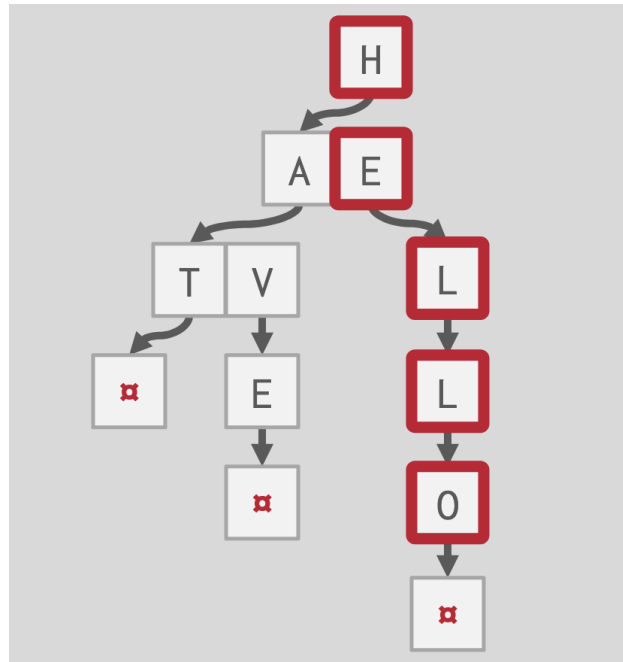


Figure 4: Finding a key "HELLO" in a trie

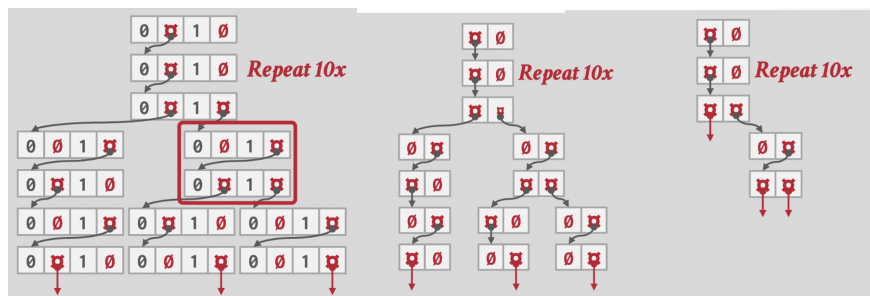


Figure 5: From left to right, horizontal compression and vertical compression are done on a trie with a one-bit span node

Compression

- If we have a known span of a level, we can horizontally compress the node to an array instead of a map. See Figure 5
- If a node has only a single child, we can vertically compress the nodes below the node as there are no branches down there. See Figure 5. This is also called Radix tree. False positives may happen because the full key is no longer embedded in the trie, so DBMS have to check the tuple a node points to see whether a key actually matches.

5 Inverted Index

The indexes we talked about before are only good for point or range searches. They do not support keyword search. For example, a query to find all Wikipedia articles that contain the word "Pavlo".

An inverted index stores an immutable mapping of terms to records (the list of records is called a posting list) that contain those terms in the target attribute.

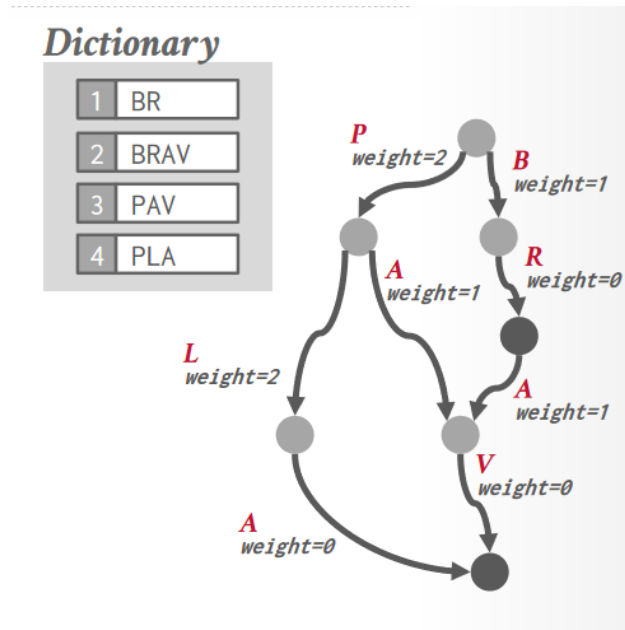


Figure 6: Finite state transducer to find the offset in the dictionary of a term

Lucene Implementation

Lucene is a specialized inverted index engine. The way it stores an inverted index is to have a trie-like data structure called a "finite state transducer" (Figure 6). Instead of storing pointers to a tuple like a trie, it stores weights on every edge. By traversing down to the key one is looking for, a rolling sum of the weights will eventually give the exact position the entry is stored in the mapping.

The data structure is immutable because the weights have been precomputed and fixed to specific dictionary entries. So for a batch of inserts, we will build an immutable transducer. To support incremental updates, a separate transducer is built to hold the newly inserted keys, and there will be a background job that merges transducers together.

In the dictionary of terms, since it's immutable and is built ahead of time, compression techniques such as delta compression, and bit-packing. Pre-aggregation is also supported to accelerate the query time of aggregation queries that group on terms.

PostgreSQL Implementation

PostgreSQL's *Generalized Inverted Index* uses a B+tree to build the term dictionary. The value of this B+tree leaf node depends on the size of the posting list. For small-sized posting lists, it will be a sorted list of record IDs. For large-sized posting lists, additional B+tree structures will be built to hold the record IDs.

A separate pending list is used to avoid small incremental updates. It logs updates and accumulates to a bulk insert to the dictionary.

Enhancements

- **Rankings:** DBMS can rank search results based on the frequency of terms in each record relative to other records.
- **Tokenizers:** Split terms into n-grams to support fuzzy text searches and autocomplete ("did you mean")

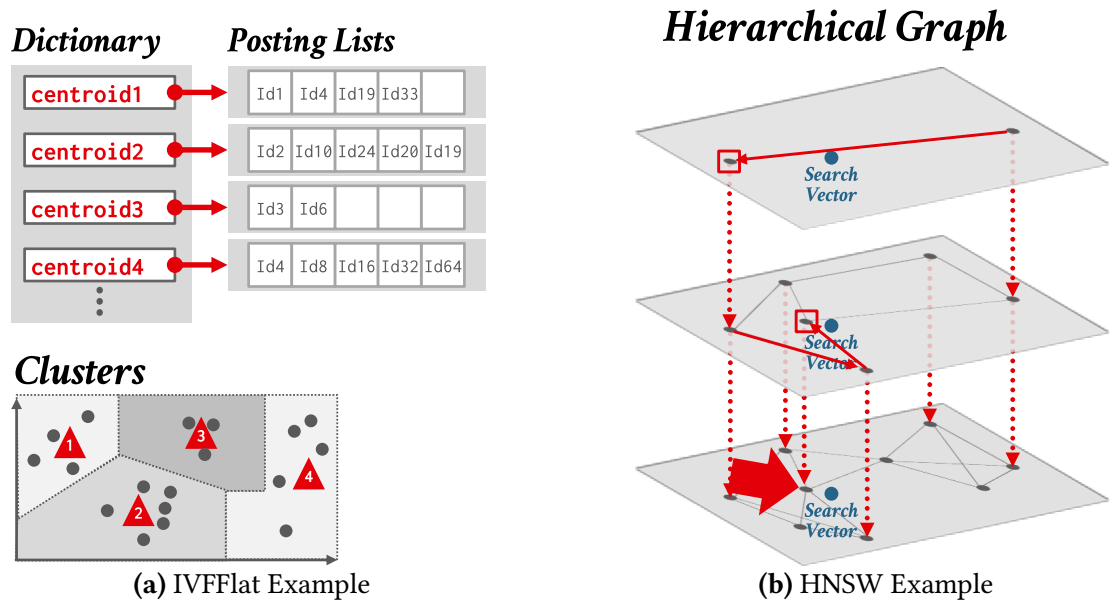


Figure 7: Vector Indexes

6 Vector Index

Inverted indexes can support keyword search, but not the semantic meaning of the content (i.e., the keyword has to be contained in the content exactly). Suppose an application wants to search for records that are **related** to a certain topic, for example, “*hip-hop groups with songs about slinging*”. In that case, we need another method other than the inverted index.

Large language models are known for generating **embeddings** for texts, an one-dimensional array of floating point numbers. Embeddings are geometrically close to each other if they have similar semantic meanings, therefore a vector index specialized for nearest-neighbor search can help in the query we were discussing.

However, there is no correct answer to this kind of query compared to those traditional queries we have seen before. There is also a need to filter data before or after finding nearest neighbors.

Inverted Indexes

Partition vectors into smaller groups by a clustering algorithm and then build an inverted index that maps cluster centroids to records. As shown in Figure 7a, the DBMS can compute k-means clustering on embeddings to find centroids. Then assign each embedding to the cluster with its nearest centroid. To search the nearest neighbors, use the same clustering algorithm to locate the query to a group, then look up all the vectors within that group (might also want to look at the groups nearby to improve accuracy).

Example: IVFFlat.

Preprocessing and quantization can be performed on the index to reduce the dimension and thus speed up the lookup time, while the original vectors are still preserved at their original locations.

Graph

The DBMS builds a graph that represents the neighbor relationship between vectors, where each node represents a vector and its edges link to its n nearest neighbors (Figure 7b). To find a match for a given vector, enter the graph and then greedily choose the next edge that moves closer to that vector. The DBMS

can also use multiple levels of graphs with the idea similar to skip lists to speed up the search.

Example: FAISS, HNSWlib

7 Optimizations

Partial Indexes

Create an index on a subset of the entire table by adding a WHERE clause to the index definition. This potentially reduces its size and the cost of maintaining it. For example, we could partition indexes by date ranges, creating a new index for each month and year.

Index Include Columns

Embed additional columns in indexes to support index-only queries. The extra columns are only stored in the leaf nodes and are not part of the search key. If all attributes a query needs are available in an index, then the DBMS does not need to retrieve the tuple.

Lecture #10: Index Concurrency Control

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Index Concurrency Control

So far, we assumed that the data structures we have discussed are single-threaded. However, most DBMSs need to allow multiple threads to safely access data structures to take advantage of additional CPU cores and hide disk I/O stalls.

There are some systems that use a single-threaded model as some arguments say that it would be faster without latching (e.g. Redis). Additionally, an easy way to convert a single-threaded data structure to a multi-threaded one is to use a single read-write lock to guard the entire structure, but this is not an efficient method.

A *concurrency control* protocol is the method that the DBMS uses to ensure “correct” results for concurrent operations on a shared object from multiple workers. We use the term “workers” here to more generically describe multiple concurrent accessors: some DBMSs use processes (e.g. Postgres) while others use threads, etc.

A protocol’s correctness criteria can vary:

- **Logical Correctness:** This means that the worker is able to read values that it expects to read, e.g. a thread should read back the value it had written previously.
- **Physical Correctness:** This means that the internal representation of the object is sound, e.g. there are no pointers in the data structure that will cause a worker to read invalid memory locations.

For the purposes of this lecture, we only care about enforcing physical correctness. We will revisit logical correctness in later lectures.

2 Locks vs. Latches

There is an important distinction between locks and latches when discussing how the DBMS protects its internal elements.

Locks

A lock is a higher-level, logical primitive that protects the contents of a database (e.g., tuples, tables, databases) from other transactions. Transactions will typically hold a lock for its entire duration. Database systems can expose to the user the locks that are being held as queries are run. There should be some higher-level mechanism to detect deadlocks and rollback changes.

Latches

Latches are the low-level protection primitives used for critical sections the DBMS’s internal data structures (e.g., data structure, regions of memory) from other workers. Latches are held for a short period for a simple operation in a database system (i.e., page latch). Unlike with locks, it is the worker’s responsibility to avoid deadlocks / roll back changes (rather than another mechanism within the system).

There are two modes for latches:

- **READ:** Multiple workers are allowed to read the same item at the same time. A worker can acquire the latch in read mode even if another thread has already acquired it in read mode.
- **WRITE:** Only one worker is allowed to access the item. A worker cannot acquire a write latch if another thread holds the latch in any mode. A worker holding a write latch also prevents other worker from acquiring a read latch.

3 Latch Implementations

There are some key goals we want to achieve when implementing latches:

- Small memory footprint (ideally measured in bytes)
- Fast execution path when there is no contention
- Decentralised management of latches
- Avoid expensive system calls

The underlying primitive that used to implement a latch is through atomic instructions that modern CPUs provide. With this, a thread can check the contents of a memory location to see whether it has a certain value.

- **Atomic Instruction Example: compare-and-swap (CAS)**
Atomic instruction that compares contents of a memory location M to a given value V
 - If values are equal, installs new given value V' in M
 - Otherwise, operation fails

There are several approaches to implementing a latch in a DBMS. Each approach has different trade-offs in terms of engineering complexity and runtime performance. These test-and-set steps are performed atomically (i.e., no other thread can update the value in between the test and set steps).

Test-and-Set Spin Latch (TAS)

Spin latches are a more efficient alternative to an OS mutex as it is controlled by the DBMSs. A spin latch is essentially a location in memory that threads try to update (e.g., setting a boolean value to true). A thread performs CAS to attempt to update the memory location. The DBMS can control what happens if it fails to get the latch. It can choose to try again (for example, using a while loop) or allow the OS to deschedule it. Thus, this method gives the DBMS more control than the OS mutex, where failing to acquire a latch gives control to the OS.

- **Example:** `std::atomic<T>`
- **Advantages:** Latch/unlatch operations are efficient (single instruction to lock/unlock on x86).
- **Disadvantages:** Not scalable nor cache-friendly because with multiple threads, the CAS instructions will be executed multiple times in different threads. These wasted instructions will pile up in high contention environments; the threads look busy to the OS even though they are not doing useful work. Furthermore, in a non-uniform memory architecture polling the memory location of the latch may be especially expensive if it is located in the local cache/memory of another CPU.

Blocking OS Mutex

One possible implementation of latches is the OS built-in mutex infrastructure. Linux provides the futex (fast user-space mutex), which is comprised of (1) a spin latch in user-space and (2) an OS-level mutex. If the DBMS can acquire the user-space latch, then the latch is set. It appears as a single latch to the DBMS even though it contains two internal latches. If the DBMS fails to acquire the user-space latch, then it goes

down into the kernel and tries to acquire a more expensive mutex. If the DBMS fails to acquire this second mutex, the OS de-schedules the thread and notifies the thread when the mutex becomes available.

OS mutex is generally a bad idea inside of DBMSs as it is managed by OS and has large overhead.

- **Example:** `std::mutex`
- **Advantages:** Simple to use and requires no additional coding in DBMS.
- **Disadvantages:** Expensive and non-scalable (about 25 ns per lock/unlock invocation) because of OS scheduling. OS has to do its own bookkeeping with its own latching to manage which threads should be asleep / which should be notified.

Reader-Writer Latches

Mutexes and Spin Latches do not differentiate between reads/writes (i.e., they do not support different modes). The DBMS needs a way to allow for concurrent reads, so if the application has heavy reads it will have better performance because readers can share resources instead of waiting.

A Reader-Writer Latch allows a latch to be held in either read or write mode. It keeps track of how many threads hold the latch and are waiting to acquire the latch in each mode. Reader-writer latches use one of the previous two latch implementations as primitives and have additional logic to handle reader-writer queues, which are queues requests for the latch in each mode. Different DBMSs can have different policies for how it handles the queues.

One thing to notice is that different reader-writer lock implementations have different waiting policies. There are reader-preferred, writer-preferred, and fair reader-writer locks. The behavior differs in different operating systems and pthread implementations.

- **Example:** `std::shared_mutex`
- **Advantages:** Allows for concurrent readers.
- **Disadvantages:** The DBMS has to manage read/write queues to avoid starvation. Larger storage overhead than spin Latches due to additional meta-data.

4 Hash Table Latching

It is easy to support concurrent access in a static hash table due to the limited ways threads access the data structure. For example, all threads move in the same direction when moving from slot to the next (i.e., top-down). Threads also only access a single page/slot at a time. Thus, deadlocks are not possible in this situation because no two threads could be competing for latches held by the other. When we need to resize the table, we can just take a global latch on the entire table to perform the operation.

Latching in a dynamic hashing scheme (e.g., extendible) is a more complicated scheme because there is more shared state to update, but the general approach is the same.

There are two approaches to support latching in a hash table that differ on the granularity of the latches:

- **Page Latches:** Each page/block has its own Reader-Writer latch that protects its entire contents. Threads acquire either a read or write latch before they access a page. This decreases parallelism because potentially only one thread can access a page at a time, but accessing multiple slots in a page will be fast for a single thread because it only has to acquire a single latch.
- **Slot Latches:** Each slot has its own latch. This increases parallelism because two threads can access different slots on the same page. But it increases the storage and computational overhead of accessing the table because threads have to acquire a latch for every slot they access, and each slot has to store data for the latches. The DBMS can use a single mode latch (i.e., Spin Latch) to reduce meta-data and

computational overhead at the cost of some parallelism.

It is also possible to create a latch-free linear probing hash table directly using compare-and-swap (CAS) instructions. Insertion at a slot can be achieved by attempting to compare-and-swap a special “null” value with the tuple we wish to insert. If this fails, we can probe the next slot, continuing until it succeeds.

5 B+Tree Latching

The challenge of B+Tree latching is preventing the two following problems:

- Threads trying to modify the contents of a node at the same time.
- One thread traversing the tree while another thread splits/merges nodes.

Latch crabbing/coupling is a protocol to allow multiple threads to access/modify B+Tree at the same time. The basic idea is as follows:

1. Get latch for the parent.
2. Get latch for the child.
3. Release latch for the parent if the child is deemed “safe”. A “safe” node is one that will not split, merge, or redistribute when updated. In other words, a node is “safe” if
 - **for insertion:** it is not full.
 - **for deletion:** it is more than half full.

Note that read operations do not need to worry about the “safe” condition (as read operations will not change the size of any node in the tree).

Basic Latch Crabbing Protocol:

- **Find:** Start at the root and go down, repeatedly acquire latch on the child and then unlatch parent.
- **Insert/Delete:** Start at the root and go down, obtaining X latches as needed. Once the child is latched, check if it is safe. If the child is safe, release latches on all its ancestors.

The order in which latches are released is not important from a correctness perspective. However, from a performance point of view, it is better to release the latches that are higher up in the tree since they block access to a larger portion of leaf nodes.

Optimistic Latch Crabbing Protocol: The problem with the basic latch crabbing algorithm is that transactions **always** acquire an exclusive latch on the root for every insert/delete operation. This limits parallelism. Instead, one can assume that having to resize (i.e., split/merge nodes) is rare, and thus transactions can acquire shared latches down to the leaf nodes. Each transaction will assume that the path to the target leaf node is safe, and use READ latches and crabbing to reach it and verify. If the leaf node is not safe, then we abort and do the previous algorithm where we acquire WRITE latches.

- **Find:** Same algorithm as before.
- **Insert/Delete:** Set READ latches as if for find, go to leaf, and set WRITE latch on leaf. If the leaf is not safe, release all previous latches, and restart the transaction using the original Insert/Delete protocol.

Some latch implementations allow for upgrading a read latch to a write latch while holding the latch. In this way it may be possible to not restart the entire traversal of the B+tree when a leaf is not safe, but it is less straightforward and possibly requires already holding some ancestor read latches.

Leaf Node Scans

The threads in these protocols acquire latches in a “top-down” manner. This means that a thread can only acquire a latch from a node that is below its current node. If the desired latch is unavailable, the thread

must wait until it becomes available. Given this, there can never be deadlocks.

However, leaf node scans are susceptible to deadlocks because now we have threads trying to acquire exclusive locks in two different directions at the same time (e.g., thread 1 tries to delete, thread 2 does a leaf node scan). Index latches do not support deadlock detection or avoidance.

Thus, the only way programmers can deal with this problem is through coding discipline. The leaf node sibling latch acquisition protocol must support a “no-wait” mode. That is, the B+tree code must cope with failed latch acquisitions. Since latches are intended to be held (relatively) briefly, if a thread tries to acquire a latch on a leaf node but that latch is unavailable, then it should abort its operation (releasing any latches that it holds) quickly and restart the operation. Some trade-off could be made here is to wait a little more before restarting itself, the wait time can depend on how much work has been done so far for the thread.

In the case the index is a high-contention data structure where a leaf scan thread could possibly fail consistently, should there be a mechanism to solve this, a higher level system thread scheduler detecting this and trying to schedule out other threads to get the leaf scan thread done. This mechanism does not necessarily need to be incorporated in a data structure’s implementation but should rely on a higher level part of the system.

Lecture #11: Sorting & Aggregation Algorithms

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Query Plan

Up to this point we have talked about access methods. Now we need to actually execute queries.

The database system will compile SQL into a query plan, which is a tree or DAG of operators. We will cover more details about operators etc. in later lectures.

For a disk-oriented database system, we will use the buffer pool to implement algorithms that need to spill to disk. We want to minimize I/O for an algorithm and prefer sequential over random I/O.

2 Sorting

Assumptions.

1. Query Results may not fit in memory.
2. Algorithms that maximize sequential I/O are faster.

Why Sort?

DBMSs need to sort because tuples in a table have no specific order under the relational model, so ORDER BY requires it. Also, operators like GROUP BY, JOIN, and DISTINCT potentially use sorting.

If the data that needs to be sorted fits in memory, then the DBMS can use a standard sorting algorithm (e.g., quicksort, mergesort). If the data does not fit, then the DBMS needs to use external sorting that keeps in mind the cost of disk operations.

The sorting algorithm sorts a given input *run* (a list of key-value pairs) and sort it based on comparison function and sorting parameters. The key is the attributes to compare to compute the sort order, and the value could be the whole tuple (*early materialization*) or the record ID only (*late materialization*).

External Merge Sort

This is the standard algorithm for sorting data which is too large to fit in memory. It is a divide-and-conquer sorting algorithm that splits the data set into separate *runs* and then sorts them individually. It can spill runs to disk as needed then read them back in one at a time. The algorithm is comprised of two phases:

- **Phase #1 – Sorting:** First, the algorithm sorts small chunks of data that fit in main memory. Sorted chunks are written to disk to make room to load more data into memory to sort.
- **Phase #2 – Merge:** Then, the algorithm combines the sorted runs into larger sorted runs.

A sorted run can be early-materialized, which means that the entire tuple is stored in the pages, or can be late-materialized, which means we only store record IDs in the pages and read them later.

Two-way Merge Sort: The most basic version of the external merge sort algorithm is the two-way merge sort. During the sorting phase, the algorithm reads each page, sorts it, and writes the sorted version back to disk. Then, in the merge phase, it uses three buffer pages. It reads two sorted pages in from disk, and merges them together into a third buffer page. Whenever the third page fills up, it is written back to disk and replaced with an empty page. Each set of sorted pages is called a *run*. The algorithm then recursively merges the runs together.

Let N be the total number of data pages. The algorithm makes

$$1 + \lceil \log_2 N \rceil$$

total passes through the data (1 for the first sorting step then

$$\lceil \log_2 N \rceil$$

for the recursive merging). The total I/O cost is $2N \times (\# \text{ of passes})$ since each pass performs a read and a write for each page.

General (K -way) Merge Sort: The generalized version of the algorithm allows the DBMS to take advantage of more than three buffer pages. Let B be the total number of buffer pages available. Then, during the sort phase, the algorithm can read and sort B pages at a time, so the DBMS writes $\lceil \frac{N}{B} \rceil$ sorted runs of length B pages back to disk. The merge phase can combine up to $B - 1$ runs in each pass, using $B - 1$ buffer pages to step through the runs and using one buffer page for the combined data, writing back to disk as needed.

In the generalized version, the algorithm performs

$$1 + \left\lceil \log_{B-1} \left\lceil \frac{N}{B} \right\rceil \right\rceil$$

passes (one for the sorting phase and

$$\left\lceil \log_{B-1} \left\lceil \frac{N}{B} \right\rceil \right\rceil$$

for the merge phase. Then, the total I/O cost is $2N \times (\# \text{ of passes})$ since it again has to make a read and write for each page in each pass.

Double Buffering Optimization: One optimization for external merge sort is prefetching the next run in the background and storing it in a second set of buffers while the system is processing the current run. This reduces the wait time for I/O requests at each step by continuously utilizing the disk. However, it reduces effective buffers available by half. This optimization requires the use of multiple threads, since the prefetching should occur while the computation for the current run is happening.

Comparison Optimizations: Comparing keys in the sorting algorithm can be expensive. DBMSs can implement different optimizations to speed up this process:

- **Code Specialization:** Instead of providing the comparator as a function pointer to the sorting algorithm, the sorting function can be hard-coded to the specific key type. An example of this is template specialization in C++.
- **Suffix Truncation:** Another optimization for string-based comparisons is only compare part of keys first. This first compares binary prefixes of long VARCHAR keys, falling back to an entire string comparison if the prefixes are equal.
- **Key Normalization:** This converts variable-length keys into a single encoded / padded fixed-length string that preserves sort order.

Top-N Heap Sort

If a query contains an ORDER BY with a LIMIT and an optional WITH TIES, then to find the top-N elements, the DBMS only needs to scan the data once. During the scan, the DBMS can maintain a priority queue storing the top-N elements seen so far. If the query contains WITH TIES, the priority queue would be extended on-demand to keep the ties. The ideal scenario is when the top-N elements fit in memory, so the DBMS can maintain the entire priority queue in-memory.

Using B+Trees

It is sometimes advantageous for the DBMS to use an existing B+tree index to aid in sorting rather than using the external merge sort algorithm. Some DBMSs support prefix key scans for sorting. In particular, if the index is a clustered index, the DBMS can just traverse the B+tree. Since the index is clustered, the I/O access will be sequential. This means it is always better than external merge sort since no computation is required. On the other hand, if the index is unclustered, traversing the tree is almost always worse, since each record could be stored in any page, so nearly all record accesses will require a disk read. The only exception might be Top-N queries, when N is small relative to the total number of tuples in a table.

3 Aggregations

An aggregation operator in a query plan collapses the values of one or more tuples into a single scalar value. There are two approaches for implementing an aggregation: (1) sorting and (2) hashing.

Sorting

The DBMS first sorts the tuples on the GROUP BY key(s). It can use either an in-memory sorting algorithm if everything fits in the buffer pool (e.g., quicksort) or the external merge sort algorithm if the size of the data exceeds memory. The DBMS then performs a sequential scan over the sorted data to compute the aggregation. The output of the operator will be sorted on the keys.

When performing sorting aggregations, it is important to order the query operations to maximize efficiency. For example, if the query requires a filter, it is better to perform the filter first and then sort the filtered data to reduce the amount of data that needs to be sorted.

Hashing

Hashing can be computationally cheaper than sorting for computing aggregations, especially when the order of the output does not matter. If the hash table fits in memory, build an ephemeral hash table while scanning the table. For each record, check whether there is already an entry in the hash table and perform the appropriate modification. If the hash table is too large, then the DBMS has to spill it to disk and do *External Hashing Aggregate*. This is a divide-and-conquer approach to compute the aggregation, with two phases:

- **Phase #1 – Partition:** Use a hash function h_1 to split tuples into partitions on disk based on the target hash key. This will put tuples that must be compared for the aggregation into the same partition. Assume B buffer pages in total. We will have $B-1$ output buffer pages for partitions and 1 buffer page for input data. If any partition is full, the DBMS will spill it to disk. Thus, this phase results in $B - 1$ partitions.
- **Phase #2 – ReHash:** For each partition on disk, read its pages into memory and build an in-memory hash table based on a second hash function h_2 (where $h_1 \neq h_2$). This will put all tuples that match into the same bucket. Then go through each bucket of this hash table to bring together matching tuples to compute the aggregation. This assumes that each partition fits in memory, which can be

achieved by recursively partitioning until this is true.

During the ReHash phase, the DBMS can store pairs of the form (GroupByKey→RunningValue) to compute the aggregation. The contents of RunningValue depends on the aggregation function (e.g. (COUNT, SUM) for AVG). To insert a new tuple into the hash table:

- If it finds a matching GroupByKey, then update the RunningValue appropriately.
- Else insert a new (GroupByKey→RunningValue) pair.

The choice between sorting and hashing aggregate is subtle and depends on optimizations done in each case, but in general hashing is often more efficient unless the data is already sorted beforehand or the output is required to be sorted (e.g. following ORDER BY).

4 Exercises

1. Suppose a point-query heavy workload is run a DBMS that currently implements vanilla merge sort. Do you expect speedup from switching to 2-way external merge sort?
2. Consider Top-N heap sort, B+ Trees, and K-way merge sort. For each sorting algorithm, describe a scenario where it is clearly preferable to the others.
3. (Textbook Practice Exercise 15.9) What is the effect on the cost of merging runs if the number of buffer blocks per run is increased while overall memory available for buffering runs remains fixed?
4. (Textbook 15.17) Suppose you need to sort a relation of 40 gigabytes, with 4-kilobyte blocks, using a memory size of 40 megabytes. Suppose the cost of a seek is 5 milliseconds, while the disk transfer rate is 40 megabytes per second.
 - (a) Find the cost of sorting the relation, in seconds, with $b_b = 1$ and with $b_b = 100$. Note b_b is the size of the output buffer.
 - (b) In each case, how many merge passes are required?
 - (c) Suppose a flash storage device is used instead of a disk, and it has a latency of 20 microsecond and a transfer rate of 400 megabytes per second. Recompute the cost of sorting the relation, in seconds, with $b_b = 1$ and with $b_b = 100$, in this setting.

Lecture #12: Joins Algorithms

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Introduction

The goal of a good database design is to minimize unnecessary repetition of information. This is why tables are often composed based on *normalization theory*. Joins are then needed to reconstruct the original tables.

We focus on **inner equijoin** algorithms for combining two tables. An *equijoin* algorithm joins tables where keys are equal. These algorithms can be tweaked to support other joins. For binary joins, we often prefer the left table (the "outer table") to be the smaller one of the two. Multi-way joins exist primarily in research literature (e.g., worst-case optimal joins).

2 Join Operators

In a query plan, the operators are arranged in a directed tree where the data flows from the leaves towards the root. The output of the root node is the result of the query. The following subsections will discuss the decisions for designing any join operator.

Operator Output

For a tuple $r \in R$ and a tuple $s \in S$ that match on join attributes, the join operator concatenates r and s together into a new output tuple.

In reality, the content of output tuples generated by a join operator varies. It depends on the DBMS's query processing model, storage model, and the query itself. There are multiple approaches to designing the join operator's output.

- **Data approach:** The join creates new output tuples containing copies of the values in the joined tables. These tuples are put into an intermediate result table for the operator. The advantage of this approach is that future query plan operators never need to return to the base tables to get data. This is more useful in the case of row stores since all the tuple data will be brought into memory anyway. The disadvantage is that this requires more memory to materialize the entire tuple. This is called *early materialization*.
The DBMS can also do additional computation and omit attributes that will not be needed later in the query to optimize this approach further.
- **Record id approach:** The DBMS only copies the join keys and the matching tuple record ids. This approach is ideal for column stores since the join can avoid bringing additional columns into memory to copy them. This is called *late materialization*.

In the real world, DBMS often mix and match between the two approaches, wherein based on the number of tuples, columns, and further operators, a combination of the two strategies can be taken.

Cost Analysis

The cost metric used here to analyze different join algorithms will be the number of disk I/Os used to compute the join. For now, compute and network costs will be ignored, as I/O costs dominate in disk-based DBMSs.

Note that I/Os needed to output the join result are not considered in this analysis. This is because the output cost depends on the data and will be the same for all join algorithms.

Given a query that joins table R with table S , assume the DBMS knows the following about those tables:

- M pages in table R (Outer Table), m tuples total
- N pages in table S (Inner Table), n tuples total

Also, note that $R \bowtie S$ (the natural join of tables R and S) is the most common operation and must be carefully optimized. One inefficient algorithm is to compute $R \times S$ (the cross product of tables R and S) and select relevant tuples. However, the cross-product is huge and results in a very inefficient approach.

In general, there will be many algorithms and optimizations which can reduce join costs in some cases, but no single algorithm works well in every scenario.

3 Nested Loop Join

At a high level, this join algorithm contains two nested for loops that iterate over the tuples in both tables and perform a pairwise comparison. If the tuples match the join predicate, then it outputs them. The table in the outer for loop is called the *outer table*, while the table in the inner for loop is called the *inner table*.

The DBMS will always want to use the “smaller” table as the outer table. Smaller generally is based on the number of pages but can sometimes mean tuples as well. We will buffer as much of the outer table in memory as possible, and use indexes where available.

Naïve Nested Loop Join

Compare each tuple in the outer table with each tuple in the inner table. This is the worst-case scenario where the DBMS must scan the entirety of the inner table for each tuple in the outer table without any caching or access locality.

Cost: $M + (m \times N)$

Block Nested Loop Join

For each block in the outer table, fetch each block from the inner table and compare all the tuples in those two blocks. Based on the # of pages and not the # of tuples, the smaller table should be the outer table. This algorithm performs fewer disk accesses because the DBMS scans the inner table for every outer table block instead of for every tuple.

Cost: $M + (M \times N)$

Buffer Pool Usage: If the DBMS has B buffer pool frames available to compute the join, it can use $B - 2$ buffers to scan the outer table. It will use one buffer to scan the inner table and one buffer to store the join's output.

Cost: $M + \left(\left\lceil \frac{M}{B-2} \right\rceil \times N \right)$

Index Nested Loop Join

The previous nested loop based algorithms perform poorly because the DBMS has to do a sequential scan to check for a match in the inner table. However, if the database already has an index for one of the tables on the join key, it can use that to speed up the comparison. The DBMS can use an existing index or build a temporary one for the join operation.

The inner table will be the one that requires an index. Assume the cost of each index probe is some constant value C per tuple and the DBMS does a sequential scan on the outer table.

Cost: $M + (m \times C)$

4 Sort-Merge Join

At a high level, a sort-merge join sorts the two tables on their join key(s). The DBMS can use the external mergesort algorithm for this. It then steps through each table with cursors and emits matches (like in Mergesort).

This algorithm is useful if one or both tables are already sorted on join attribute(s) (like with a clustered index) or if the output needs to be sorted on the join key anyway.

The worst-case scenario for this algorithm is if the join attribute for all the tuples in both tables contains the same value, which is very unlikely to happen in real databases. In this case, the cost of merging would be $M \cdot N$ since, for each outer page, we will have to match the entire inner table. Most of the time, though, the keys are mostly unique, so the merge cost can be assumed to be approximately $M + N$.

Assume that the DBMS has B buffers to use for the algorithm:

- Sort cost for table R : $2M \times (1 + \lceil \log_{B-1} \lceil \frac{M}{B} \rceil \rceil)$
- Sort cost for table S : $2N \times (1 + \lceil \log_{B-1} \lceil \frac{N}{B} \rceil \rceil)$
- Merge cost: $M + N$
- Worst-Case Merge Cost: $M \times N$

Total Cost: Sort + Merge

5 Hash Join

Hash join algorithms use a hash table to split the tuples into smaller chunks based on their join attribute(s). This reduces the number of comparisons that the DBMS needs to perform per tuple to compute the join. Hash joins can only be used for equi-joins on the complete join key.

If a tuple $r \in R$ and a tuple $s \in S$ satisfy the join condition, then they have the same value for the join attributes. If that value is hashed to some value i , the R tuple has to be in bucket r_i , and the S tuple has to be in bucket s_i . Thus, the R tuples in bucket r_i need only to be compared with the S tuples in bucket s_i .

Basic Hash Join

- **Phase #1 – Build:** First, scan the outer relation and populate a hash table using the hash function h_1 on the join attributes. The key in the hash table is the join attributes. The value depends on the implementation, and can be a full tuple or a record ID (early vs. late materialization).
- **Phase #2 – Probe:** Scan the inner relation and use the hash function h_1 on each tuple's join attributes to jump to the corresponding location in the hash table and find matching tuples. Since there may be collisions in the hash table, the DBMS must examine the original values of the join attribute(s) to determine whether tuples satisfy the join condition. The probe table can be of any size, with only the build side needing to fit in memory.

One optimization for the probe phase is the usage of a **Bloom Filter**, which is created as the DBMS builds the hash table on the outer table in the first phase. This is a probabilistic data structure that can fit in CPU caches and answer the question “*is key x in the hash table?*” with either *definitely no* or *probably yes*. It reduces the amount of disk I/O by preventing disk reads that will not result in a match. Such techniques of providing extra metadata are called **sideways information passing**.

If the DBMS knows the size of the outer table, the join can use a static hash table. If it does not know the size, the join must use a dynamic hash table or allow for overflow pages.

Grace Hash Join / Partitioned Hash Join

When the tables do not fit in the main memory, the DBMS has to swap tables in and out at random, leading to poor performance and not leveraging the buffer pool. The Grace Hash Join is an extension of the basic hash join that also hashes the inner table into partitions that are written out to disk.

- **Phase #1 – Build:** First, scan both the outer and inner tables and populate a hash table using the hash function h_1 on the join attributes. The hash table’s buckets are written out to disk as needed. If a single bucket does not fit in memory, the DBMS can use *recursive partitioning* with different hash function h_2 (where $h_1 \neq h_2$) to further divide the bucket. This can continue recursively until the buckets fit into memory.
- **Phase #2 – Probe:** For each bucket level, retrieve the corresponding outer and inner table pages. Then, perform a hash join on their contents.

Partitioning phase cost: $2 \times (M + N)$

Probe phase cost: $M + N$

Total Cost: $3 \times (M + N)$

One edge case is that if a single join key has too many matching records that do not fit in memory, we can use a **block nested loop join** just for that key, which avoids random I/O in exchange for sequential I/O.

Hybrid hash join optimization: adapts between basic hash join and Grace hash join; if the keys are skewed, keep the hot partition in memory and immediately perform the comparison instead of spilling it to disk. Difficult to implement correctly and rarely done in practice.

6 Conclusion

Joins are an essential part of relational databases, and it’s critical to have efficient algorithms for them.

Algorithm	I/O Cost	Example
Simple Nested Loop Join	$M + (m \cdot N)$	1.4 hours
Block Nested Loop Join	$M + \left(\left\lceil \frac{M}{B-2} \right\rceil \cdot N\right)$	6.5 seconds
Index Nested Loop Join	$M + (m \cdot C)$	Varies
Sort-Merge Join	$M + N + (\text{sort cost})$	0.75 seconds
Hash Join	$3 \cdot (M + N)$	0.45 seconds

Figure 1: The table above assume the following: $M = 1000, m = 100000, N = 500, n = 40000, B = 100$ and 0.1 ms per I/O. Sort cost is $R + S = 4000 + 2000$ IOs, where $R = 2 \cdot M \cdot (1 + \lceil \log_{B-1} \lceil M/B \rceil \rceil) = 2000 \cdot (1 + \lceil \log_{99} \lceil 1000/100 \rceil \rceil) = 4000$ and $S = 2 \cdot N \cdot (1 + \lceil \log_{B-1} \lceil N/B \rceil \rceil) = 1000 \cdot (1 + \lceil \log_{99} \lceil 500/100 \rceil \rceil) = 2000$.

Hash joins are almost always better than sort-based join algorithms, but there are cases in which sort-based

joins would be preferred. This includes queries on non-uniform data, when the data is already sorted on the join key, or when the result needs to be sorted. Good DBMSs will use either or both.

Lecture #13: Query Processing I

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Query Plan

The DBMS converts a SQL statement into a query plan. Operators in the query plan are arranged in a DAG, oftentimes a tree. Data flows from the leaves of this tree towards the root. The output of the root node in the tree is the result of the query. Typically operators are unary or binary (1 to 2 children). The same query plan can be executed in multiple ways.

A **pipeline** refers to a sequence of operators where tuples can continuously flow in-between each pair without intermediate storage. A **pipeline breaker** is an operator that cannot finish until all of its children have emit their tuples. Some examples include Joins (build Side), Subqueries and Order By.

2 Processing Models

A DBMS's *processing model* defines how the system executes a query plan. It specifies things like the direction in which the query plan is evaluated and what kind of data is passed between operators along the way. There are several types of processing models with trade-offs for different workloads (For eg: OLTP vs OLAP).

Each processing model is comprised of two types of execution paths: control flow and data flow. **control flow** dictates how the DBMS invokes the operator, while **data flow** decides how an operator sends its results. An operator's output can be either whole tuples (NSM) or subsets of columns (DSM).

The three execution models that we consider are:

- Iterator Model
- Materialization Model
- Vectorized / Batch Model

Iterator Model

The *iterator model*, also known as the Volcano or Pipeline model, is the most common processing model. It is used by almost every (row-based) DBMS.

The iterator model implements a *Next* function for every operator. Nodes in the query plan, except for leaf nodes, call *Next* on its children. When called, children nodes emit tuples one by one to their parent nodes until there's none. Each tuple is then processed and passed up the plan as far as possible before the next one is retrieved. This design is efficient for disk-based systems, as it maximizes use of in-memory data before accessing new tuples or pages. A sample diagram is shown in Figure 1.

Query plan operators in an iterator model are highly composable and easy to reason about. This is because each operator can be implemented independently from their parent or child operators in the query plan tree as long as it implements a *Next* function as follows:

- On each call to *Next*, the operator returns either a single tuple or a null marker if there are no more tuples to emit.

- The operator implements a loop that calls Next on its children to retrieve their tuples and then process them. In this way, one node's Next calls will trigger Next calls on its child. In response, the child node will return the next tuple that the parent must process.

The iterator model allows for *pipelining* where the DBMS can process a tuple through as many operators as possible before having to retrieve the next tuple. The series of tasks performed for a given tuple in the query plan is called a *pipeline*.

Some operators will block until children emit all of their tuples. Examples of such operators include joins, subqueries, and ordering (ORDER BY). Such operators are known as *pipeline breakers*.

Output control works easily with this approach (LIMIT) because an operator can stop invoking Next on its child (or children) operator(s) once it has all the tuples that it requires.

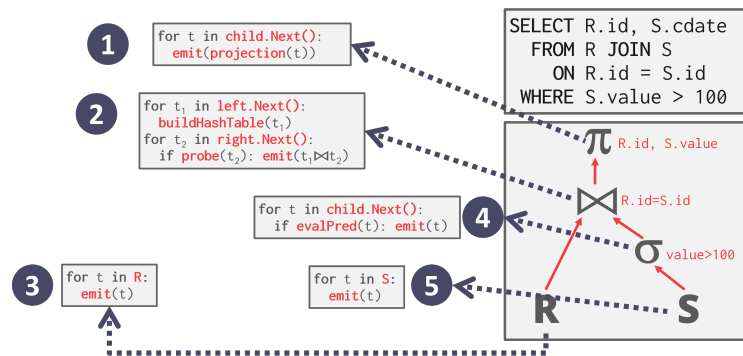


Figure 1: Iterator Model Example – Pseudo code of the different Next functions for each of the operators. The Next functions are essentially for-loops that iterate over the output of their child operator. For example, the root node calls Next on its child, the join operator, which is an access method that loops over the relation R and emits a tuple up that is then operated on. After all tuples have been processed, a null pointer (or another indicator) is sent that lets the parent nodes know to move on.

Materialization Model

The *materialization model* is a specialization of the iterator model where each operator processes its input all at once and then emits its output all at once. Instead of having a next function that returns a single tuple, each operator returns all of its tuples every time it is reached. To avoid scanning too many tuples, the DBMS can propagate down information about how many tuples are needed for subsequent operators (e.g. LIMIT). The operator “materializes” its output as a single result. The output can be either a whole tuple (NSM) or a subset of columns (DSM). A diagram of the materialization model is shown in Figure 2.

Every query plan operator implements an Output function:

- The operator processes all the tuples from its children at once.
- The return result of this function is all the tuples that operator will ever emit. When the operator finishes executing, the DBMS never needs to return to it to retrieve more data.

The materialization model is better for OLTP workloads where queries typically only access a small number of tuples at a time. Thus, there are fewer function calls to retrieve tuples. It is however not suited for OLAP queries with large intermediate results because the DBMS may have to spill those results to disk between operators.

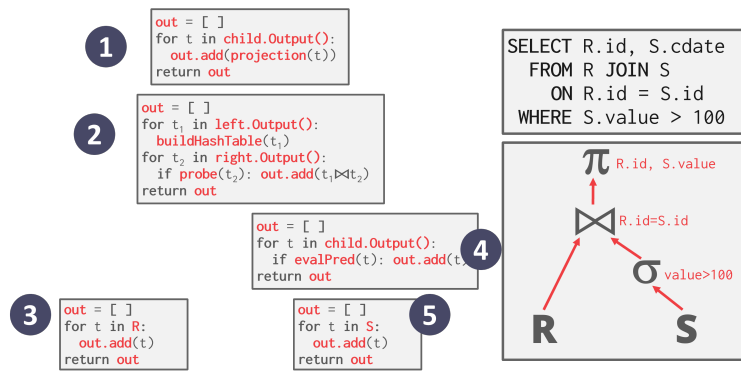


Figure 2: Materialization Model Example – Starting at the root, the `child.Output()` function is called, which invokes the operators below, which returns all tuples back up.

Vectorization Model

Like the iterator model, each operator in the *vectorization model* implements a `Next` function. However, each operator emits a *batch* (i.e. vector) of data instead of a single tuple. The operator's internal loop implementation is optimized for processing batches of data instead of a single item at a time. The size of the batch can vary based on hardware or query properties. See Figure 3 for an example of the vectorization model.

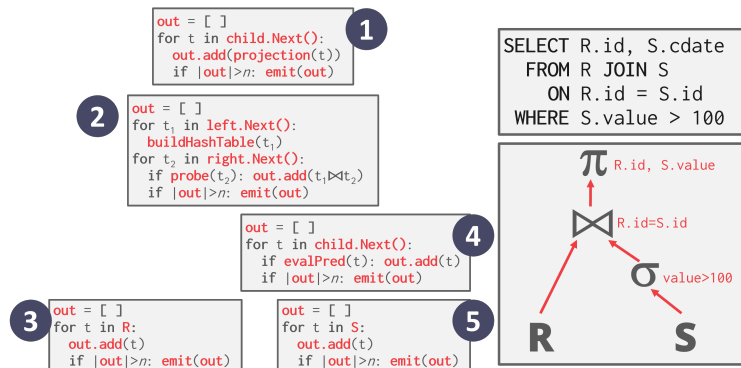


Figure 3: Vectorization Model Example – The vectorization model is very similar to the iterator model except at every operator, an output buffer is compared to the desired emission size. If the buffered output size is larger than the emission size, the buffered tuple batch will be sent up.

The vectorization model approach is ideal for OLAP queries that have to scan a large number of tuples because there are fewer invocations of the `Next` function.

The vectorization model allows operators to more easily use vectorized (SIMD) instructions to process batches of tuples. Modern out-of-order CPUs also work efficiently processing tuple batches. This is due to the operators working in tight for-loops over arrays of equal-sized items and having no data or control dependencies.

Processing Direction

The models are implemented to invoke the operators either from **top-to-bottom (pull)** or from **bottom-to-top (push)**. Although the top-to-bottom approach is much more common, the bottom-to-top approach can allow for tighter control of caches/registers in *pipelines*.

- **Approach #1: Top-to-Bottom**

- Start with the root and “pull” data from children to parents
- Tuples are always passed with function calls
- Easy to control via LIMIT
- Parent operator has to block until its child returns with a tuple
- Additional overhead from Next functions implemented as virtual functions
- Each Next call can involve branching costs

- **Approach #2: Bottom-to-Top**

- Start with leaf nodes and “push” data from children to parents
- Allows for tighter control of caches / registers in operator pipelines
- Might not have exact control of intermediate result sizes
- Some iterator might be hard to implement in this design.

3 Access Methods

An *access method* is how the DBMS accesses the data stored in a table. These methods do not have corresponding operators defined in relational algebra because physical retrieval of data is not represented in relational algebra. There are three basic approaches: sequential scan, index scan (with many variants), and multi-index scans.

Sequential Scan

The sequential scan operator iterates over every page in the table and retrieves it from the buffer pool. As the scan iterates over all the tuples on each page, it evaluates the predicate to decide whether or not to emit the tuple to the next operator. The DBMS maintains an internal cursor to track the last examined page/slot.

A sequential table scan is almost always the least efficient method by which a DBMS may execute a query. There are a number of optimizations available to help make sequential scans faster:

- **Compression:** Compression schemes such as RLE (run length encoding) can help retrieve multiple tuples in a single fetch.
- **Prefetching:** Fetch the next few pages in advance so that the DBMS does not have to block on storage I/O when accessing each page.
- **Buffer Pool Bypass:** The scan operator stores pages that it fetches from disk in its local memory instead of the buffer pool in order to avoid sequential flooding.
- **Parallelization:** Execute the scan using multiple threads/processes in parallel.
- **Late Materialization:** DSM DBMSs can delay stitching together tuples until reaching the upper parts of the query plan. This allows each operator to pass the minimal amount of information needed to the next operator (e.g. record ID, offset to record in column). This is only useful in column-store systems.
- **Heap Clustering:** Tuples are stored in the heap pages using an order specified by a clustering index.
- **Result Caching/Materialized Views:** Storing (caching) the results of subquery/query which are more frequently occurring.
- **Code specialization/compilation:** Pre compiling functions ahead of time (JIT) in order to obtain

results faster when it's actually required.

- **Approximate Queries (Lossy Data Skipping):** Execute queries on a sampled subset of the entire table to produce approximate results. This is typically done for computing aggregations in a scenario that allow a low error to produce a nearly accurate answer.
- **Zone Map (Lossless Data Skipping):** Pre-compute aggregations for each tuple attribute in a page. The DBMS can then decide whether it needs to access a page by checking its Zone Map first. The Zone Maps for each page are stored in separate pages and there are typically multiple entries in each Zone Map page. Thus, it is possible to reduce the total number of pages examined in a sequential scan. Zone maps are particularly valuable in the cloud database systems where data transfer over a network incurs a bigger cost. See Figure 4 for an example of a Zone Map.

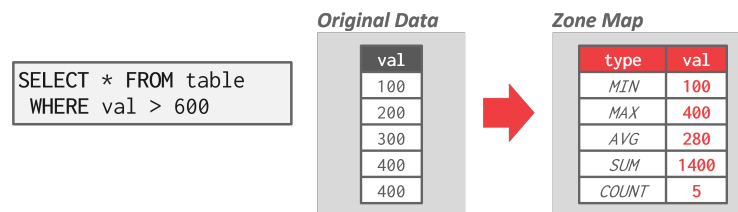


Figure 4: Zone Map Example – The zone map stores pre-computed aggregates for values in a page. In the example above, the select query realizes from the zone map that the max value in the original data is only 400. Then, instead of having to iterate through every tuple in the page, the query can avoid accessing the page at all since none of the values will be greater than 600.

The limitations of the sequential model include function overhead and lack of parallelization (e.g. not able to take advantage of vector operations).

Index Scan

In an *index scan*, the DBMS picks an index to find the tuples that a query needs.

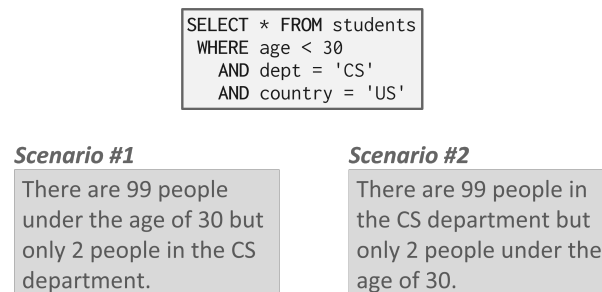


Figure 5: Index Scan Example – Consider a single table with 100 tuples and two indexes: age and department. In the first scenario, it is better to use the department index in the scan because it only has two tuples to match. Choosing the age index would not be much better than a simple sequential scan. In the second scenario, the age index would eliminate more unnecessary scans and is the optimal choice.

There are many factors involved in the DBMSs' index selection process, including:

- What attributes the index contains;
- What attributes the query references;

- The attribute's value domains;
- Predicate composition;
- Whether the index has unique or non-unique keys.

A simple example of an index scan is shown in Figure 5.

Multi Index Scan

More advanced DBMSs support multi-index scans. When using multiple indexes for a query, the DBMS computes sets of record IDs using each matching index, combines these sets based on the query's predicates, and retrieves the records and apply any predicates that may remain. The DBMS can use bitmaps, hash tables, or Bloom filters to compute record IDs through set intersection. See Figure 6 for an example that makes use of a multi-index scan.

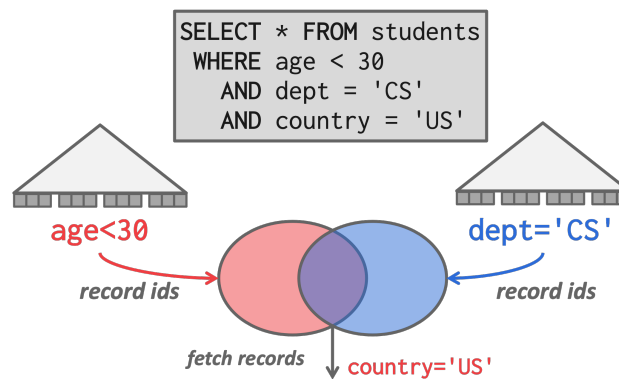


Figure 6: Multi-Index Scan Example – Consider the same table in Figure 5. With multi-index scan support, we first compute the sets of record IDs satisfying the predicate for age and dept respectively, using the corresponding index. We then compute the intersection of the two sets, fetch the corresponding records, and apply the remaining predicate `country='US'`.

4 Modification Queries

Operators that modify the database (INSERT, UPDATE, DELETE) are responsible for checking constraints and updating indexes. For UPDATE/DELETE, child operators pass Record IDs for target tuples and must keep track of previously seen tuples.

There are two implementation choices on how to handle INSERT operators:

- **Choice #1:** Materialize tuples inside of the operator.
- **Choice #2:** Operator inserts any tuple passed in from child operators.

Halloween Problem - also known as the Update Query Problem

The Halloween Problem is an anomaly in which an update operation changes the physical location of a tuple, causing a scan operator to visit the tuple multiple times. This can occur on clustered tables or index scans.

This phenomenon was originally discovered by IBM researchers while building **System R** on Halloween day in 1976. The solution to this problem is to keep track of the modified record IDs for each query.

5 Expression Evaluation

The DBMS represents a WHERE clause as an *expression tree* (see Figure 7 for an example). The nodes in the tree represent different expression types.

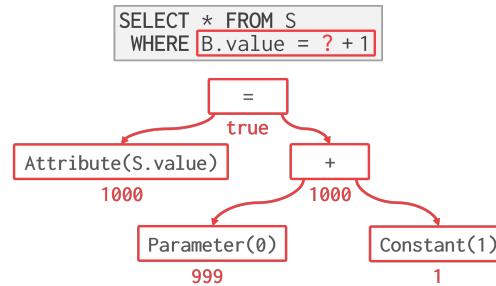


Figure 7: Expression Evaluation Example – A WHERE clause and a diagram of its corresponding expression.

Some examples of expression types that can be stored in tree nodes:

- Comparisons (=, <, >, !=);
- Conjunction (AND), Disjunction (OR);
- Arithmetic Operators (+, -, *, /, %);
- Constant and Parameter Values;
- Tuple Attribute References.

To evaluate an expression tree at runtime, the DBMS maintains a context handle that contains metadata for the execution, such as the current tuple, the parameters, and the table schema. The DBMS then walks the tree to evaluate its operators and produce a result.

Evaluating predicates in this manner is slow for the CPU because the DBMS must traverse the entire tree and determine the correct action to take for each operator. A better approach is to just evaluate the expression directly (think JIT compilation). Based on an internal cost model, the DBMS would determine whether code generation will be adopted to accelerate a query.

A even better approach is to **vectorize** it to evaluate batch of tuples at the same time.

There are ways to optimize expression evaluations:

- **Constant folding:** Reducing the overhead by identifying operations that can be performed just once (for example: UPPER on a constant value) and using it's results rather than performing it every single time.
- **Sub Expression limitation:** Identifying repeated subexpressions in an expression tree and computing its result once to reuse it for all occurrences in the plan.

Lecture #14: Query Execution II

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Background

Previous discussions of query executions assumed that the queries executed with a single worker (i.e. thread). However, in practice, queries are often executed in parallel with multiple workers.

Parallel execution spreads the database over multiple resources. These resources may be computational (e.g., CPU cores, CPU sockets, GPUs, additional machines) or storage (e.g., disks, memory). Spreading the database over many resources allows it to:

- Deal with large data sets that don't fit on a single machine/node
- Have higher performance due to parallel processing
- Have a higher level of redundancy/fault-tolerance

There are two types of parallelism that DBMSs support: inter-query parallelism and intra-query parallelism.

2 Parallel vs Distributed Databases

It is important to distinguish between parallel and distributed systems:

- **Parallel DBMS** In a *parallel DBMS*, resources, or nodes, are physically close to each other. These nodes communicate over a high-speed interconnect. It is assumed that communication between resources is not only fast, but also cheap and reliable.
- **Distributed DBMS** In a *distributed DBMS*, resources may be far away from each other; this might mean the database spans racks or data centers in different parts of the world. As a result, resources communicate using a slower interconnect (often over a public network). Communication costs between nodes are higher and failures cannot be ignored.

Even though a database may be physically divided over multiple resources, it still appears as a single logical database instance to the application. Thus, a SQL query executed against a single-node DBMS should generate the same result on a parallel or distributed DBMS.

3 Process Models

A DBMS *process model* defines how the system supports concurrent requests from a multi-user application/environment. The DBMS is comprised of one or more *workers* that are responsible for executing tasks on behalf of the client and returning the results. An application may send a large request or multiple requests at the same time that must be divided across different workers.

There are two major process models that a DBMS may adopt: process per worker and thread per worker. A third common database usage pattern takes an embedded approach.



Figure 1: Process per Worker Model

Process per Worker

The most basic approach is *process per worker*. Here, each worker is a separate OS process, and thus relies on the OS scheduler. An application sends a request and opens a connection to the database system. When the dispatcher receives this request, it selects one of its worker processes to manage the connection. The application then communicates directly with the worker who is responsible for executing the request that the query wants. This sequence of events is shown in Figure 1.

Relying on the operating system for scheduling effectively reduces the DBMS's control over execution. Further, this model depends on shared memory to maintain global data structures or relies on message passing, which has a higher overhead.

An advantage of the process per worker approach is that a process crash doesn't disrupt the whole system because each worker runs in the context of its own OS process.

This process model raises the issue of multiple workers on separate processes making numerous copies of the same page. A solution to maximize memory usage is to use shared-memory for global data structures so that they can be shared by workers running in different processes.

Examples of systems that utilize the process-per-worker process model include IBM DB2, Postgres, and Oracle. When these DBMSs were developed, pthreads had not yet become the standard threading model. The semantics of threading varied from OS to OS while `fork()` was better defined.

Thread per Worker

The most common model nowadays is *thread per worker*. Instead of having different processes doing different tasks, each database system has only one process with multiple worker threads. In this environment, the DBMS has full control over the tasks and threads, it can manage its own scheduling. The multi-threaded model may or may not use a dispatcher thread. A diagram of the thread per worker model is shown in Figure 2.

Using multi-threaded architecture provides certain advantages. For one, there is less overhead per context switch. Additionally, a shared model does not have to be maintained. However, it is possible that a thread crash can kill the entire database process. Also, the thread per worker model does not necessarily imply that the DBMS supports intra-query parallelism.

Almost every DBMS created in the last 20 years uses this approach, including Microsoft SQL Server and MySQL. IBM DB2 and Oracle have updated their models to provide support for this approach, as well. Postgres and Postgres-derived databases largely still use the process-based approach.

Embedded DBMS

A very different usage pattern for databases involves running the system in the same address space of the application, as opposed to a client-server model where the database stands independent of the application. In this scenario, the application will set up the threads and tasks to run on the database system. The

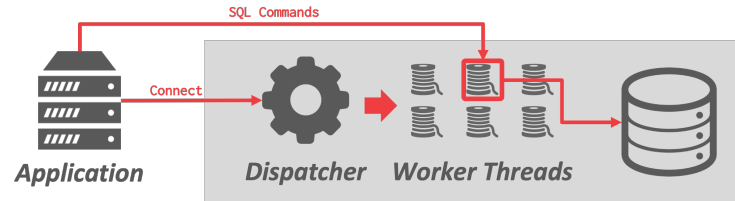


Figure 2: Thread per Worker Model

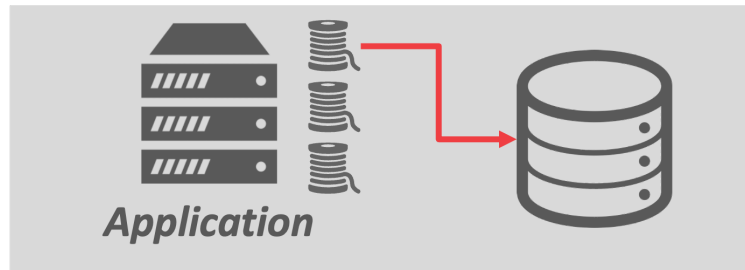


Figure 3: Embedded DBMS Scheduling

application itself will largely be responsible for scheduling. A diagram of an embedded DBMS's scheduling behaviors is shown in Figure 3.

DuckDB, SQLite, and RocksDB are the most famous embedded DBMSs.

Scheduling

For each query plan, the DBMS has to decide where, when, and how to execute. Relevant questions include:

- How many tasks should it use?
- How many CPU cores should it use?
- What CPU cores should the tasks execute on?
- Where should a task store its output?

When making decisions regarding query plans, the DBMS **always** knows more than the OS and should be prioritized as such.

4 Inter-Query Parallelism

In *inter-query parallelism*, the DBMS executes different queries concurrently. Since multiple workers are running requests simultaneously, overall performance is improved. This increases throughput and reduces latency.

If the queries are read-only, then little coordination is required between queries. However, if multiple queries are updating the database concurrently, more complicated conflicts arise. These issues are discussed further in lecture 17.

5 Intra-Query parallelism

In *intra-query parallelism*, the DBMS executes the operations of a single query in parallel. This decreases latency for long-running queries.

The organization of intra-query parallelism can be thought of in terms of a *producer/consumer* paradigm. Each operator is a producer of data as well as a consumer of data from some operator running below it.

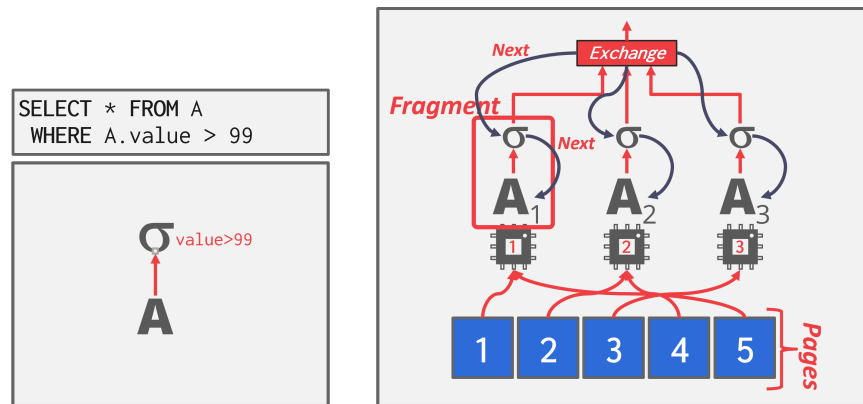


Figure 4: Intra-Operator Parallelism – The query plan for this SELECT is a sequential scan on A that is fed into a filter operator. To run this in parallel, the query plan is partitioned into disjoint fragments. A given plan fragment is operated on by a distinct worker. The exchange operator calls Next concurrently on all fragments which then retrieve data from their respective pages.

Parallel algorithms exist for every relational operator. The DBMS can either have multiple threads access centralized data structures or use partitioning to divide work up.

Within intra-query parallelism, there are three types of parallelism: intra-operator, inter-operator, and bushy. These approaches are not mutually exclusive. It is the DBMS' responsibility to combine these techniques in a way that optimizes performance on a given workload.

Intra-Operator Parallelism (Horizontal)

In *intra-operator parallelism*, the query plan's operators are decomposed into independent *fragments* that perform the same function on different (disjoint) subsets of data.

The DBMS inserts an *exchange* operator into the query plan to coalesce results from child operators. The exchange operator prevents the DBMS from executing operators above it in the plan until it receives all of the data from the children. An example of this is shown in Figure 4.

In general, there are three types of exchange operators:

- **Gather:** Combine the results from multiple workers into a single output stream. This is the most common type used in parallel DBMSs.
- **Distribute:** Split a single input stream into multiple output streams.
- **Repartition:** Reorganize multiple input streams across multiple output streams. This allows the DBMS take inputs that are partitioned one way and then redistribute them in another way.

Inter-Operator Parallelism (Vertical)

In *inter-operator parallelism*, the DBMS overlaps operators in order to pipeline data from one stage to the next without materialization. This is sometimes called *pipelined parallelism*. See example in Figure 5.

This approach is widely used in *stream processing systems*, which are systems that continually execute a query over a stream of input tuples.

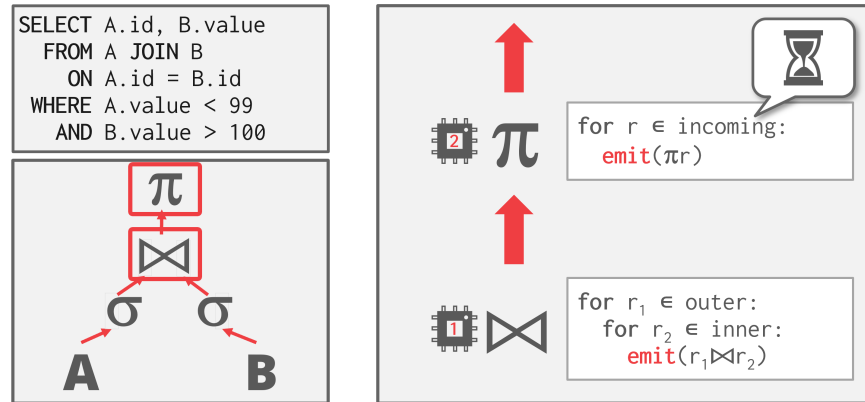


Figure 5: Inter-operator Parallelism – In the JOIN statement to the left, a single worker performs the join and then emits the result to another worker that performs the projection and then emits the result again.

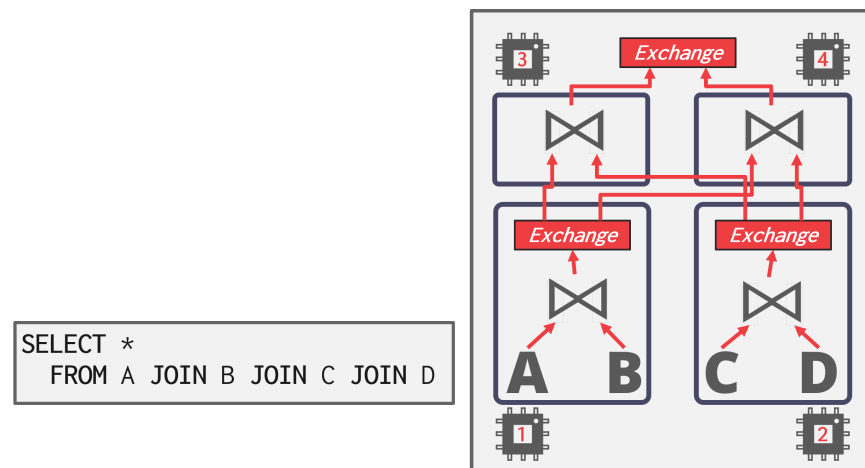


Figure 6: Bushy Parallelism – To perform a 4-way JOIN on three tables, the query plan is divided into four fragments as shown. Different portions of the query plan run at the same time, in a manner similar to inter-operator parallelism.

Bushy Parallelism

Bushy parallelism is a hybrid of intra-operator and inter-operator parallelism where workers execute multiple operators from different segments of the query plan at the same time.

The DBMS still uses exchange operators to combine intermediate results from these segments. An example is shown in Figure 6.

Data Parallelism

Modern CPUs provide vector registers that allow a single instruction to operate on multiple values (SIMD) packed into a register. For example, on x86 we have AVX-512 instructions which operates on 512 bits at a time. This can provide large speedups over un-vectorized versions (however, note that certain CPUs actually reduce clock frequency when using SIMD instructions due to thermal constraints).

To exploit modern, speculative CPU architectures further, it is usually a good idea to avoid branching logic or unpredictable control flow, as branch mispredictions can lead to high penalty in CPU cycles and pipeline

flushing. Refer to the lecture slides for an example of using SIMD instructions instead of if-statements.

6 I/O Parallelism

Using additional processes/threads to execute queries in parallel will not improve performance if the disk is always the main bottleneck. Therefore, it is important to be able to split a database across multiple storage devices.

To get around this, DBMSs use I/O parallelism to *split installation across multiple devices*. Two approaches to I/O parallelism are multi-disk parallelism and database partitioning.

Multi-Disk Parallelism

In *multi-disk parallelism*, the OS/hardware is configured to store the DBMS's files across multiple storage devices. This can be done through storage appliances or RAID configuration based on performance, durability, and capacity constraints. All of the storage setup is transparent to the DBMS so workers cannot operate on different devices because the DBMS is unaware of the underlying parallelism.

Database Partitioning

In *database partitioning*, the database is split up into disjoint subsets that can be assigned to discrete disks. Some DBMSs allow for specification of the disk location of each individual database. This is easy to do at the file-system level if the DBMS stores each database in a separate directory. The log file of changes made is usually shared.

The idea of *logical partitioning* is to split single logical table into disjoint physical segments that are stored/-managed separately. Such partitioning is ideally transparent to the application. That is, the application should be able to access logical tables without caring how things are stored.

We will cover these approaches later in the semester when discussing distributed databases.

Lecture #15: Query Planning & Optimization I

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Overview

Because SQL is declarative, the query only tells the DBMS what to compute, but not how to compute it. Thus, the DBMS needs to translate a SQL statement into an executable query plan. But there are different ways to execute each operator in a query plan (e.g., join algorithms) and there will be differences in performance among these plans. The job of the DBMS's optimizer is to pick an optimal plan for any given query.

The first implementation of a query optimizer was IBM System R and was designed in the 1970s. Prior to this, people did not believe that a DBMS could ever construct a query plan better than a human. Many concepts and design decisions from the System R optimizer are still in use today.

Query optimization is the most difficult part of building a DBMS. Some systems have attempted to apply machine learning to improve the accuracy and efficiency of optimizers, and modern DBMSs have begun incorporating machine learning techniques into certain components, but no major DBMS currently deploys a fully machine learning-based optimizer.

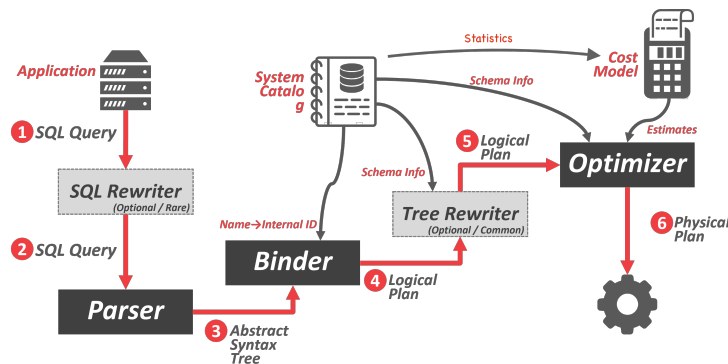


Figure 1: Architecture Overview – The application connected to the database system and sends a SQL query, which may be rewritten to a different format. The SQL string is parsed into tokens that make up the syntax tree. The binder converts named objects in the syntax tree to internal identifiers by consulting the system catalog. The binder emits a logical plan which may be fed to a tree rewriter for additional schema info. The logical plan is given to the optimizer which selects the most efficient procedure to execute the plan.

Logical vs. Physical Plans

The optimizer generates a mapping of a *logical algebra expression* to the optimal equivalent physical algebra expression. The logical plan is roughly equivalent to the relational algebra expressions in the query.

Physical operators define a specific execution strategy using an access path for the different operators in the query plan. Physical plans may depend on the physical format of the data that is processed (i.e. sorting, compression).

There does not always exist a one-to-one mapping from logical to physical plans.

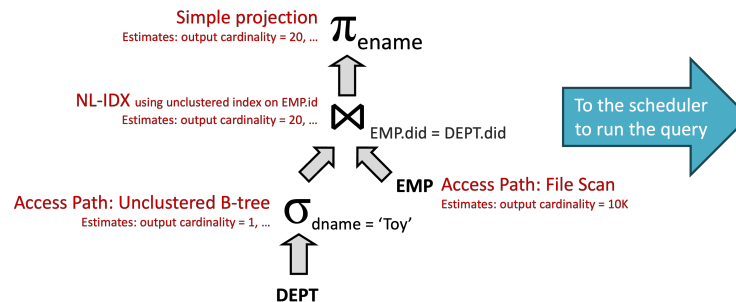


Figure 2: Physical plan as an annotated relational algebra tree: – You could view them as a relational algebra tree that is annotated with the information needed for the physical implementation.

Optimization Granularity

- **Single Query Optimization:** Smaller search space and simpler optimization. However, the DBMS usually does not reuse results across queries, and the cost model may need to consider resource contention from other running queries.
- **Multiple Query Optimization:** More efficient when many similar queries exist, as it enables sharing of data access and intermediate results. However, the search space becomes much larger, making optimization more complex.

2 Relational Algebra Equivalence

Transformation: The optimizer generates different query plans that are semantically equivalent and logically correct, ensuring they always produce the same result as the original query. The goal of these transformations is to reduce execution cost and enable further optimizations. This process relies on **relational algebra equivalences**, together with information from both the query and the underlying data (logical and physical).

Much of query optimization relies on the underlying concept that the high level properties of relational algebra are preserved across equivalent expressions. Two relational algebra expressions are *equivalent* if they generate the same set of tuples.

This technique of transforming the underlying relational algebra representation of a logical plan is known as *query rewriting*.

One example of relational algebra equivalence is *predicate pushdown*, in which a predicate is applied in a different position of the sequence to avoid unnecessary work. Figure 3 shows an example of predicate pushdown.

3 Logical Query Optimization

Transform a logical plan into an equivalent logical plan using pattern matching rules.

Some selection optimizations include:

- Predicate Pushdown - Perform filters as early as possible.
- Reorder predicates so that the DBMS applies the most selective one first.
- Split Conjunctive Predicates - Breakup a complex predicate and pushing it down.
- Replace Cartesian Products with Joins - Transform into inner joins using the join predicates.

An example of predicate pushdown is shown in Figure 3.

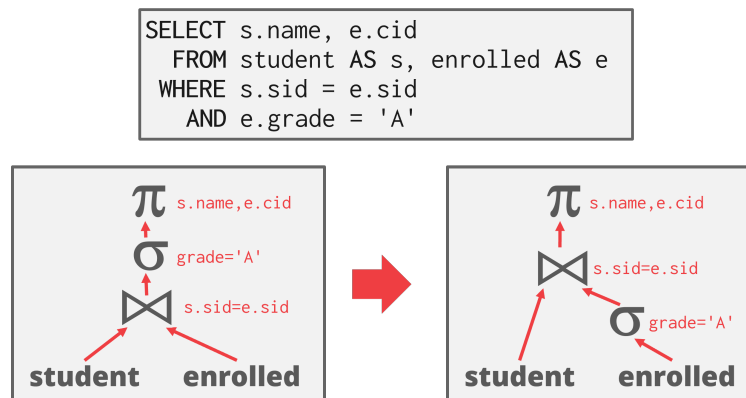


Figure 3: Predicate Pushdown: – Instead of performing the filter after the join, the filter can be applied earlier in order to pass fewer elements into the filter.

Some projection optimizations include:

- Perform projections as early as possible to create smaller tuples and reduce intermediate results (*projection pushdown*).
- Project out all attributes except the ones requested or requires.
- Optimization from projection pushdown depends on number of columns.

An example of projection pushdown is shown in Figure 4.

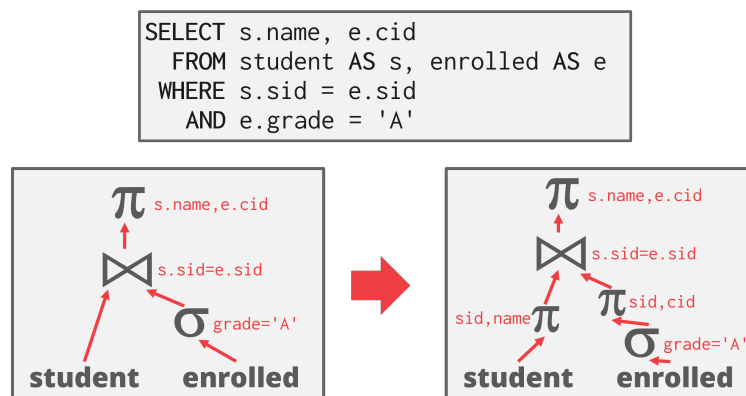


Figure 4: Projection Pushdown – Since the query only asks for the student name and ID, the DBMS can remove all columns except for those two before applying the join.

4 Types of Query Optimization

Search Algorithms: *Given a set of transformation rules, the optimizer searches for a good physical plan for a query. This search process may not always be fully guided by a cost model. At search time, the optimizer starts from a query's logical plan but may not have complete information, such as unknown input values in prepared statements or missing statistical information.*

There are two high-level strategies (Search Algorithms) for query optimization.

1. The first approach is to use static rules, or *heuristics*. Heuristics match portions of the query with known patterns to assemble a plan. These rules transform the query to remove inefficiencies. Although these rules may require consultation of the catalog to understand the structure of the data, they never need to examine the data itself.

Heuristic-based optimization is easy to implement and works well for simple queries, but it relies on fixed assumptions and is nearly impossible to produce good plans for complex queries with inter-dependent operators.

2. An alternative approach is to use *cost-based search* to read the data and estimate the cost of executing equivalent plans. The cost model chooses the plan with the lowest cost.

5 Cost-Based Query Optimization

After performing rule-based rewriting, the DBMS enumerates alternative query plans and estimates their costs. It then selects the best plan found, either after considering all plans or when a search termination condition is reached.

6 Search Termination

Optimizers must decide when to stop exploring new plans. Common termination criteria include:

- Wall-clock Time: Stop after running for a fixed duration.
- Cost Threshold: Stop when a plan below a cost threshold is found.
- Exhaustion: Stop when no more plan enumerations remain.
- Transformation Count: Stop after a fixed number of rule applications.

7 Access Path Transformation

The optimizer selects access methods that minimize data retrieval cost from base relations and may reorder predicate evaluations.

Key cost factors include:

- Predicate selectivity
- Data structures (e.g., B+Tree vs. hash index)
- Sort order of table or index
- Data features (e.g., INCLUDE, zone maps)
- Compression and encoding

8 Single-Relation Query Plans

For single-relation query plans, the biggest obstacle is choosing the best access method (i.e., sequential scan, binary search, index scan, etc.) Most new database systems just use heuristics, instead of a sophisti-

cated cost model, to pick an access method.

For OLTP queries, this is especially easy because they are *sargable* (Search Argument Able), which means that there exists a best index that can be selected for the query. This can also be implemented with simple heuristics.

9 Multi-Relation Query Plans

For Multi-Relation query plans, as number of joins increases, the number of alternative plans grow rapidly. Consequently, it is important to restrict the search space so as to be able to find the optimal plan in a reasonable amount of time. There are two ways to approach this search problem:

- **Generative / Bottom-up:** Start with nothing and then build up the plan to get to the outcome that you want. Examples: IBM System R, DB2, MySQL, Postgres, most open-source DBMSs.
- **Transformation / Top-down:** Start with the outcome that you want, and then work down the tree to find the optimal plan that gets you to that goal. Examples: MSSQL, Greenplum, CockroachDB, Volcano

10 Bottom-up optimization example - System R

Use static rules to perform initial optimization. Then use dynamic programming to determine the best join order for tables using a divide-and conquer search method. Most open source database systems do this.

- Break query up into blocks and generate the logical operators for each block
- For each logical operator, generate a set of physical operators that implement it
- Then, iteratively construct a "left-deep" tree that minimizes the estimated amount of work to execute the plan

11 Top-down optimization example - Volcano

The Volcano optimizer begins with an initial logical query plan. It then explores the plan space by applying transformation rules (logical-to-logical) and implementation rules (logical-to-physical), recursively finding equivalent expressions and optimal implementations for sub-plans. Memoization prevents redundant exploration of identical plans.

- Keep track of global best plan during search.
- Treat physical properties of data as first-class entities during planning.

$$\begin{aligned}
\sigma_{P_1}(\sigma_{P_2}(R)) &\equiv \sigma_{P_2}(\sigma_{P_1}(R)) \quad (\sigma \text{ commutativity}) \\
\sigma_{P_1 \wedge P_2 \dots \wedge P_n}(R) &\equiv \sigma_{P_1}(\sigma_{P_2}(\dots \sigma_{P_n}(R))) \quad (\text{cascading } \sigma) \\
\Pi_{a_1}(R) &\equiv \Pi_{a_1}(\Pi_{a_2}(\dots \Pi_{a_k}(R)\dots)), a_i \subseteq a_{i+1} \quad (\text{cascading } \Pi) \\
R \bowtie S &\equiv S \bowtie R \quad (\text{join commutativity}) \\
R \bowtie (S \bowtie T) &\equiv (R \bowtie S) \bowtie T \quad (\text{join associativity}) \\
\sigma_P(R \times S) &\equiv (R \bowtie_P S), \text{ if } P \text{ is a join predicate} \\
\sigma_P(R \times S) &\equiv \sigma_{P_1}(\sigma_{P_2}(R) \bowtie_{P_4} \sigma_{P_3}(S)), \text{ where } P = p_1 \wedge p_2 \wedge p_3 \wedge p_4 \\
\Pi_{A_1, A_2, \dots, A_n}(\sigma_P(R)) &\equiv \Pi_{A_1, A_2, \dots, A_n}(\sigma_P(\Pi_{A_1, \dots, A_n, B_1, \dots, B_M}(R))), \text{ where } B_1 \dots B_M \text{ are columns in } P \\
&\dots
\end{aligned}$$

Figure 5: Example Logical Rules – The set of rules in the system defines the search space the optimizer operates on.

12 Nested Sub-Queries

The DBMS treats nested sub-queries in the where clause as functions that take parameters and return a single value or set of values.

- Re-write the query by de-correlating and / or flattening nested subqueries. An example of this is shown in Figure 6.
- Decompose the nested query and store the result to a temporary table. An example of this is shown in Figure 7.

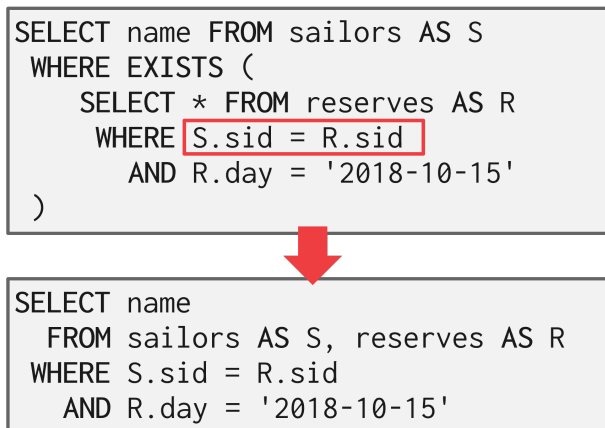


Figure 6: Subquery Optimization - Rewriting The former query can be rewritten as the latter query by rewriting the subquery as a JOIN. Removing a level of nesting in this way effectively *flattens* the query.

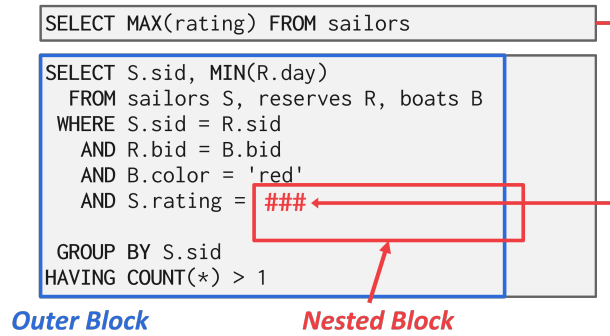


Figure 7: Subquery Optimization - Decomposition – For complex queries with subqueries, the DBMS optimizer may break up the original query into blocks and focus on optimizing each individual block at a time. In this example, the optimizer decomposes a query with a nested aggregation by pulling the nested query out into its own query, and subsequently using this result to realize the logic of the original query.

13 Expression Rewriting

An optimizer transforms a query's expression (e.g. WHERE/ON clause predicates) into a minimal set of expressions.

- Search for expressions that match a pattern.
- When a match is found, rewrite the expression.
- Halt if there are no more rules that match.

Some examples of expression rewriting

- Impossible predicates as shown in Figure 8.
- Merging predicate as shown in Figure 9.

Impossible / Unnecessary Predicates

```
SELECT * FROM A WHERE 1 = 0;
```

Figure 8: Impossible Predicates – Evaluate the expression if possible at optimization time.

```
SELECT * FROM A
WHERE val BETWEEN 1 AND 100
OR val BETWEEN 50 AND 150;
```

```
SELECT * FROM A
WHERE val BETWEEN 1 AND 150;
```

Figure 9: Merging Predicates – The WHERE predicate in query 1 has redundancy as what it is searching for is any value between 1 and 150. Query 2 shows the more succinct way to express request in query 1.

Lecture #16: Query Planning & Optimization II

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Multi-Relation Query Plans

For Multi-Relation query plans, as number of joins increases, the number of alternative plans grows rapidly. Consequently, it is important to restrict the search space so as to be able to find the optimal plan in a reasonable amount of time. There are two ways to approach this search problem:

- **Generative / Bottom-up:** Start with nothing and then build up the plan to get to the outcome that you want. Examples: IBM System R, DB2, MySQL, Postgres, most open-source DBMSs.
- **Transformation / Top-down:** Start with the outcome that you want, and then work down the tree to find the optimal plan that gets you to that goal. Examples: MSSQL, Greenplum, CockroachDB, Volcano

Bottom-up optimization example - System R

Use static rules to perform initial optimization. Then use dynamic programming to determine the best join order for tables using a divide-and conquer search method. Most open source database systems do this.

- Break query up into blocks and generate the logical operators for each block
- For each logical operator, generate a set of physical operators that implement it
- Then, iteratively construct a "left-deep" tree that minimizes the estimated amount of work to execute the plan

Top-down optimization example - Volcano

The Volcano optimizer begins with an initial logical query plan. It then explores the plan space by applying transformation rules (logical-to-logical) and implementation rules (logical-to-physical), recursively finding equivalent expressions and optimal implementations for sub-plans. During this search, a branch can be pruned early if its estimated cost already exceeds the lowest-cost plan found so far. Memoization prevents redundant exploration of identical plans. Enforcers are physical operators that ensure the properties of the output of a sub-plan or expression.

2 Data Statistics

Histograms

Real data is often skewed and is tricky to make assumptions about. However, storing every single value of a data set is expensive. One way to reduce the amount of memory used is by storing data in a *histogram* to group together values. An example of a graph with buckets is shown in Figure 1.

Another approach is to use a *equi-depth* histogram that varies the width of buckets so that the total number of occurrences for each bucket is roughly the same. An example is shown in Figure 2.

If certain keys occur frequently, one approach is to use an *end-biased* histogram that uses N-1 buckets to store the exact count for the most frequent keys and uses the last bucket to store the average frequency of all remaining values. An example is shown in Figure 3.

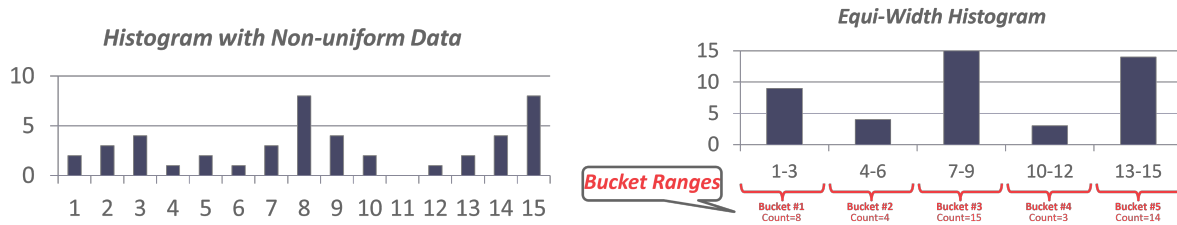


Figure 1: Equi-Width Histogram: The first figure shows the original frequency count of the entire data set. The second figure is an equi-width histogram that combines together the counts for adjacent keys to reduce the storage overhead.

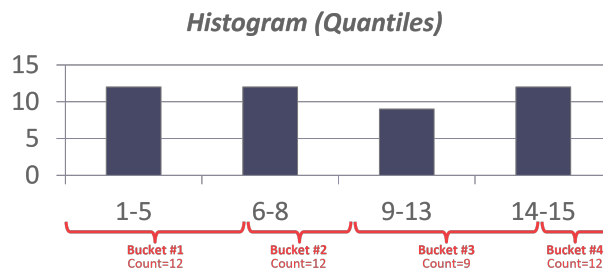


Figure 2: Equi-Depth Histogram – To ensure that each bucket has roughly the same number of counts, the histogram varies the range of each bucket.

In place of histograms, some systems may use *sketches* to generate approximate statistics about a data set, including Count-Min Sketch and HyperLogLog.

Sampling

DBMSs can use *sampling* to apply predicates to a subset of the table with a similar distribution (see Figure 4). There are two approaches to do this: maintaining read-only copy of the sample, or sampling real tables. The DBMS updates the sample whenever the amount of changes to the underlying table exceeds some threshold (e.g., 10% of the tuples).

3 Cost Estimations

DBMSs use cost models to estimate the cost of executing a plan. These models evaluate equivalent plans for a query to help the DBMS select the most optimal one.

The cost of a query depends on both physical and logical cost estimates:

- **Physical costs:** CPU work, disk I/O, memory usage, and network communication.
- **Logical costs:** estimated output sizes (cardinalities) of operators, which are used to predict the cost of later operators in the plan.

Exhaustive enumeration of all valid plans for a query is much too slow for an optimizer to perform. For joins alone, which are commutative and associative, there are 4^n different orderings of every n -way join. Optimizers must limit their search space in order to work efficiently.

To approximate costs of queries, DBMSs maintain internal *statistics* about tables, attributes, and indexes in their internal catalogs. Different systems maintain these statistics in different ways. Most systems attempt to avoid on-the-fly computation by maintaining an internal table of statistics. These internal tables may then be updated in the background.

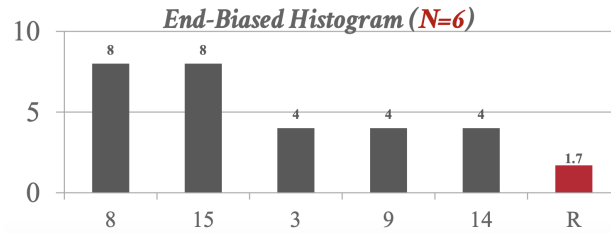


Figure 3: End-Biased Histogram – The exact counts for the most frequent keys are stored in the first $N-1$ buckets. The last bucket (R) stores the average frequency of all remaining values.

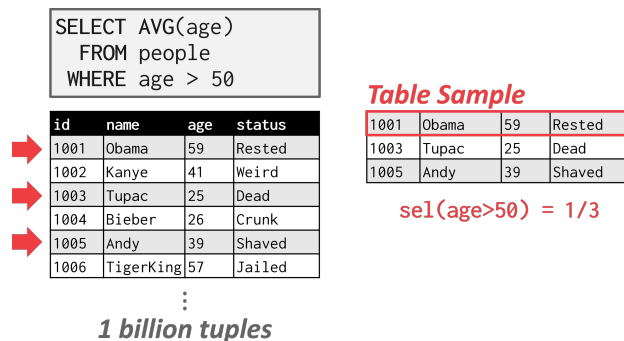


Figure 4: Sampling – Instead of using one billion values in the table to estimate selectivity, the DBMS can derive the selectivities for predicates from a subset of the original table.

For each relation R , the DBMS maintains the following information:

- N_R : Number of tuples in R
- $V(A, R)$: Number of distinct values of attribute A

With the information listed above, the optimizer can derive the *selection cardinality* $SC(A, R)$ statistic. The selection cardinality is the average number of records with a value for an attribute A given $\frac{N_R}{V(A, R)}$. Note that this assumes data uniformity. This assumption is often incorrect, but it simplifies the optimization process.

Selection Statistics

The selection cardinality can be used to determine the number of tuples that will be selected for a given input.

Equality predicates on unique keys are simple to estimate (see Figure 5). A more complex predicate is shown in Figure 6.

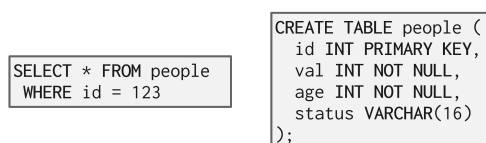


Figure 5: Simple Predicate Example – In this example, determining what index to use is easy because the query contains an equality predicate on a unique key.

```
SELECT * FROM people
WHERE val > 1000
```

```
SELECT * FROM people
WHERE age = 30
AND status = 'Lit'
AND age+id IN (1,2,3)
```

Figure 6: Complex Predicate Example – More complex predicates, such as range or conjunctions, are harder to estimate because the selection cardinalities of the predicates must be combined in non-trivial ways.

The *selectivity* (sel) of a predicate P is the fraction of tuples that qualify. The formula used to compute selectivity depends on the type of predicate. Selectivity for complex predicates is hard to estimate accurately which can pose a problem for certain systems. An example of a selectivity computation is shown in Figure 7.

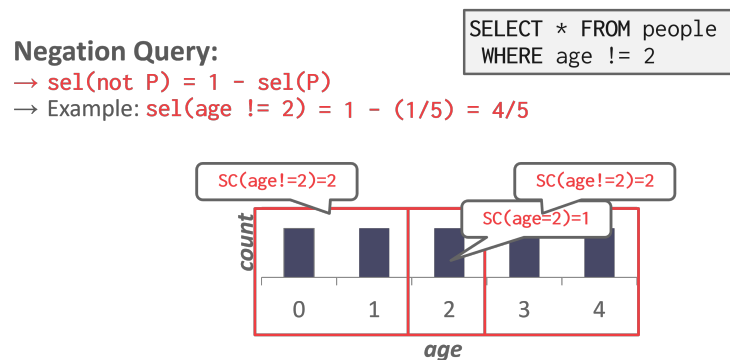


Figure 7: Selectivity of Negation Query Example – The selectivity of the negation query is computed by subtracting the selectivity of the positive query from 1. In the example, the answer comes out to be $\frac{4}{5}$ which is accurate.

Observe that the selectivity of a predicate is equivalent to the probability of that predicate. This allows probability rules to be applied in many selectivity computations. This is particularly useful when dealing with complex predicates. For example, if we assume that multiple predicates involved in a conjunction are *independent*, we can compute the total selectivity of the conjunction as the product of the selectivities of the individual predicates.

Selectivity Computation Assumptions

In computing the selection cardinality of predicates, the following three assumptions are used.

- **Uniform Data:** The distribution of values (except for the heavy hitters) is the same.
- **Independent Predicates:** The predicates on attributes are independent.
- **Containment Principle:** The domain of join keys overlap such that each key in the inner relation will also exist in the outer table.

These assumptions are often not satisfied by real data. For example, *correlated attributes* break the assumption of independence of predicates.

Join Size Estimation

For a join between relations R and S , the goal is to determine how many tuples in S each tuple in R could match, assuming that every key in the inner relation exists in the outer table. We make the same three assumptions as above when estimating the join size.

In the general case where R and S share a non-primary-key attribute A , the estimated join cardinality is approximately $\frac{N_R \times N_S}{\max(V(A,S), V(A,R))}$.

However, estimation errors can easily propagate through query plans, as inaccuracies in one operator's estimate may cascade and amplify in subsequent operations, leading to poor overall cost estimates and suboptimal query plans.

Lecture #17: Concurrency Control Theory

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Motivation

- **Lost Update Problem (Concurrency Control):** How do we handle two or more transactions trying to update the same data at the same time?
- **Durability Problem (Recovery):** How can we ensure the correct state in case of a power failure?

2 Transactions

A *transaction* is the execution of a sequence of one or more operations (e.g., SQL queries) on a shared database to perform some higher level function. They are the basic unit of change in a DBMS. Partial transactions are not allowed (i.e. transactions must be atomic).

Example: Pay \$25 from Andy's bank account with balance \$100 to a concert promoter.

1. **Read** Andy's account balance.
2. **Check** if balance > \$25.
3. **Deduct** \$25 from his account.
4. **Write** Andy's new balance \$75 back to the database.

Either all of the steps need to be completed or none of them should be completed.

The Strawman System

A simple system for handling transactions is to execute one transaction at a time using a single worker (i.e. serial order). Thus, only one transaction can be running at a time in the DBMS. before a transaction starts, the DBMS copies the entire database to a new file and makes all changes to that file.

- If the transaction succeeds, the new file becomes the current database file.
- If the transaction fails, the DBMS discards the dirty copy and none of the transaction's changes would have been saved.

This method (also known as shadow paging) is slow as it does not allow for concurrent transactions and throws away any parallelism, while also requires copying the whole database file for every transaction.

A (potentially) better approach is to allow concurrent execution of independent transactions while also maintaining correctness and fairness (as in all transactions are treated with equal priority and don't get "starved" by never being executed). Taking advantage of all the additional parallelism available in modern hardware (multiple cores, multiple disks, etc.) can significantly improve throughput and latency of transactions. But executing concurrent transactions in a DBMS is challenging. It is difficult to ensure correctness (for example, if Andy only has \$100 and tries to pay off two promoters at once, who should get paid?) while also executing transactions quickly (our strawman example guarantees sequential correctness, but at the cost of parallelism).

Arbitrary interleaving of operations can lead to:

- **Temporary Inconsistency:** Unavoidable, but not an issue.
- **Permanent Inconsistency:** Unacceptable, cause problems with correctness and integrity of data.

The DBMS is only concerned about what data is read/written from/to the database. Changes to the outside world are beyond the scope of the DBMS. For example, if a transaction causes an email to be sent, this cannot be rolled back by the DBMS if the transaction is aborted. We want formal correctness criteria to determine whether an interleaving is valid.

3 Definitions

Formally, a *database* can be represented as a fixed set of named data objects ($A, B, C, \dots$). These objects can be attributes, tuples, pages, tables, or even databases. The algorithms that we will discuss work on any type of object but all objects must be of the same type.

A *transaction* is a sequence of read and write operations (e.g., $R(A), W(B)$) on those objects, which is DBMS's abstract view of a user program.

The boundaries of transactions are defined by the client. In SQL, a new transaction starts with the BEGIN command. Then the transaction stops with either COMMIT or ABORT. For COMMIT, either all of the transaction's modifications are saved to the database, or the DBMS overrides this and aborts instead. For ABORT, all of the transaction's changes are undone so that it is like the transaction never happened. Aborts can be either self-inflicted or caused by the DBMS.

The criteria used to ensure the correctness of a database is given by the acronym **ACID**.

- **A**tomicity: Atomicity ensures that either all actions in the transaction happen, or none happen.
- **C**onsistency: If each transaction is consistent and the database is consistent at the beginning of the transaction, then the database is guaranteed to be consistent when the transaction completes. Data is consistent if it satisfies all validation rules such as constraints, cascades and triggers.
- **I**solation: Isolation means that when a transaction executes, it should have the illusion that it is isolated from other transactions. Isolation ensures that concurrent execution of transactions should have the same resulting database state as a sequential execution of the transactions.
- **D**urability: If a transaction commits, then its effects on the database should persist no matter what happens (e.g. power failure, OS crash).

4 ACID: Atomicity

The DBMS guarantees that transactions are **atomic**. From application's point of view: the transaction either executes all its actions or none of them. Two possible outcomes of executing a transaction:

- **Commit:** Commit after completing all its actions
- **Abort:** Abort (or be aborted by the DBMS) after executing some actions

There are two approaches to ensure atomicity:

Approach #1: Logging

DBMS logs all actions in an ordered ledger so that it can undo the actions in case of an aborted transaction. It maintains undo records both in memory and on disk. The logs will be replayed after crash to put database back in correct state. Logging is used by almost all modern systems for audit and efficiency reasons.

Approach #2: Shadow Paging

The DBMS makes copies of pages modified by the transactions and transactions make changes to those copies. Only when the transaction successfully commits is the page made visible to other transactions.

This approach is typically slower at runtime than a logging-based DBMS. However, one benefit is, if you are only single threaded, there is no need for logging, so there are less writes to disk when transactions modify the database. This also makes recovery instant and simple, as all you need to do is delete all pages from uncommitted transactions. In general, though, better runtime performance is preferred over better recovery performance, so this is rarely used in practice.

5 ACID: Consistency

At a high level, consistency means the “world” represented by the database is **logically** correct. SQL has methods to specify integrity constraints (e.g., key definitions, CHECK, ADD CONSTRAINT) and the DBMS will enforce them. Then it is up to the application to define these constraints. The DBMS is responsible for ensuring that all integrity constraints are true before and after the transaction ends.

Eventual Consistency: A committed transaction may see inconsistent results (e.g. may not see the updates of an older committed transaction immediately). Then it becomes difficult for developers to reason about such semantics, so the trend is to move away from eventual consistency and towards stronger consistency models.

6 ACID: Isolation

The DBMS provides transactions the illusion that they are running alone in the system. They do not see the effects of concurrent transactions. This is equivalent to a system where transactions are executed in serial order (i.e., one at a time). But to achieve better performance, the DBMS has to interleave the operations of concurrent transactions while maintaining the illusion of isolation.

Concurrency Control

A *concurrency control protocol* is how the DBMS decides the proper interleaving of operations from multiple transactions at runtime.

There are two categories of concurrency control protocols:

1. **Pessimistic:** The DBMS assumes that transactions will conflict, so it doesn't let problems arise in the first place.
2. **Optimistic:** The DBMS assumes that conflicts between transactions are rare, so it chooses to deal with conflicts after they happen.

Example: assuming A and B both start with \$ 1000



There are many possible outcomes of running T_1 and T_2 concurrently. But the ground rule is $A + B$ should be $2000 * 1.06 = 2120$. There is no guarantee on the order of transaction execution, but the outcome of the database must be equivalent to some serial execution of the transactions.

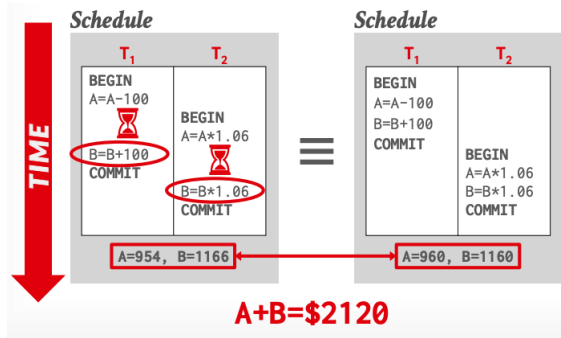


Figure 1: Good interleaving schedule

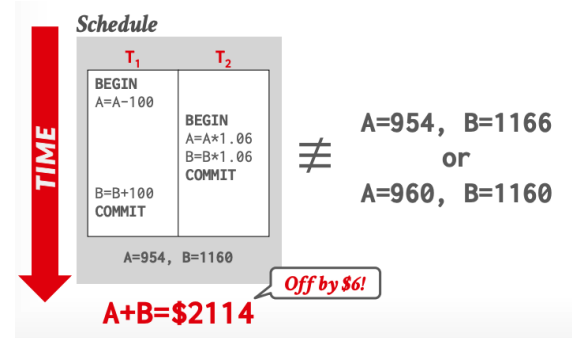


Figure 2: Bad interleaving schedule

The order in which the DBMS executes operations is called an *execution schedule*. We want to interleave transactions to maximize concurrency while ensuring that the output is “correct”. The goal of a concurrency control protocol is to generate an execution schedule that is equivalent to some serial execution:

- **Serial Schedule:** Schedule that does not interleave the actions of different transactions.
- **Equivalent Schedules:** For any database state, if the effect of executing the first schedule is identical to the effect of executing the second schedule, the two schedules are equivalent.
- **Serializable Schedule:** A serializable schedule is a schedule that is equivalent to some serial execution of the transactions. Different serial executions can produce different results, but all are considered “correct”. Note that if each transaction preserves the consistency, every serializable schedule also preserves the consistency.

A *conflict* between two operations occurs if the operations are for different transactions, they are performed on the same object, and at least one of the operations is a write. There are three variations of conflicts (note that there are also Phantom Reads and Write-Skew to be covered later):

- **Read-Write Conflicts (“Unrepeatable Reads”):** A transaction is not able to get the same value when reading the same object multiple times.
- **Write-Read Conflicts (“Dirty Reads”):** A transaction sees the write effects of a different transaction before that transaction commits its changes.
- **Write-Write Conflicts (“Lost Updates”):** One transaction overwrites the uncommitted data of another uncommitted transaction.

There are two types for serializability: (1) *conflict serializability* and (2) *view serializability*. Neither definition allows all schedules that one would consider serializable. In practice, DBMSs support conflict serializability because it can be enforced efficiently.

Conflict Serializability

Two schedules are *conflict equivalent* if and only if they involve the same operations of the same transactions and every pair of conflicting operations is ordered in the same way in both schedules. A schedule S is *conflict serializable* if it is conflict equivalent to some serial schedule.

One can verify that a schedule is conflict serializable by swapping consecutive non-conflicting operations of different transactions until a serial schedule is formed. For schedules with many transactions, this becomes too expensive. A better way to verify schedules is to use a *dependency graph* (precedence graph).

In a dependency graph, each transaction is a node in the graph. There exists a directed edge from node T_i to T_j iff an operation O_i from T_i conflicts with an operation O_j from T_j and O_i occurs before O_j in the schedule. Then, a schedule is conflict serializable iff the dependency graph is acyclic.

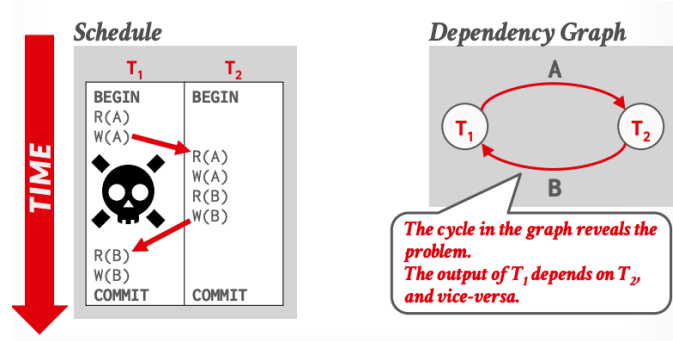


Figure 3: Dependency Graph Example

View Serializability

View serializability is a weaker notion of serializability that allows for all schedules that are conflict serializable and “blind writes” (i.e. performing writes regardless of its original value). Thus, it allows for more schedules than conflict serializability, but is difficult to enforce efficiently. This is because the DBMS does not know how the application will “interpret” values. As such, view serializability is not used in practice.

Formally, two schedules S_1 and S_2 are *view equivalent* if

- If T_1 reads initial value of A in S_1 , then T_1 also reads initial value of A in S_2 .
- If T_1 reads value of A written by T_2 in S_1 , then T_1 also reads value of A written by T_2 in S_2 .
- If T_1 writes final value of A in S_1 , then T_1 also writes final value of A in S_2 .

About “*blind writes*”: From a database perspective, all that matters is whether the database state is equivalent to some sort of serial execution, so blind writes can still make a set of transactions view serializable even if its dependency graph has cycles.

Universe of Schedules

$\text{SerialSchedules} \subset \text{ConflictSerializableSchedules} \subset \text{ViewSerializableSchedules} \subset \text{AllSchedules}$

7 ACID: Durability

All of the changes of committed transactions must be **durable** (i.e., persistent) after a crash or restart, so there will be no torn updates nor changes from failed transactions. The DBMS can either use logging or shadow paging to ensure that all changes are durable. This usually requires that committed transactions are stored in non-volatile memory.

Lecture #18: Two-Phase Locking

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Transaction Locks

Our notions of serializability thus far have assumed we know all reads and writes while constructing a schedule, but in practice, we need a way to guarantee correctness on the fly. A DBMS uses *locks* to dynamically generate an execution schedule for transactions that is serializable. These locks protect database objects during concurrent access when there are multiple transactions trying to read/write. The DBMS contains a centralized *lock manager* that decides whether a transaction can acquire a lock on each particular object. This is how we avoid deadlocks (rather than being disciplined about their usage like with latches as we don't have control over application logic).

Importantly, locks are different from *latches* such as those used in the B+ tree crabbing algorithm (slide 6). Latches protect the DBMS's internal data structures from concurrent threads whereas locks protect values in the database from concurrent transactions. For example, in a B+ tree, you only hold latches over individual leaf nodes in a scan because that's all you need to do to ensure correctness, but if one transaction attempts a leaf scan while another attempts to write to two arbitrary values, the leaf scan needs to lock the whole table, not just the current leaf, to avoid seeing only one of the two writes. For this reason, locks are usually held for entire transactions while latches are held for short periods to facilitate thread-safe operations on DBMS data structures.

There are two basic lock types (slide 9):

- **Shared Lock (S-LOCK):** A shared lock allows multiple transactions to read the same object at the same time. If one transaction holds a shared lock, then another transaction can also acquire that same shared lock.
- **Exclusive Lock (X-LOCK):** An exclusive lock allows a transaction to modify an object. This lock prevents other transactions from taking any other lock (S-LOCK or X-LOCK) on the object. Only one transaction can hold an exclusive lock at a time.

Most real world DBMSs have many more types of locks that build on these and the concepts discussed in this lecture.

Transactions must request locks (or upgrades) from the lock manager. The lock manager grants or blocks requests based on what locks are currently held by other transactions. Transactions must release locks when they no longer need them to free up the object. The lock manager updates its internal lock-table with information about which transactions hold which locks and which transactions are waiting to acquire locks.

The DBMS's lock-table does not need to be durable since any transaction that is active (i.e., still running) when the DBMS crashes is automatically aborted.

However, locks alone are not enough to achieve serializability (slide 11 example). Locks need to be complemented by a concurrency control protocol that ensures locks are used in a way that satisfy correctness guarantees.

2 Two-Phase Locking

Two-Phase locking (2PL) is a pessimistic concurrency control protocol that uses locks to determine whether a transaction is allowed to access an object in the database on the fly. The protocol does not need to know all of the queries that a transaction will execute ahead of time.

Phase #1– Growing: In the growing phase, each transaction requests the locks that it needs from the DBMS's lock manager. The lock manager grants/denies these lock requests.

Phase #2– Shrinking: Transactions enter the shrinking phase immediately after they release their first lock. In the shrinking phase, transactions are only allowed to release locks. They are not allowed to acquire new ones.

On its own, 2PL is sufficient to guarantee conflict serializability. It generates schedules whose precedence graph is acyclic. But it is susceptible to *cascading aborts*, a performance issue where a transaction aborts and then another transaction must be rolled back. This results in wasted work (slide 18), since any computation performed must be rolled back.

2PL can still have dirty reads and it can also lead to deadlocks. There are also potential schedules that are serializable but would not be allowed by 2PL (locking can limit concurrency).

Strong Strict Two-Phase Locking

A schedule is *strict* if any value written by a transaction is never read or overwritten by another transaction until the first transaction commits. *Strong Strict 2PL* (also known as *Rigorous 2PL*) is a variant of 2PL where the transactions only release locks when they commit (slide 19).

The advantage of this approach is that the DBMS does not incur cascading aborts. The DBMS can also reverse the changes of an aborted transaction by restoring the original values of modified tuples. However, Strong Strict 2PL generates more cautious/pessimistic schedules that limit concurrency.

Strict 2PL is another variant that makes slightly weaker guarantees (reintroducing the dirty read / cascading aborts problems) by only releasing shared locks during the shrinking phase.

Universe of Schedules (slide 26)

$\text{SerialSchedules} \subset \text{StrongStrict2PL} \subset \text{ConflictSerializableSchedules}$
 $\subset \text{ViewSerializableSchedules} \subset \text{AllSchedules}$

3 Deadlock Handling

A *deadlock* is a cycle of transactions waiting for locks to be released by each other. A classic example of a deadlock occurs when Transaction 1 holds a lock on resource A and needs resource B, while Transaction 2 holds resource B and needs resource A. There are two approaches to handling deadlocks in 2PL: detection and prevention.

Approach #1: Deadlock Detection

To detect deadlocks, the DBMS creates a *waits-for* graph where transactions are nodes (slides 30, 31), and there exists a directed edge from T_i to T_j if transaction T_i is waiting for transaction T_j to release a lock. The system will periodically check for cycles in the waits-for graph (usually with a background thread) and then make a decision on how to break it. Latches are not needed when constructing the graph since if the DBMS misses a deadlock in one pass, it will find it in the subsequent passes. Note that there is a

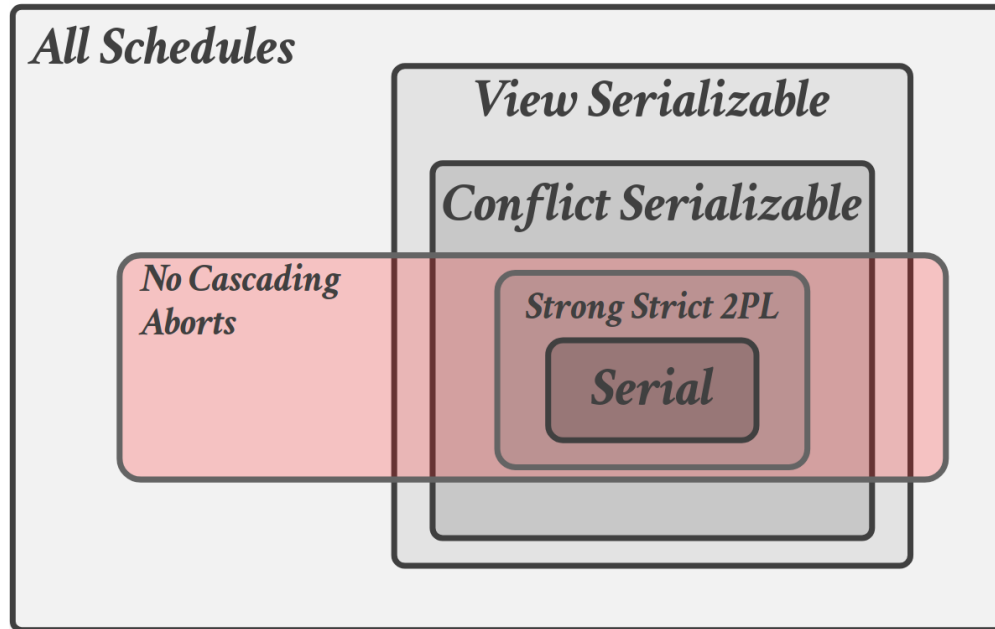


Figure 1: Relationship between schedules, including Serial, Strong Strict 2PL, Conflict Serializable, View Serializable, and schedules without cascading aborts

trade-off between the frequency of deadlock checks (uses CPU cycles) and the wait time until a deadlock is broken. A common heuristic may be to quickly search for any 2-cycles and then fall back to a full cycle detection algorithm if none are found or breaking the 2-cycle isn't sufficient.

When the DBMS detects a deadlock, it will select a “victim” transaction to abort to break the cycle. The victim transaction will either restart or abort depending on how the application invoked it. The DBMS can consider multiple transaction properties when selecting a victim to break the deadlock:

1. By age (newest or oldest timestamp).
2. By progress (least/most queries executed).
3. By the # of items already locked.
4. By the # of transactions needed to rollback with it.
5. # of times a transaction has been restarted in the past (to avoid starvation).

There is no one choice that is better than others. Many systems use a combination of these factors.

After selecting a victim transaction to abort, the DBMS can also decide on how far to rollback the transaction's changes. It can either rollback the entire transaction or just enough queries to break the deadlock (some DBMSs will establish “savepoints” throughout a transaction that can be rolled back to).

Approach #2: Deadlock Prevention

Instead of letting transactions try to acquire any lock they need and then deal with deadlocks afterwards, deadlock prevention 2PL stops transactions from causing deadlocks before they occur. When a transaction tries to acquire a lock held by another transaction (which could cause a deadlock), the DBMS can kill one of them. To implement this, transactions are assigned priorities (potentially based on timestamps with older transactions have higher priority). These schemes guarantee no deadlocks because only one type of direction is allowed when waiting for a lock. When a transaction restarts, the DBMS reuses the same timestamp to prevent it from getting starved for resources.

There are two ways to kill transactions under deadlock prevention (slide 36):

- **Wait-Die (“Old Waits for Young”)**: If the requesting transaction has a higher priority than the holding transaction, it waits. Otherwise, it aborts (dies).
- **Wound-Wait (“Young Waits for Old”)**: If the requesting transaction has a higher priority than the holding transaction, the holding transaction aborts (gets wounded) and releases the lock. Otherwise, the requesting transaction waits.

To remember these - if the protocol is X-Y, then if the requesting transaction has a higher priority, it will X, and if the requesting transaction has a lower priority, it will Y.

4 Lock Granularities

If a transaction wants to update one billion tuples, it has to ask the DBMS’s lock manager for a billion locks. This will be slow because the transaction has to take latches in the lock manager’s internal lock table data structure as it acquires/releases locks.

Alternatively, if a transaction locks the entire table when it only needs to read one value, there are less opportunities for parallelism. To handle this trade-off, the DBMS uses a *lock hierarchy* to simultaneously handle locks at different granularity levels. For example, it could acquire a single lock on the table with one billion tuples instead of one billion separate locks.

When a transaction acquires a lock for an object in this hierarchy, it implicitly acquires the locks for all its children objects, so the one-write lock couldn’t grab any of the tuple locks. However, if no lock is held on the table, multiple tuple-level locks are allowed on different tuples, allowing parallelism.

Database Lock Hierarchy:

1. Database level (Slightly Rare)
2. Table level (Very Common)
3. Page level (Common)
4. Tuple level (Very Common)
5. Attribute level (Rare)

Importantly, if a transaction is using tuple-level locks, it needs to communicate that no other transaction can grab a page-level lock (or anything higher) since that would conflict. To facilitate this, **intention locks** are implicit locks that signal that there are explicit locks held at lower levels.

- **Intention-Shared (IS)**: Indicates explicit locking at a lower level with shared locks.
- **Intention-Exclusive (IX)**: Indicates explicit locking at a lower level with exclusive or shared locks.
- **Shared+Intention-Exclusive (SIX)**: The sub-tree rooted at that node is locked explicitly in shared mode, and explicit locking is being done at a lower level with exclusive-mode locks.

With hierarchical locking, the DBMS can automatically switch to coarser-grained locks when a transaction acquires many lower level locks. This reduces the number of requests the Lock Manager has to handle.

5 Locking in Practice

In practice, applications don’t usually explicitly take locks and the DBMS infers which locks are necessary for a query. Sometimes explicit locking is necessary to improve performance: some databases support a FOR UPDATE/FOR SHARE syntax that can go at the end of a SELECT query to indicate that a DBMS should take a particular kind of lock. This is particularly useful for read-modify-write workloads: without it, it’s possible getting the first shared lock for the read query would succeed and the exclusive lock for the write

would fail, resulting in a rollback and wasted work. With it, you can directly try and acquire an exclusive lock for the read (knowing it will be immediately followed by a write).

Some systems also support a SKIP LOCKED suffix that skips any tuples that the DBMS can't acquire a lock for. This is handy for accessing queues inside a DBMS.

Lecture #19: Timestamp Ordering Concurrency Control

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Timestamp Ordering Concurrency Control

Timestamp ordering (T/O) is an optimistic class of concurrency control protocols where the DBMS assumes that transaction conflicts are rare. Instead of requiring transactions to acquire locks before they are allowed to read/write to a database object, the DBMS instead uses timestamps to determine the serializability order of transactions.

Each transaction T_i is assigned a unique fixed timestamp $TS(T_i)$ that is monotonically increasing. Different schemes assign timestamps at different times during the transaction. Some advanced schemes even assign multiple timestamps per transaction.

If $TS(T_i) < TS(T_j)$, then the DBMS must ensure that the execution schedule is equivalent to the serial schedule where T_i appears before T_j .

There are multiple timestamp allocation implementation strategies. The DBMS can use the system clock as a timestamp, typically using UTC to avoid issues from daylight saving. Some DBMSs can get accurate clock time from satellites. Another option is to use a logical counter. However, this has issues with overflow and with maintaining the counter across a distributed system with multiple machines. There are also hybrid approaches that use a combination of both methods.

2 Optimistic Concurrency Control (OCC)

Optimistic concurrency control (OCC) is an optimistic concurrency control protocol which uses timestamps to validate transactions. OCC works best when the number of conflicts is low. This is when either all of the transactions are read-only or when transactions access disjoint subsets of data. If the database is large and the workload is not skewed, then there is a low probability of conflict, making OCC a good choice.

In OCC, the DBMS creates a *private workspace* for each transaction. All modifications of the transaction are applied to this workspace. Any object written is copied to the workspace and modified there, any object read is copied to the workspace only if the isolation level guarantees repeatable read. No other transaction can read the changes made by another transaction in its private workspace.

When a transaction commits, the DBMS compares the transaction's workspace *write set* to see whether it conflicts with other transactions. If there are no conflicts, the write set is installed into the "global" database.

OCC consists of three phases:

1. **Read Phase:** Here, the DBMS tracks the read/write sets of transactions and stores their writes in a private workspace. The DBMS copies every tuple accessed to its private workspace to ensure repeatable reads.
2. **Validation Phase:** When a transaction commits, it is assigned a unique timestamp and the DBMS checks whether it conflicts with other transactions.

3. **Write Phase:** If validation succeeds, the write timestamp is assigned to all modified objects in the private workspace, and the DBMS installs the private workspace's changes to the global database atomically. Otherwise, it aborts and restarts the transaction.

The read phase is where the transaction performs all the works, the validation and the write phase consists the commit protocol.

Validation Phase

The DBMS assigns transactions timestamps when they enter the validation phase. To ensure only serializable schedules are permitted, the DBMS checks T_i against other transactions for RW and WW conflicts and makes sure that all conflicts go one way.

- **Approach 1:** Forward validation (from older transactions to younger transactions)
- **Approach 2:** Backward validation (from younger transactions to older transactions)

In forward validation, the DBMS checks the timestamp ordering of the committing transaction with all other running transactions that have not yet committed. Transactions that have not yet entered the validation phase are assigned a timestamp of ∞ .

If $TS(T_i) < TS(T_j)$, then one of the following three conditions must hold:

1. T_i completes its Write phase before T_j begins its Read phase (serial ordering) (slide 16).
2. T_i completes its Write phase before T_j starts its Write phase, and T_i does not write to any object read by T_j (slide 17).
 - $WriteSet(T_i) \cap ReadSet(T_j) = \emptyset$.
3. T_i completes its Read phase before T_j completes its Read phase, and T_i does not write to any object that is either read or written by T_j (slide 20).
 - $WriteSet(T_i) \cap ReadSet(T_j) = \emptyset$, and $WriteSet(T_i) \cap WriteSet(T_j) = \emptyset$.

In backward validation, the DBMS checks the timestamp ordering of the committing transaction with the Read/Write sets of transactions that have already committed since the current transaction started, using the same three conditions as above. This is the more common approach compared to forward validation.

Ideal scenario for OCC: OCC works well when the number of conflicts is low, ideally when all transactions are read-only, or when transactions access disjoint subsets of data.

Potential Issues:

- High overhead for copying data locally into the transaction's private workspace.
- Validation/Write phase bottlenecks.
- Aborts are potentially more wasteful than in other protocols because they only occur after a transaction has already executed.
- Suffers from timestamp allocation bottleneck.

3 Dynamic Databases and The Phantom Problem

In our previous discussions, we have considered transactions that operate on a static set of objects within the database. However, when transactions perform insertions, updates, and deletions, we encounter a new set of complications.

When a transaction scans a range more than once and another transaction inserts or removes tuples that fall within the range between the scans, the scanning transaction can get different results across multiple scans. This is referred to as phantom read.

The phantom problem arises when transactions only lock existing records, neglecting those that are in the process of being created. Conflict serializability on reads and writes of individual items guarantees serializability **only** if the database's set of objects is fixed.

Approaches to Address the Phantom Problem:

1. **Lock Everything:**

Transactions may take the lock of the entire table or every page, preventing insertions and deletions from other transactions during its execution. This approach is expensive as it locks more than the transaction needs.

2. **Re-Execute Scans:**

Transactions may re-run queries at commit time to check for different results, indicating missed changes due to new or deleted records. The DBMS keeps track of the WHERE clauses for all queries executed by the transaction. At commit time, it re-executes the scans to ensure that the results remain consistent.

3. **Predicate Locking:**

This involves acquiring locks based on the predicates of the queries, ensuring that any data that satisfies the predicate cannot be modified by other transactions. Originally proposed in System R, this scheme is not widely implemented because of its complexity. However, systems like HyPer utilize a form of precision locking that is akin to predicate locking.

4. **Index Locking:**

Utilizing index keys to protect ranges of data, preventing phantoms by ensuring that no new data can fall within the locked ranges. Different schemes are employed to prevent phantoms using index locking:

- **Key-Value Locks:** Locks on individual key-values in an index, including virtual keys for non-existent values.
- **Gap Locks:** Locks on the gap following a key-value, preventing insertion in these gaps.
- **Key-Range Locks:** Locks on a range of keys, from one existing key to the next.
- **Hierarchical Locking:** Allows transactions to hold broader key-range locks with different modes, reducing lock manager overhead.

In the absence of a suitable index, transactions must fall back to the first approach to lock every page in the table or the entire table itself to prevent changes that could lead to phantoms. The Index Locking approach is the most common among the four.

4 Isolation Levels

Serializability is useful because it allows programmers to ignore concurrency issues but enforcing it may allow too little parallelism and limit performance. We may want to use a weaker level of consistency to improve scalability.

Isolation levels control the extent that a transaction is exposed to the actions of other concurrent transactions.

Anomalies:

- **Dirty Read:** Reading uncommitted data.
- **Unrepeatable Reads:** Redoing a read retrieves a different result.
- **Lost Updates:** Transaction overwrites data of another concurrent transaction.
- **Phantom Reads:** Insertion or deletions result in different results for the same range scan queries.

Isolation Levels (Strongest to Weakest):

1. **SERIALIZABLE**: No Phantoms, all reads repeatable, and no dirty reads.
 - Possible implementation: Strict 2PL + Phantom Protection (e.g., index locks).
2. **REPEATABLE READS**: Phantoms may happen.
 - Possible implementation: Strict 2PL.
3. **READ-COMMITTED**: Phantoms, unrepeatable reads, and lost updates may happen.
 - Possible implementation: Strict 2PL for exclusive locks, immediate release of the shared lock after a read.
4. **READ-UNCOMMITTED**: All anomalies may happen.
 - Possible implementation: Strict 2PL for exclusive locks, no shared locks for reads.

The isolation levels defined as part of SQL-92 standard only focused on anomalies that can occur in a 2PL-based DBMS. An application sets a per-transaction isolation level *before* it starts executing queries.

Lecture #20: Multi-Version Concurrency Control I

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Multi-Version Concurrency Control

Multi-Version Concurrency Control (MVCC) is a larger concept than just a concurrency control protocol. It involves all aspects of the DBMS's design and implementation. MVCC is the most widely used scheme in DBMSs. It is now used in almost every new DBMS implemented in last 10 years. Even some systems (e.g., NoSQL) that do not support multi-statement transactions use it.

With MVCC, the DBMS maintains multiple physical versions of a single logical object in the database. When a transaction writes to an object, the DBMS creates a new version of that object. When a transaction reads an object, it reads the newest version that existed when the transaction started.

The fundamental concept/benefit of MVCC is that writers do not block readers and readers do not block writers. This means that one transaction can modify an object while other transactions read old versions. Writers may still block other writers if they are writing the same object, since there is still a lock on the versions related to the database object. See slide 8 for an example.

One advantage of using MVCC is that read-only transactions can read a consistent **snapshot** of the database without using locks of any kind, and it naturally supports Snapshot Isolation (SI). In MVCC, timestamps are used to determine visibility of versions to transactions. Additionally, MVCC without garbage collection allows the DBMS to support *time-travel queries*, which are queries based on the state of the database at some other point in time (e.g. performing a query on the database as it was 3 hours ago).

A typical MVCC-based database design will:

1. Have a versioned storage which stores different versions of the same logical object. (*Note: Do not do this!*)
2. Takes a snapshot of the database (by copying the transaction status table) when a transaction starts.
3. Use the snapshot to determine which versions of objects are visible to the transaction.

Snapshot Isolation

Snapshot Isolation involves providing a transaction with a consistent snapshot of the database when the transaction started. Data values from a snapshot consist of only values from committed transactions, and the transaction operates in complete isolation from other transactions until it finishes. This is ideal for read-only transactions since they do not need to wait for writes from other transactions. Writes are maintained in a transaction's private workspace or written to the storage with transaction metadata and only become visible to the database once the transaction successfully commits.

Write Conflicts If two transactions update the same object, the first writer wins.

Write Skew Anomaly can occur in Snapshot Isolation when two concurrent transactions modify different objects resulting in non-serializable schedules. For example, if one transaction changes all white marbles to black and the other changes all black marbles to white, the outcome may not correspond to any serializable schedule. See slides 28 to 31 for details.

Serializable Snapshot Isolation

Serializable snapshot isolation keeps the benefits of snapshot isolation (writers do not block readers, readers do not block writers) and also avoids the aforementioned anomalies, thus providing serializable isolation. Essentially, there are runtime checks to ensure that possible anomalies are avoided, with transactions that may cause anomalies being aborted and retried.

A crucial ingredient of efficiently ensuring that anomalies are avoided are anti-dependencies. There is an anti-dependency from a transaction T1 to a transaction T2 if T1 reads a version of item x, and T2 produces a version of x that is later in the version order (i.e. newer) than the version read by T1. If there is an anti-dependency from T1 to T2 and from T2 to T1, serializability is violated and at least one of T1 and T2 must be aborted and retried.

There are five important MVCC design considerations:

1. Concurrency Control Protocol
2. Version Storage
3. Garbage Collection
4. Index Management
5. Deletes

The choice of concurrency protocol is between the approaches discussed in previous lectures (two-phase locking, timestamp ordering, optimistic concurrency control).

Lecture #21: Multi-Version Concurrency Control II

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Design consideration: Version Storage

This is how the DBMS will store the different physical versions of a logical object and how transactions find the newest version visible to them.

The DBMS uses the tuple's pointer field to create a **version chain** per logical tuple, which is essentially a linked list of versions sorted by timestamp. This allows the DBMS to find the version that is visible to a particular transaction at runtime. Indexes always point to the “head” of the chain, which is either the newest or oldest version depending on implementation. A thread traverses chain until it finds the correct version. Different storage schemes determine where/what to store for each version.

Approach #1: Append-Only Storage

All physical versions of a logical tuple are stored in the same table space. Versions are mixed together in the table and each update just appends a new version of the tuple into the table and updates the version chain. The chain can either be sorted *oldest-to-newest* (O2N) which requires chain traversal on look-ups, or *newest-to-oldest* (N2O), which requires updating index pointers for every new version (i.e. no need to traverse the chain. This is typically the better approach because most txns only care about the newest version). See slides 8 to 9 for an example.

Approach #2: Time-Travel Storage

The DBMS maintains a separate table called the time-travel table which stores older versions of tuples. On every update, the DBMS copies the old version of the tuple to the time-travel table and overwrites the tuple in the main table with the new data. Pointers of tuples in the main table point to past versions in the time-travel table. See slides 10 for an example.

Approach #3: Delta Storage

Like time-travel storage, but instead of the entire past tuples, the DBMS only stores the deltas, or changes between tuples in what is known as the delta storage segment. Transactions can then recreate older versions by iterating through the deltas in reverse order and applying them. This results in faster writes than time-travel storage but slower reads. See slides 11 for an example.

2 Design consideration: Versioning with Large Values

When using Append-Only or Time-Travel storage, if a txn does not modify data belonging to the columns that make use of overflow pages, then most DBMS will reuse the overflow pointers instead of copying the data. If multiple physical versions of a tuple can point to the same overflow page, the DBMS must now track these shared references to determine when an overflow page can be safely reclaimed later on. These are some approaches that can be taken.

Approach #1: Reference Counting

The DBMS uses an internal metadata table that tracks the number of physical versions that point to an overflow page.

Approach #2: Garbage Collection

The DBMS uses the version chain to identify what physical tuples point to the same overflow page before deciding to reclaim the space. This is the more commonly used approach.

3 Design consideration: Garbage Collection

The DBMS needs to remove *reclaimable* physical versions from the database over time. A version is reclaimable if no active transaction can “see” that version or if it was created by a transaction that was aborted. We can either perform tuple level, or transaction level garbage collection.

Approach #1: Tuple-level GC

With tuple-level garbage collection, the DBMS finds old versions by examining tuples directly. There are two approaches to achieve this:

- **Background Vacuuming:** Separate threads periodically scan the table and look for reclaimable versions. This works with any version storage scheme. A simple optimization is to maintain a “dirty page bitmap,” which keeps track of which pages have been modified since the last scan. This allows the threads to skip pages which have not changed. See slides 54 to 60 for an example.
- **Cooperative Cleaning:** Worker threads identify reclaimable versions as they traverse version chain. This only works with O2N chains. The data will never be cleaned if it is not accessed.

Approach #2: Transaction-level GC

Under transaction-level garbage collection, each transaction is responsible for keeping track of their own old versions so the DBMS does not have to scan tuples. Each transaction maintains its own read/write set. When a transaction completes, the garbage collector can use that to identify which tuples to reclaim. The DBMS determines when all versions created by a finished transaction are no longer visible. See slides 66 to 73 for an example.

How PostgreSQL prevents Transaction ID Wraparound Failures

In PostgreSQL, since transaction IDs have a fixed size (32 bits), a cluster that runs long enough could suffer from transaction ID wraparound. The counter wraps back to zero, and now transactions that were in the past appear to be in the future. PostgreSQL fixes this by running the **Background Vacuum** on every single tuple which will mark rows as “frozen”, indicating that they were inserted by transactions that were committed sufficiently far in the past.

4 Representation of MVCC Deleted Tuples

There are two main ways of handling the slots vacated by deleted tuples

Approach #1: Reuse Slot

Allow workers to insert new tuples back into the empty slots. This is easy to do with append-only storage. If done with delta storage, it could destroy the temporal locality of the tuples due to possible intermixing of tuple versions.

Approach #2: Leave Slot Unoccupied

Workers are restricted to insert new tuples to slots that were not previously occupied. This ensures that new versions are written to slots that are closer together in physical storage. The trade-off is that the DBMS needs a separate compaction mechanism to reclaim the holes left behind.

5 Design Consideration: Block Compaction

Block compaction is the process of coalescing/compacting less-than-full data blocks into fewer blocks and releasing the empty blocks back to storage. PostgreSQL implements this using *VACUUM FULL*.

Ideally the DBMS will want to store tuples that are likely to be accessed together within a time window with the help of Block Compaction. This can be implemented in several ways.

Approach #1: Time Since Last Update

Leverage the BEGIN-TS in each tuple to group together tuples that were modified around the same time.

Approach #2: Time Since Last Access

Expensive to maintain unless tuple has READ-TS

Approach #3: Application-level Semantics

Use Application-level semantics to identify higher-level relations amongst tuples from the same table.

6 Design consideration: Index Management

All primary key (pkey) indexes always point to version chain head. How often the DBMS has to update the pkey index depends on whether the system creates new versions when a tuple is updated. If a transaction updates a pkey attribute(s), then this is treated as a DELETE followed by an INSERT.

Managing secondary indexes is more complicated. There are two approaches to handling them.

Approach #1: Logical Pointers

The DBMS uses a fixed identifier per tuple that does not change. This requires an extra indirection layer that maps the logical id to the physical location of the tuple. Then, updates to tuples can just update the mapping in the indirection layer.

Approach #2: Physical Pointers

The DBMS uses the physical address to the version chain head. This requires updating every index when the version chain head is updated, which can be very expensive.

Imagine a table with a primary index, and multiple secondary indices, using physical pointers requires each secondary index to point to the tuple version chain's physical address, and each update to the tuple requires an update to all the secondary indices. With a logical pointer approach, the secondary indices point to the location in the primary index corresponding to the tuple, thus requiring less overhead when the tuple is updated.

MVCC duplicate key problem

MVCC DBMS indexes (usually) do not store version information about tuples with their keys. Instead, every index must support duplicate keys from different snapshots, since the same key may point to different logical tuples in different snapshots.

MVCC Duplicate Key Problem describes this need to support duplicate keys in MVCC DBMS indexes when multiple transactions need multiple versions of the same logical tuple. For example, say TXN 1 points to version 0 of a logical tuple, and TXN 2 creates a version 1 of that same logical tuple. Our index would need to point to both of these versions for both TXNs so the proper version is accessible to each transaction at all times.

Workers may get back multiple entries for a single fetch, and they then must follow the pointers to find their proper physical version.

7 Design consideration: Deletes

The DBMS physically deletes a tuple from the database only when all versions of a logically deleted tuple are not visible. If a tuple is deleted, then there cannot be a new version of that tuple after the newest version. This means no write-write conflicts, and the first-writer wins.

We need a way to denote that a tuple has been logically deleted at some point in time. There are two approaches to this.

Approach #1: Deleted Flag

Maintain a flag to indicate that the logical tuple has been deleted after the newest physical version. This can either be in the tuple header or a separate column.

Approach #2: Tombstone Tuple

Create an empty physical version to indicate that a logical tuple is deleted. Use a separate pool for tombstone tuples with only a special bit pattern in version chain pointer to reduce storage overhead.

Lecture #22: Database Logging

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Crash Recovery

Recovery algorithms are techniques to ensure database consistency, transaction atomicity, and durability despite failures. When a crash occurs, all the data in volatile memory that has been committed but not yet flushed to disk is at risk of being lost. This is not ideal since the user expects their committed data changes to be persisted. Recovery algorithms act to prevent loss of information after a crash.

Why this matters: Volatile memory is significantly faster than non-volatile storage (like SSDs or HDDs), so DBMSs use volatile memory extensively for performance. However, this creates the recovery challenge - we need smart algorithms that allow us to benefit from fast memory while ensuring we don't lose data.

Every recovery algorithm has two parts:

- Actions during normal transaction processing to ensure that the DBMS can recover from a failure.
- Actions after a failure to recover the database to a state that ensures atomicity, consistency, and durability.

The key primitives used in recovery algorithms are UNDO and REDO. Not all algorithms use both primitives.

- **UNDO:** The process of removing the effects of an incomplete or aborted transaction.
- **REDO:** The process of re-applying the effects of a committed transaction for durability.

2 Buffer Pool Management Policies

The DBMS needs to ensure the following guarantees:

- The changes for any transaction are durable once the DBMS has told somebody that it committed.
- No partial changes are durable if the transaction aborted.

A *steal policy* dictates whether the DBMS allows an uncommitted transaction to overwrite the most recent committed value of an object in non-volatile storage.

- **STEAL:** The DBMS can write uncommitted changes to disk before the transaction completes. (*Mnemonic: The system "steals" dirty pages from active transactions and writes them to disk.*)
- **NO-STEAL:** The DBMS cannot write uncommitted changes to disk before the transaction completes. (*Mnemonic: The system cannot "steal" dirty pages - they must stay in memory until commit time.*)

A *force policy* dictates whether the DBMS requires that all updates made by a transaction are reflected on non-volatile storage before the transaction is allowed to commit.

- **FORCE:** All changes must be written to disk at commit time. (*Mnemonic: The system "forces" all changes to disk at commit time.*)
- **NO-FORCE:** Changes are not required to be written to disk at commit time. (*Mnemonic: The system does not "force" changes to disk - they can be written later.*)

These policies create four possible combinations, but only two are commonly used in practice:

- **NO-STEAL + FORCE**: Simple recovery but poor runtime performance
- **STEAL + NO-FORCE**: Excellent runtime performance but more complex recovery

The other combinations (STEAL + FORCE, NO-STEAL + NO-FORCE) offer few practical advantages and are rarely implemented.

3 Naive NO-STEAL + FORCE

The simplest buffer pool management policy to implement is called *NO-STEAL + FORCE*. In this policy, the DBMS never has to undo changes of an aborted transaction because the changes were not written to disk. It also never has to redo changes of a committed transaction because all the changes are guaranteed to be written to disk at commit time. An example of NO-STEAL + FORCE is shown in Figure 1.

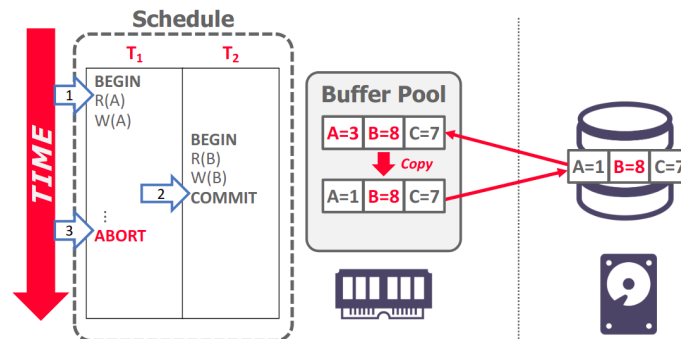


Figure 1: DBMS using NO-STEAL + FORCE Example – All changes from a transaction are only written to disk when the transaction is committed. Once the schedule begins at Step #1, changes from T_1 and T_2 are written to the buffer pool. Because of the FORCE policy, when T_2 commits at Step #2, all of its changes must be written to disk. To do this, the DBMS makes a copy of the page in memory, applies only the changes from T_2 , and writes it back to disk. This is because NO-STEAL forbids the system from writing uncommitted changes from T_1 to disk. At Step #3, it is trivial for the DBMS to rollback T_1 since no dirty changes from T_1 are on disk.

A critical limitation of this naive NO-STEAL + FORCE approach is that **all of the data (i.e., the write set) that a transaction needs to modify must fit into memory**. Otherwise, that transaction cannot execute because the DBMS is not allowed to write out dirty pages to disk before the transaction commits. More frequent writes can also lead to faster wearing of storage devices like SSDs.

4 Shadow Paging (Smarter NO-STEAL + FORCE)

Shadow Paging is an improvement upon the naive NO-STEAL + FORCE scheme where the DBMS implements a copy-on-write approach to maintain two separate versions of the database:

- *master*: Contains only changes from committed transactions.
- *shadow*: Temporary database with changes made from uncommitted transactions.

Updates are only made in the shadow copy. When a transaction commits, the shadow copy is atomically switched to become the new master. The old master is eventually garbage collected. This is an example of a NO-STEAL + FORCE system. A high-level example of shadow paging is shown in Figure 2.

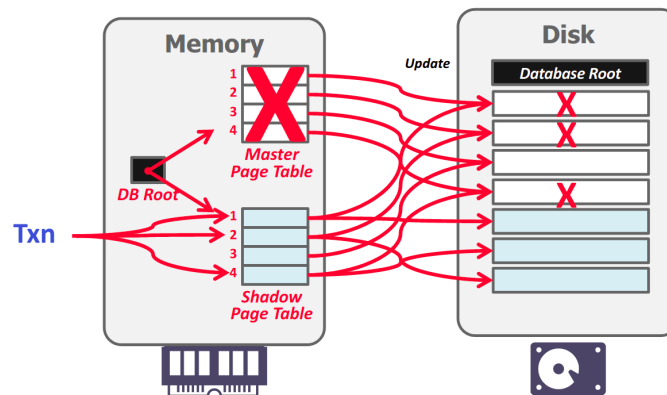


Figure 2: Shadow Paging – The database root points to a master page table which in turn points to the pages on disk (all of which contain committed data). When an updating transaction occurs, a shadow page table is created that points to the same pages as the master. Modifications are made to a temporary space on disk and the shadow table is updated. Read-only transactions will continue to read from the master page table. To complete the commit, the database root pointer is redirected to the shadow table, which becomes the new master. After that, all pages that were modified are now part of the committed database. (Note: For this example, we are assuming only one transaction is updating the database at a time. If multiple concurrent transactions are updating the database, multiple shadow page tables would be needed, and group fitting would be required to merge their changes when redirecting the root pointer on commit.)

Implementation

The DBMS organizes the database pages in a tree structure where the root is a single disk page. There are two copies of the tree, the *master* and *shadow*. The root always points to the current master copy. When a transaction executes, it only makes changes to the shadow copy.

When a transaction wants to commit, the DBMS must install its updates. To do this, it only has to overwrite the root to make it point to the shadow copy of the database, thereby swapping the master and shadow. Before overwriting the root, none of the transaction's updates are part of the disk-resident database. After overwriting the root, all the transaction's updates are part of the disk resident database. This overwriting of the root can be done atomically.

Recovery

- **Undo:** Remove the shadow pages. Leave the master and DB root pointer alone.
- **Redo:** Not needed at all.

Disadvantages

Shadow paging has several key limitations:

- **High overhead:** Copying the entire page table is expensive. In practice, only paths in the tree that lead to updated leaf nodes need to be copied, not the entire tree.
- **Expensive commits:** Commits require the page table, root page, and every updated page to be flushed, causing lots of writes to random non-contiguous pages.
- **Data fragmentation:** Potentially related data gets divided between different pages as updates occur.
- **Concurrency limitations:** While technically possible to support multiple shadow pages for different transactions, this approach becomes increasingly inefficient with concurrent writers.

5 Journal File (SQLite Pre-2010)

When a transaction modifies a page, the DBMS copies the original page to a separate journal file before overwriting the master version. After restarting, if a journal file exists, the DBMS collects the original pages and blindly overwrites the pages with those original pages to restore data to the state before the uncommitted transaction.

The journal file approach allows for STEAL (unlike shadow paging), solving the memory limitation problem by permitting dirty pages to be written to disk before commit. However, it's typically limited to one writer at a time, which was acceptable for SQLite's design goals prior to 2010. After 2010, SQLite switched their implementation to use a write-ahead log instead for better performance.

6 Write-Ahead Logging

With *write-ahead logging*, the DBMS records all the changes made to the database in a log file (on stable storage) before the change is made to a disk page. The log contains sufficient information to perform the necessary undo and redo actions to restore the database after a crash. The DBMS must write to disk the log file records that correspond to changes made to a database object before it can flush that object to disk. An example of WAL is shown in Figure 3.

WAL is an example of a STEAL + NO-FORCE system:

- STEAL allows dirty pages to be written to disk before commit, solving the memory limitation issue
- NO-FORCE means we don't need to write all changes to disk at commit time, improving performance
- However, this introduces the need to handle both UNDO (for rolled-back transactions that wrote to disk) and REDO (for committed transactions whose changes weren't forced to disk)

In shadow paging, the DBMS was required to perform writes to random non-contiguous pages on disk. Write-ahead logging instead works with sequential writes, which are much faster. Thus, almost every modern DBMS uses write-ahead logging (WAL) because it has the fastest runtime performance, even though the DBMS's recovery time with WAL is slower than shadow paging because it has to replay the log.

Implementation

The DBMS first stages all of a transaction's log records in volatile storage. All log records pertaining to an updated page are then written to non-volatile storage before the page itself is allowed to be overwritten in

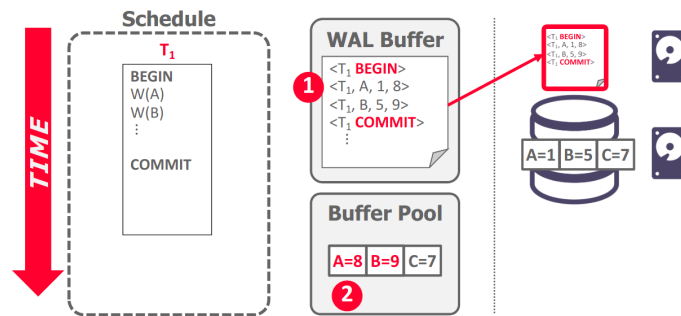


Figure 3: Write Ahead Logging – When the transaction begins, all changes are recorded in the WAL buffer in memory before being made to the buffer pool. At the time of commit, the WAL buffer is flushed out to disk. The transaction result can be written out to disk, once the WAL buffer is safely on disk.

non-volatile storage. A transaction is not considered committed until all its log records have been written to stable storage.

When the transaction starts, write a <BEGIN> record to the log for each transaction to mark its starting point.

When a transaction finishes, write a <COMMIT> record to the log and make sure all log records are flushed before it returns an acknowledgment to the application.

Each log entry contains information necessary to rewind or replay the changes to a single object:

- Transaction ID.
- Object ID.
- Before Value (used for UNDO).
- After Value (used for REDO).
- ...Some system dependent data (e.g. timestamp, checksum)

Note: When using Multi-Version Concurrency Control (MVCC), the "Before Value" may not be needed for UNDO operations since previous versions are already maintained by the system - an example of convenient synergy between concurrency control and recovery mechanisms.

The DBMS must flush all of a transaction's log entries to disk before it can tell the outside world that a transaction has successfully committed. The system can use the "group commit" optimization to batch multiple log flushes together to amortize overhead. Flushes happen either when the log buffer is full, or if sufficient time has passed between successive flushes. The DBMS can write dirty pages to disk whenever it wants to, as long as it is after flushing the corresponding log records.

Buffer Pool Policies Tradeoff

Most DBMSs employ the NO-FORCE + STEAL policy due to its superior runtime performance compared to FORCE + NO-STEAL. However, this tradeoff means:

- **Runtime Performance:** STEAL + NO-FORCE is much faster during normal operation
- **Recovery Speed:** FORCE + NO-STEAL recovers faster after a crash

Change Data Capture

WAL can also serve purposes beyond recovery. Since the log contains a complete sequence of changes to the database, it can be used to propagate changes to external systems. This approach, known as Change Data Capture (CDC), allows other systems to subscribe to database changes and stay synchronized with the primary database. The same log that ensures durability for recovery can efficiently feed data to replicas, data warehouses, or microservices.

7 Logging Schemes

The contents of a log record can vary based on the implementation.

Physical Logging:

- Record the byte-level changes made to a specific location in the database.
- Example: git diff

Logical Logging:

- Record the high level operations executed by transactions.
- Not necessarily restricted to a single page.
- Requires less data written in each log record than physical logging because each record can update multiple tuples over multiple pages. However, it is difficult to implement recovery with logical logging when there are concurrent transactions in a non-deterministic concurrency control scheme. Additionally recovery takes longer because you must re-execute every transaction.
- Example: The UPDATE, DELETE, and INSERT queries invoked by a transaction.

Physiological Logging:

- Hybrid approach where log records target a single page but does not specify data organization of the page. That is, identify tuples based on a slot number in the page without specifying exactly where in the page the change is located. Therefore the DBMS can reorganize pages after a log record has been written to disk.
- Most common approach used in DBMSs.

8 Checkpoints

The main problem with a WAL-based DBMS is that the log file will grow forever. Some database systems have been running continuously for decades, potentially accumulating petabytes of logs if not managed properly. After a crash, the DBMS would have to replay the entire log, which can take an impractically long time.

Thus, the DBMS periodically takes a *checkpoint* where it flushes all buffers out to disk. Checkpoints serve two critical purposes:

- **Log Pruning:** Old log records before the checkpoint can be safely discarded
- **Recovery Efficiency:** The DBMS only needs to perform REDO/UNDO operations from the most recent checkpoint, not from the beginning of time

How often the DBMS should take a checkpoint depends on the application's performance and downtime requirements. Taking a checkpoint too often causes the DBMS's runtime performance to degrade. But

waiting a long time between checkpoints can potentially be just as bad, as the system's recovery time after a restart increases. Therefore, it is common for DBMSs to provide tunable options for checkpoint frequency based on the application's recovery time requirements.

Blocking Checkpoint Implementation:

- The DBMS stops accepting new transactions and waits for all active transactions to complete.
- Flush all log records and dirty blocks currently residing in main memory to stable storage.
- Write a <CHECKPOINT> entry to the log and flush to stable storage.

In this implementation, the DBMS must halt everything when it takes a checkpoint to ensure a consistent snapshot. This is bad for runtime performance but makes recovery straightforward. Meanwhile, scanning the log to find uncommitted transactions is also time consuming.

Lecture #22: Database Crash Recovery

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Crash Recovery

The DBMS relies on its recovery algorithms to ensure database consistency, transaction atomicity, and durability despite failures. Each recovery algorithm is comprised of two parts:

- Actions during normal transaction processing to ensure that the DBMS can recover from a failure
- Actions after a failure to recover the database to a state that ensures the atomicity, consistency, and durability of transactions.

The key to database resilience is the management of transaction integrity and durability, particularly in failure scenarios. This foundational concept sets the stage for the introduction of the ARIES recovery algorithm.

Algorithms for Recovery and Isolation Exploiting Semantics (ARIES) is a recovery algorithm developed at IBM research in early 1990s for the DB2 system.

There are three key concepts in the ARIES recovery protocol:

- **Write Ahead Logging:** Any change is recorded in log on stable storage before the database change is written to disk (STEAL + NO-FORCE).
- **Repeating History During Redo:** On restart, retrace actions and restore database to exact state before crash.
- **Logging Changes During Undo:** Record undo actions to log to ensure action is not repeated in the event of repeated failures.

2 WAL Records

Write-ahead log records extend the DBMS's log record format to include a globally unique *log sequence number* (LSN). All log records have an LSN. A high level diagram of how log records with LSN's are written is shown in Figure 1.

Each data page contains a pageLSN, which is the LSN of the most recent update to that page. The pageLSN is updated every time a transaction modifies a record in the page.

The DBMS also keeps track of the max *LSN* flushed so far (flushedLSN). The flushedLSN in memory is updated every time the DBMS writes out the WAL buffer to disk.

Various components in the system keep track of **LSNs** that pertain to them. Section 2 shows a summary of these LSNs.

3 Normal Execution

Every transaction invokes a sequence of reads and writes, followed by a commit or abort. It is this sequence of events that recovery algorithms must have.

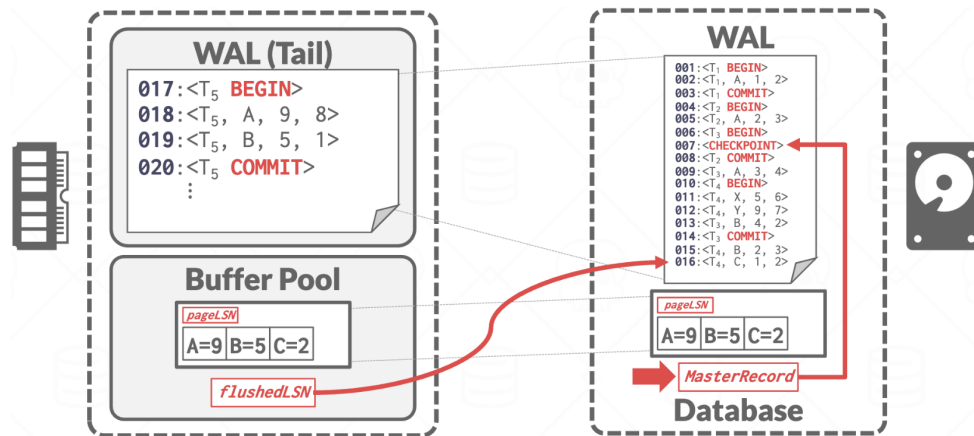


Figure 1: Writing Log Records – Each WAL has a counter of LSNs that is incremented at every step. The page also keeps a pageLSN, which stores the most recent log record that updated this page. The flushedLSN is a pointer to the last LSN that was written out to disk. Before the DBMS can write page i to disk, it must flush log at least to the point where $\text{pageLSN}_i \leq \text{flushedLSN}$. The MasterRecord points to the last successful checkpoint passed.

Name	Location	Definition
flushedLSN	Memory	Last LSN in log on disk
pageLSN	page _{x}	Newest update to page _{x}
recLSN	Dirty Page Table	Oldest update to page _{x} since it was last flushed
lastLSN	Active Transaction Table	Latest record of txn T_i (managed by transaction)
MasterRecord	Disk	LSN of latest checkpoint

Table 1: Log Sequence Number Types – A list of the different LSNs that a DBMS maintains in its internal components and data structures.

Transaction Commit

When a transaction goes to commit, the DBMS first writes COMMIT record to log buffer in memory. Then the DBMS flushes all log records up to and including the transaction's COMMIT record to disk. Note that these log flushes are sequential, synchronous writes to disk. There can be multiple log records per log page. A diagram of a transaction commit is shown in Figure 2.

Once the COMMIT record is safely stored on disk, the DBMS returns an acknowledgment back to the application that the transaction has committed. At some later point, the DBMS will write a special TXN-END record to log. This indicates that the transaction is completely finished in the system and there will not be anymore log records for it. These TXN-END records are used for internal bookkeeping and do not need to be flushed immediately.

Transaction Abort

Aborting a transaction is a special case of the ARIES undo operation applied to only one transaction.

An additional field is added to the log records called the prevLSN. This corresponds to the previous LSN for the transaction. The DBMS uses these prevLSN values to maintain a linked-list for each transaction that makes it easier to walk through the log to find its records.

A new type of record called the *compensation log record* (CLR) is also introduced. A CLR describes the

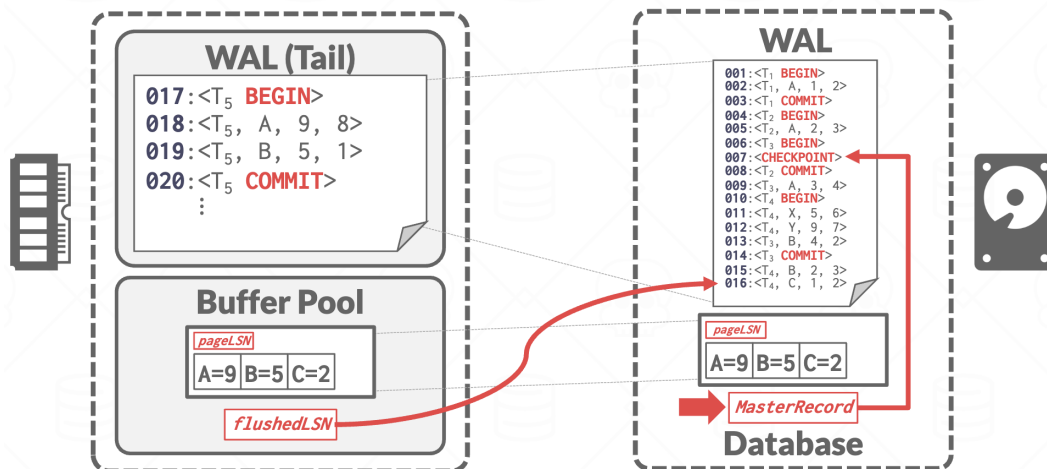


Figure 2: Transaction Commit – After the transaction commits (015), the log is flushed out and the flushedLSN is modified to point to the last log record generated. At some later point, a transaction end message is written to signify in the log that this transaction will not appear again. Then we can trim the in-memory log up to flushedLSN.

actions taken to undo the actions of a previous update record. It has all the fields of an update log record plus the *undoNextLSN* pointer (i.e., the next-to-be-undone LSN). The DBMS adds CLRs to the log like any other record but they never need to be undone. Moreover, the DBMS does not wait for CLRs to be flushed to disk before notifying the application that the transaction has been aborted. This approach ensures efficient transaction management, especially in scenarios involving transaction rollbacks.

To abort a transaction, the DBMS first appends a ABORT record to the log buffer in memory. It then undoes the transaction's updates in reverse order to remove their effects from the database. For each undone update, the DBMS creates CLR entry in the log and restore old value. After all of the aborted transaction's updates are reversed, the DBMS then writes a TXN-END log record. A diagram of this is shown in Figure 3.

4 Checkpointing

The DBMS periodically takes *checkpoints* where it writes the dirty pages in its buffer pool out to disk. This is used to minimize how much of the log it has to replay upon recovery.

The first two blocking checkpoint methods discussed below pause transactions during the checkpoint process. This pausing is necessary to ensure that the DBMS does not miss updates to pages during the checkpoint. Then, a better approach that allows transactions to continue to execute during the checkpoint but requires the DBMS to record additional information to determine what updates it may have missed is presented.

Non-Fuzzy Checkpoints

The DBMS halts the execution of transactions and queries when it takes a checkpoint to ensure that it writes a consistent snapshot of the database to disk. The is the same approach discussed in previous lecture:

- Halt the start of any new transactions.
- Wait until all active transactions finish executing.
- Flush dirty pages to disk.

TRANSACTION ABORT - CLR EXAMPLE

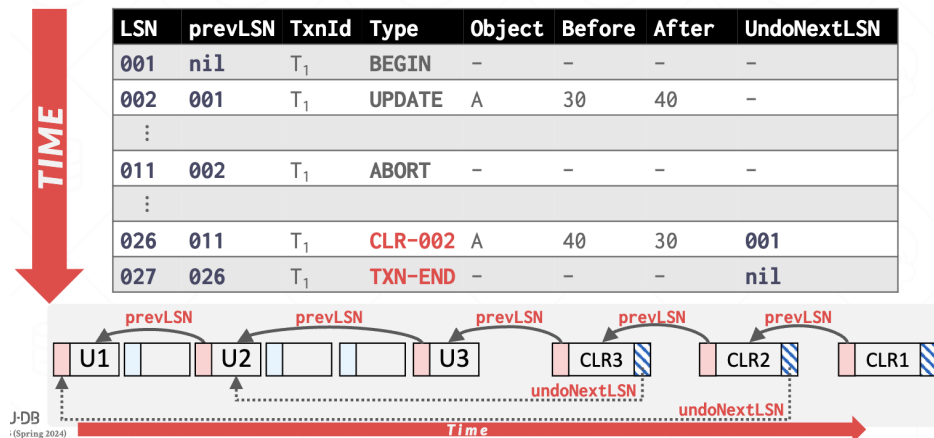


Figure 3: Transaction Abort – The DBMS maintains an LSN and prevLSN for each log record that the transaction creates. When the transaction aborts, all of the previous changes are reversed. After the log entries of the reversed changes make it to disk, the DBMS appends the TXN-END record to the log for the aborted transaction.

While this process impacts runtime performance, it significantly simplifies recovery.

Slightly Better Blocking Checkpoints

Like previous checkpoint scheme except that you the DBMS does not have to wait for active transactions to finish executing. The DBMS now records the internal system state as of the beginning of the checkpoint.

- Halt the start of any new transactions.
- Pause transactions while the DBMS takes the checkpoint.

The checkpoint process requires recording the internal state at its commencement. This includes two key components: the Active Transaction Table (ATT), which tracks ongoing transactions, and the Dirty Page Table (DPT), which lists all modified pages not yet written to disk.

Active Transaction Table (ATT): The ATT represents the state of transactions that are actively running in the DBMS. A transaction's entry is removed after the DBMS completes the commit/abort process for that transaction. For each transaction entry, the ATT contains the following information:

- `transactionId`: Unique transaction identifier
- `status`: The current "mode" of the transaction (Running, Committing, Undo Candidate).
- `lastLSN`: Most recent LSN written by transaction

Note that the ATT contains every transaction without the TXN-END log record. This includes both transactions that are either committed or abort.

Dirty Page Table (DPT): The DPT contains information about the pages in the buffer pool that were modified by uncommitted transactions. There is one entry per dirty page containing the `recLSN` (i.e., the LSN of the log record that first caused the page to be dirty).

The DPT contains all pages that are dirty in the buffer pool. It doesn't matter if the changes were caused by a transaction that is running, committed, or aborted.

Overall, the ATT and DPT are vital in both checkpointing and recovery processes. During checkpointing, they capture the database's current state, with the ATT tracking active transactions and the DPT listing

unflushed modified pages. In recovery, such as with the ARIES protocol, these tables aid in restoring the database to its consistent pre-crash state.

Fuzzy Checkpoints

A *fuzzy checkpoint* is where the DBMS allows other transactions to continue to run. This is what ARIES uses in its protocol.

The DBMS uses additional log records to track checkpoint boundaries:

- **<CHECKPOINT-BEGIN>**: Indicates the start of the checkpoint. At this point, the DBMS takes a snapshot of the current ATT and DPT, which are referenced in the **<CHECKPOINT-END>** record. Transactions that start after the checkpoint initiation are not included in the ATT.
- **<CHECKPOINT-END>**: When the checkpoint has completed. It contains the ATT + DPT, captured just as the **<CHECKPOINT-BEGIN>** log record is written.

Upon the completion of the checkpoint, the LSN of the **<CHECKPOINT-BEGIN>** record is recorded in the MasterRecord.

Fuzzy checkpoints strike a balance between performance and recoverability by minimizing transaction disruption while still capturing sufficient recovery information.

5 ARIES Recovery

The ARIES protocol is comprised of three phases. Upon start-up after a crash, the DBMS will execute the following phases as shown in Figure 4:

1. **Analysis**: Read the WAL to identify dirty pages in the buffer pool and active transactions at the time of the crash. At the end of the analysis phase the *ATT* tells the DBMS which transactions were active at the time of the crash. The *DPT* tells the DBMS which dirty pages might not have made it to disk.
2. **Redo**: Repeat all actions starting from an appropriate point in the log (even txns that will abort).
3. **Undo**: Reverse the actions of transactions that did not commit before the crash.

Analysis Phase

Start from last checkpoint found via the database's MasterRecord *LSN*.

1. Scan log forward from the checkpoint.
2. If the DBMS finds a **TXN-END** record, remove its transaction from ATT.
3. All other records, add transaction to ATT with status **UNDO**, and on commit, change transaction status to **COMMIT**.
4. For **UPDATE** log records, if page *P* is not in the DPT, then add *P* to DPT and set *P*'s *reclSN* to the log record's *LSN*.

Redo Phase

The goal of this phase is for the DBMS to repeat history to reconstruct its state up to the moment of the crash. It will reapply all updates (even aborted transactions) and redo **CLRs**.

The DBMS scans forward from log record containing smallest *reclSN* in the DPT. For each update log record or CLR with a given *LSN*, the DBMS re-applies the update unless:

- Affected page is not in the DPT, or

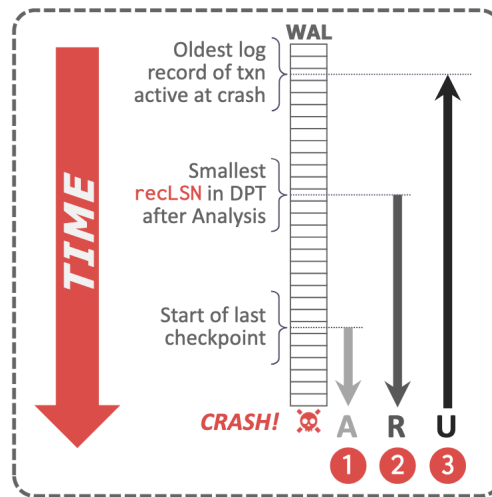


Figure 4: ARIES Recovery: The DBMS starts the recovery process by examining the log starting from the last BEGIN-CHECKPOINT found via MasterRecord. It then begins the Analysis phase by scanning forward through time to build out ATT and DPT. In the Redo phase, the algorithm jumps to the smallest recLSN, which is the oldest log record that may have modified a page not written to disk. The DBMS then applies all changes from the smallest recLSN. The Undo phase starts at the oldest log record of a transaction active at crash and reverses all changes up to that point.

- Affected page is in DPT but that log record's LSN is less than the page's recLSN, (the update was propagated to disk), or
- Affected pageLSN (on disk) $\geq$ LSN. (Note, we must fetch the page from the disk to read the page value.)

To redo an action, the DBMS re-applies the change in the log record and then sets the affected page's *pageLSN* to that log record's *LSN*. Also, there is no additional logging or forced flushes.

At the end of the redo phase, write TXN-END log records for all transactions with status COMMIT and remove them from the ATT.

Undo Phase

In the last phase, the DBMS reverses all transactions that were active at the time of crash. These are all transactions with UNDO status in the ATT after the Analysis phase.

The DBMS processes transactions in reverse *LSN* order using the lastLSN to speed up traversal. At each step, pick the largest lastLSN across all transactions in the ATT. As it reverses the updates of a transaction, the DBMS writes a CLR entry to the log for each modification.

Once the last transaction has been successfully aborted, the DBMS flushes out the log and then is ready to start processing new transactions.

Lecture #24: Introduction to Distributed Databases

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Distributed DBMSs

A distributed DBMS divides a single logical database across multiple physical resources. The application is (usually) unaware that data is split across separated hardware. The system relies on the techniques and algorithms from single-node DBMSs to support transaction processing and query execution in a distributed environment. Important goals of using a distributed DBMS are fault tolerance (avoiding a single node failure taking down the entire system) and scalability (storing data that does not fit in a single node).

The differences between **parallel** and **distributed** DBMSs are:

Parallel Database:

- Nodes are physically close to each other.
- Nodes are connected via high-speed LAN (fast, reliable communication fabric).
- The communication cost between nodes is assumed to be small. As such, one does not need to worry about nodes crashing or packets getting dropped when designing internal protocols.

Distributed Database:

- Nodes can be far from each other.
- Nodes are potentially connected via a public network, which can be slow and unreliable.
- The communication cost and connection problems cannot be ignored. Nodes can crash and packets can get dropped.

2 System Architectures

A DBMS's system architecture specifies what shared resources are directly accessible to CPUs. It affects how CPUs coordinate with each other and where they retrieve and store objects in the database.

A single-node DBMS uses what is called a *shared everything* architecture. This single node executes workers on a local CPU(s) with its own local memory and a local disk.

Shared Nothing

In a *shared nothing* environment, each node has its own CPU, memory, and disk. Nodes only communicate with each other via network. Before the rise of cloud storage platforms, the shared nothing architecture was the common way to build distributed DBMSs since it can be built using the off-the-shelf servers.

It is more difficult to increase capacity in this architecture because the DBMS has to physically move data to new nodes. It is also difficult to ensure consistency across all nodes in the DBMS, since the nodes must coordinate with each other on the state of transactions. The advantage, however, is that shared nothing DBMSs can potentially achieve better performance and are more efficient than other types of distributed DBMS architectures.

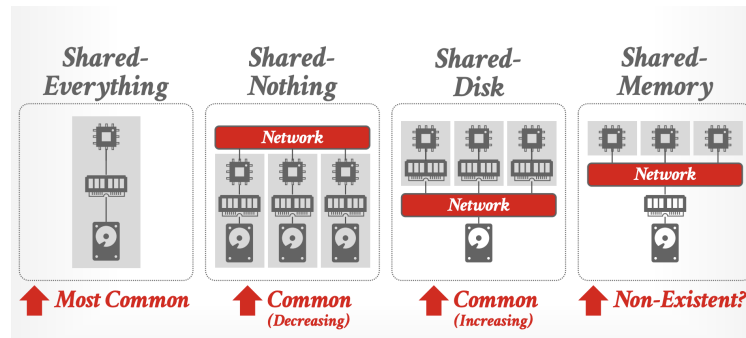


Figure 1: Database System Architectures – Four system architecture approaches ranging from sharing everything (used by non distributed systems) to sharing memory, disk, or nothing.

Shared Disk

In a *shared disk* architecture, all CPUs can read and write to a single logical disk through a network, but each have its own private memory. Each compute node usually has a small local disk to cache the data from the shared disk. This approach is more common in cloud-based DBMSs, facilitating data lakes and serverless systems.

By decoupling DBMS's storage layer from its execution layer, we can scale them independently. Adding new storage nodes or execution nodes does not affect the layout or data location in the other layer.

Nodes must send messages between them to learn about other node's current state. That is, since memory is local, if data is modified, changes must be communicated to other CPUs in the case that piece of data is in main memory for the other CPUs.

Shared Memory

In a *shared memory* architecture, CPUs have access to common memory address space via a fast interconnect. They also share the same disk. Each processor has a global view of all the in-memory data structures. In practice, most DBMSs do not use this architecture because the network as fast as CPU and memory is expensive.

3 Design Issues

Distributed DBMSs aim to maintain *data transparency*, meaning that users should not be required to know where data is physically located, or how tables are partitioned or replicated. The details of how data is being stored is hidden from the application. In other words, a SQL query that works on a single-node DBMS should work the same on a distributed DBMS.

The key design questions that distributed database systems must address are the following:

- How does the application find data?
- Where does the application send queries?
- How should queries be executed on a distributed data? Should the query be pushed to where the data is located? Or should the data be pulled into a common location to execute the query?
- How should the database be divided across resources?
- How does the DBMS ensure correctness?

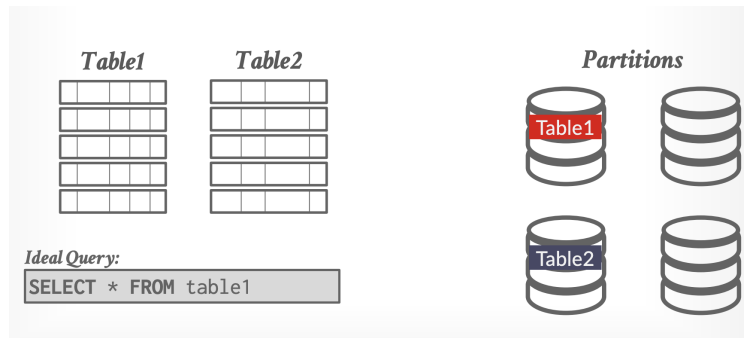


Figure 2: Naive Table Partitioning – Given two tables, place all the tuples in table one into one partition and the tuples in table two into the other.

Push vs. Pull: In the *push query to data* approach, the DBMS sends the query (or portion of it) to the node that contains the data and performs as much filtering and processing as possible where the data resides before transmitting the results over the network. In the *pull data to query* approach, the DBMS brings the needed data to the node executing the query. This may be necessary when compute resources are not available where the database files are stored.

4 Partitioning Schemes

Distributed system must partition the database across multiple resources, including disks, nodes, processors. This process is sometimes called *sharding* in NoSQL systems. When the DBMS receives a query, it first analyzes the data that the query plan needs to access. The DBMS may potentially send fragments of the query plan to different nodes, then combines the results to produce a single answer.

The goal of a partitioning scheme is to maximize single-node transactions, or transactions that only access data contained on one partition. This allows the DBMS to not need to coordinate the behavior of concurrent transactions running on other nodes. On the other hand, a distributed transaction accesses data at one or more partitions. This requires expensive, difficult coordination, discussed in the below section.

Implementation

The simplest way to partition tables is *naive data partitioning*. Each node stores one table, assuming enough storage space for a given node. This is easy to implement because a query is just routed to a specific partitioning. However, this does not scale and is suboptimal when queries join data across tables or when there is non-uniform access patterns (some nodes are more utilized than others). See Figure 2 for an example.

Another way of partitioning is *vertical partitioning*, which splits a table's attributes into separate partitions. Each partition must also store tuple information for reconstructing the original record.

A more common approach is *horizontal partitioning* that splits a table's tuples into disjoint subsets. Choose column(s) that divides the database equally in terms of size, load or usage. These keys are called the *partitioning key(s)*. The DBMS can partition a database physically (shared nothing) or logically (shared disk) based on hashing, data ranges or predicates. See Figure 3 for an example.

Logical Partitioning: A node is responsible for a set of keys, but it doesn't actually store those keys. This is commonly used in a shared disk architecture.

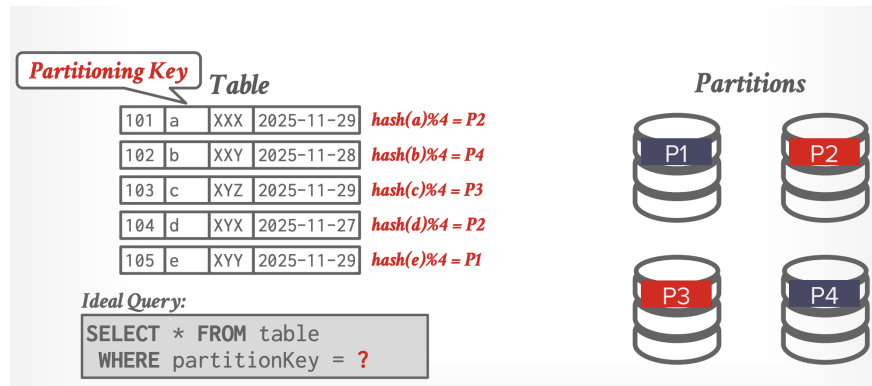


Figure 3: Horizontal Table Partitioning – Use hash partitioning to decide where to send the data. When the DBMS receives a query, it will use the table's partitioning key(s) to find out where the data is.

Physical Partitioning: A node is responsible for a set of keys, and it physically stores those keys. This is commonly used in a shared nothing architecture.

Handling Cluster Size Changes

The problem of hash partitioning is that when a node is added or removed, a lot of data has to be shuffled around. The solution is to use a hashing scheme that can add/remove partitions without having to reorganize the entire database.

Consistent Hashing: This approach assigns every node to a location on some logical ring. Then the hash of every partition key maps to a location on the ring. The node that is closest to the key in the clockwise direction is responsible for that key. See Figure 4 for an example. When a node is added or removed, keys are only moved between nodes adjacent to the new/removed node and so only $1/n$ fraction of the keys are moved. A replication factor of k means that each key is replicated at the k closest nodes in the clockwise direction.

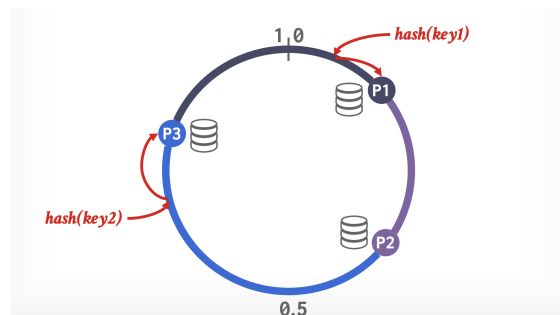


Figure 4: Consistent Hashing – All nodes are responsible for some portion of hash ring. Here, node P1 is responsible for storing *key1* and node P3 is responsible for storing *key2*.

Rendezvous Hashing: Rendezvous Hashing (also called Highest-Random Weight hashing) assigns each key to a partition by computing a hash score for every partition and choosing the one with the highest score. For a key k and partition identifier p , the DBMS computes a hash value over the concatenation $h(k||p)$; the partition whose score ranks highest *wins* the key. This avoids the ring structure required by

Consistent Hashing and provides stable mappings: when nodes join or leave, only keys whose top-ranked partition changed must be moved.

Consistent Hashing can be viewed as a specialized optimization of Rendezvous Hashing.

5 Replication

The DBMS can replicate a database across redundant nodes to increase availability.

Replica Configuration

- **Primary-Replica:** All updates go to a designated primary for each object. The primary propagates updates to its replicas by shipping logs. Read-only txns may be allowed to access replicas. If the primary fails, the system holds an election to select a new primary.
- **Multi-Primary:** Txns can update data objects at any replica. Replicas must synchronize with each other using an atomic commit protocol.

Propagation Scheme

- **Synchronous:** The primary sends updates to replicas and waits for them to acknowledge that they fully applied (i.e., logged) the changes.
- **Asynchronous:** The primary immediately returns the acknowledgement to the client without waiting for replicas to apply the changes.

Propagation Timing

- **Continuous:** The DBMS sends log messages immediately as it generates them. It still needs to send a commit or abort message later.
- **On Commit:** The DBMS only sends log messages for a transaction to replicas once the transaction commits. This avoids sending updates for transactions that later abort.

Update Method

- **Active-Active:** A transaction executes at each replica independently. Need to check that the replicas end up with the same result.
- **Active-Passive:** A transaction executes at a single location, and the changes are propagated to the replicas. This can use either physical or logical replication.

6 Federated Databases

These are distributed architectures that connect together multiple DBMSs into a single logical system. This is more popular in bigger companies. A query can access data at any location. This is hard due to different data models, query languages, and limitations of each individual DBMS. Additionally, there is no easy way to optimize queries. Lastly, there is a lot of data copying involved.

For example, say there is an application server which makes some queries. These queries then go through a middleware layer (which will convert the query into a readable format for a given DBMS used in the bigger system) that via *connectors*, will go through the multiple back-end DBMSs that are deployed in the system. The middleware will then handle the results returned from the DBMSs.

Lecture #25: Distributed Database Systems II

15-445/645 Database Systems (Spring 2026)

<https://15445.courses.cs.cmu.edu/spring2026/>

Carnegie Mellon University

Andy Pavlo

1 Distributed Concurrency Control

A *single-node transaction* accesses data that is contained in one partition and does not require inter-node coordination. In contrast, a *distributed transaction* accesses data at one or more partitions, which requires expensive coordination. There are two approaches for the coordination: *centralized* and *decentralized*.

Centralized coordinator

The centralized coordinator acts as a global “traffic cop” that coordinates all the behavior. See Figure 1 for a diagram.

Centralized coordinators can be implemented as a *middleware*, which accepts query requests and routes queries to correct partitions.

Decentralized coordinator

In a decentralized approach, nodes organize themselves. The client directly sends queries to one of the nodes. This *leader node* will send results back to the client. The leader node is in charge of communicating with other partitions in other nodes and ensuring correct commit behavior.

2 Atomic Commit Protocols

When a multi-node transaction finishes, the DBMS needs to ask all of the nodes involved whether it is safe to commit. Depending on the protocol, a majority of the nodes or all of the nodes may be needed to commit. Examples include:

- Two-Phase Commit (1970s)
- Three-Phase Commit (1983)
- Viewstamped Replication (1988)
- Paxos (1989)
- ZAB (2008? Zookeeper Atomic Broadcast protocol, **Apache Zookeeper**)
- Raft (2013)

All of these atomic commit protocols have a common structure. They usually have a notion of **Resource Managers (RMs)** that manage resources or databases (or part of a database) on different nodes. The RMs need to coordinate together to decide the fate of each transaction: *Commit* or *Abort*. Using the RMs, an atomic commit protocol needs to guarantee the following properties:

- **Stability:** Once the fate of a transaction is decided, it cannot be changed.
- **Consistency:** All the RMs end up in the same state, even after failures.
- **Liveness:** The protocol must always have some way of progressing forward (e.g., enough nodes are alive and connected for the duration of the protocol).

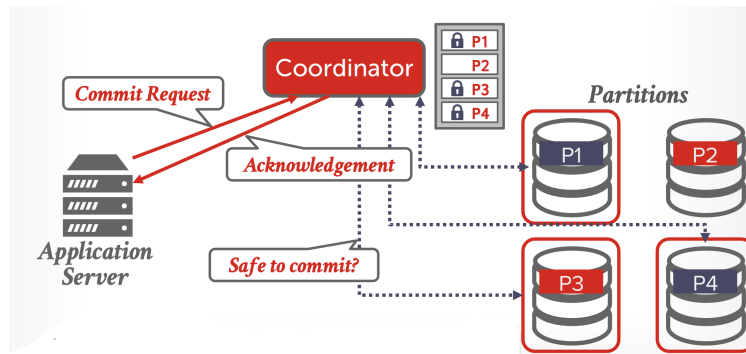


Figure 1: Centralized Coordinator – The client communicates with the coordinator to acquire locks on the partitions that the client wants to access. Once it receives an acknowledgement from the coordinator, the client sends its queries to those partitions. Once all queries for a given transaction are done, the client sends a commit request to the coordinator. The coordinator then communicates with the partitions involved in the transaction to determine whether the transaction is allowed to commit.

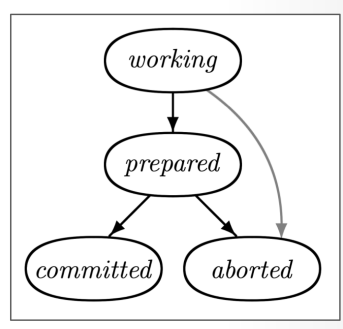


Figure 2: Atomic commit protocols can usually be modeled as state machines.

In the following sections, we will focus on Two-Phase Commit and Paxos. If the coordinator fails after the prepare message is sent, Two-Phase Commit (2PC) blocks until the coordinator recovers. On the other hand, Paxos is non-blocking if a majority of participants are alive, provided that there is a sufficiently long period without further failures. If the nodes are in the same data center, do not fail often, and are not malicious, then 2PC is often preferred over Paxos as 2PC usually results in fewer round trips.

Two-Phase Commit

The client sends a *Commit Request* to the coordinator. In the first phase of this protocol, the coordinator sends a *Prepare* message, essentially asking the participant nodes if the current transaction is allowed to commit. If a given participant verifies that the given transaction is valid, they send an *OK* to the coordinator. If the coordinator receives an *OK* from all the participants, the system can now go into the second phase in the protocol. If anyone sends an *Abort* to the coordinator, the coordinator sends an *Abort* to the client.

The coordinator sends a *Commit* to all the participants, telling those nodes to commit the transaction if all the participants have sent an *OK*. Once the participants respond with an *OK*, the coordinator can tell the client that the transaction is committed. If the transaction was aborted in the first phase, the participants receive an *Abort* from the coordinator, to which they should respond with an *OK*. Either everyone commits or no one does. The coordinator can also be a participant in the system.

Additionally, in the case of a crash, all nodes keep track of a non-volatile log of the outcomes of each phase. Two-Phase Commit is a **blocking protocol**, which means nodes block until they can figure out the next course of action. If the coordinator crashes, the participants must decide what to do. A safe option is just to abort. Alternatively, the nodes can communicate with each other to see if they can commit without the explicit permission of the coordinator. If a participant crashes, the coordinator assumes that it responded with an abort if it has not sent an acknowledgement yet.

Optimizations:

- *Early Prepare Voting* – If the DBMS sends a query to a remote node that it knows will be the last one executed there, then that node will also return their vote for the prepare phase with the query result.
- *Early Acknowledgement after Prepare* – If all nodes vote to commit a transaction, the coordinator can send the client an acknowledgement that their transaction was successful before the commit phase finishes.

Paxos

Paxos (along with Raft) is more prevalent in modern systems than 2PC. 2PC is a degenerate case of Paxos; Paxos uses $2F + 1$ coordinators and makes progress as long as at least $F + 1$ of them are working properly, 2PC sets $F = 0$.

Paxos is a consensus protocol where a coordinator proposes an outcome (e.g., commit or abort) and then the participants vote on whether that outcome should succeed. This protocol does not block if a majority of participants are available and has provably minimal message delays in the best case. For Paxos, the coordinator is called the **proposer** and participants are called **acceptors**.

The client will send a *Commit Request* to the proposer. The proposer will send a *Propose* to the other nodes in the system, or the acceptors. A given acceptor will send an *Agree* if they have not already sent an *Agree* on a higher logical timestamp. Otherwise, they send a *Reject*.

Once the majority of the acceptors have sent an *Agree*, the proposer will send a *Commit*. The proposer must wait to receive an *Accept* from a majority of acceptors before sending the final message to the client saying that the transaction is committed, unlike 2PC.

Use exponential backoff times when trying to propose again after a failed proposal, to avoid dueling proposers.

Multi-Paxos: If the system elects a single leader that oversees proposing changes for some period, then it can skip the propose phase. The system periodically renews who the leader is using another Paxos round. When there is a failure, the DBMS can fall back to full Paxos.

3 CAP Theorem

The *CAP Theorem*, proposed by Eric Brewer and later proved in 2002 at MIT, explained that it is impossible for a distributed system to always be Consistent, Available, and Partition Tolerant. Only two of these three properties can be chosen.

Consistency is synonymous with linearizability for operations on all nodes. Once a write completes, all future reads should return the value of that write applied or a later write applied. Additionally, once a read has been returned, future reads should return that value or the value of a later applied write. NoSQL systems compromise this property in favor of the latter two. Other systems will favor this property and one of the latter two.

Availability is the concept that all nodes that are up can satisfy all requests.

Partition tolerance means that the system can still operate correctly despite some message loss between nodes that are trying to reach consensus on values. If consistency and partition tolerance are chosen for a system, updates will not be allowed until a majority of nodes are reconnected, typically done in traditional or NewSQL DBMSs.

There is a modern version that considers consistency vs. latency trade-offs: *PACELC Theorem*. In case of network partitioning (P) in a distributed system, one has to choose between availability (A) and consistency (C), else (E), even when the system runs normally without network partitions, one has to choose between latency (L) and consistency (C).

4 Distributed Join Algorithms

For analytical workloads, the majority of time is spent computing joins and reading from disk, so optimizing joins is important. The efficiency of a distributed join depends on the target tables' partitioning schemes.

One approach is to put entire tables on a single node and then perform the join there. However, the DBMS loses the parallelism of a distributed DBMS, defeating the purpose of making it distributed. This option also entails costly data transfer over the network.

To join tables R and S , the DBMS needs to get the proper tuples on the same node. Once there, it executes the same join algorithms discussed earlier in the semester. One should always send the minimal amount needed to compute the join.

There are four scenarios for distributed join algorithms.

Scenario 1

One of the tables is replicated at every node and the other table is partitioned across nodes. Each node joins its local data in parallel and then sends its results to a coordinating node.

Scenario 2

Both tables are partitioned on the join attribute, with IDs matching on each node. Each node performs the join on local data and then sends them to a node for coalescing.

Scenario 3

Both tables are partitioned on different keys. If one of the tables is small, then the DBMS broadcasts that table to all nodes. This takes us back to Scenario 1. Local joins are computed and then those joins are sent to a common node to perform the final join. This is known as a *broadcast join*.

Scenario 4

This is the worst-case scenario. Both tables are not partitioned on the join key. The DBMS partitions the tables across nodes via the shuffle operator such that nodes have data matching on the join ID. This recovers Scenario 2. Local joins are computed and then the results are sent to a common node for the final join. If there isn't enough disk space, a failure is unavoidable. This is called a *shuffle join*.

Semi-Join Optimization

Before pulling data from another node, a *semi-join filter* can be used to reduce data movement. This filter blocks rows from the data that will be discarded in the result anyway due to a predicate. An approximate filter like a Bloom filter can be used here.

Shuffle Operation

In addition to being useful in joins, a shuffle operation is generally useful. It can be used to rebalance data based on observed characteristics and adjust the capacity allocated to a subcomputation. The shuffle operator is basically the repartition type of exchange operator discussed previously in lecture.